TTM4100 – Communication, Services and Networks

NTNU – Norwegian University of Science and Technology

Study Guide

Chapters 1 – 9

Computer Networking: A Top-Down Approach, 8th ed.
Kurose & Ross

Markus Selander

Spring 2026

Contents

Chapter 1: Introduction . . . **8**

1 What is the Internet? . . . **9**

1.1 "Nuts and Bolts" – The Component Perspective . . . 9

1.2 "Service" – The Service Perspective . . . 9

2 What is a protocol? . . . **9**

3 The Network Edge . . . **10**

3.1 Hosts: Clients and Servers . . . 10

3.2 Access Networks . . . 10

3.3 Home Network . . . 10

3.4 Hosts send packets . . . 11

3.5 Physical Media . . . 11

4 The Network Core . . . **11**

4.1 Packet Switching . . . 11

4.2 Circuit Switching . . . 12

4.3 Internet Structure . . . 12

5 Performance: Delay, Loss, and Throughput . . . **12**

5.1 Four types of delay . . . 13

5.2 Traffic intensity and queuing delay . . . 13

5.3 Packet Loss . . . 14

5.4 Throughput . . . 14

5.5 Traceroute . . . 14

6 Security . . . **14**

7 Protocol Layers and Encapsulation . . . **15**

7.1 The Internet's 5-layer protocol stack . . . 15

7.2 Encapsulation . . . 15

8 Exam Questions – Chapter 1 . . . **15**

8.1 1.1 Nuts and bolts vs. Service view . . . 16

8.2 1.2 Network Edge . . . 16

8.3 1.3 Network Core . . . 17

8.4 1.4 Performance . . . 17

8.5 1.5 Layering and 1.6 Security . . . 18

9 Summary – Key Formulas . . . **18**

Chapter 2: The Application Layer . . . **19**

10 Principles of Network Applications (2.1) . . . **20**

10.1 Application Architectures . . . 20

10.2 Processes and Sockets . . . 20

10.3 Addressing Processes . . . 21

10.4 Application-Layer Protocols . . . 21

10.5 Transport Service Requirements . . . 21

11 Web and HTTP (2.2) . . . **23**

11.1 HTTP Basics . . . 23

11.2 Persistent vs. Non-persistent HTTP . . . 23

11.3 HTTP Messages . . . 23
11.4 Cookies . . . 24
11.5 Web Caching (Proxy Server) . . . 24
11.6 HTTP/2 and HTTP/3 . . . 25

12 Email: SMTP and IMAP (2.3) . . . 27
12.1 Components of the Email System . . . 27
12.2 The SMTP Protocol . . . 27
12.3 Email Format and IMAP . . . 27

13 DNS – Domain Name System (2.4) . . . 29
13.1 What is DNS? . . . 29
13.2 The DNS Hierarchy . . . 29
13.3 DNS Queries: Iterative vs. Recursive . . . 30
13.4 DNS Caching and Records . . . 30

14 P2P File Distribution (2.5) . . . 32
14.1 Client-Server vs. P2P Distribution . . . 32

15 Video Streaming and CDN (2.6) . . . 34
15.1 Multimedia: Video . . . 34
15.2 Streaming of Stored Video . . . 34
15.3 DASH – Dynamic Adaptive Streaming over HTTP . . . 34
15.4 CDN – Content Distribution Networks . . . 35

16 Socket Programming (2.7) . . . 37
16.1 Socket Basics . . . 37
16.2 Socket Programming with UDP . . . 37
16.3 Socket Programming with TCP . . . 38
16.4 Key Differences: UDP vs. TCP Sockets . . . 39

17 Summary – Chapter 2 . . . 40

Chapter 3: The Transport Layer . . . 41

18 Transport Layer Services and Protocols (3.1) . . . 42
18.1 The Role of the Transport Layer . . . 42
18.2 Transport Layer vs. Network Layer . . . 42
18.3 Internet Transport Protocols . . . 42

19 Multiplexing and Demultiplexing (3.2) . . . 43
19.1 Basic Concept . . . 43
19.2 How Demultiplexing Works . . . 43
19.3 Connectionless Demultiplexing (UDP) . . . 44
19.4 Connection-Oriented Demultiplexing (TCP) . . . 44

20 Connectionless Transport: UDP (3.3) . . . 44
20.1 UDP – Overview . . . 44
20.2 Use Cases for UDP . . . 44
20.3 UDP Segment Format . . . 45
20.4 UDP Checksum . . . 45

21 Principles of Reliable Data Transfer (3.4) . . . 45
21.1 Problem Statement . . . 45
21.2 rdt1.0: Transfer over a Perfectly Reliable Channel . . . 46
21.3 rdt2.0: Channel with Bit Errors . . . 46
21.4 rdt2.1: Sequence Numbers . . . 46

21.5 rdt2.2: NAK-Free Protocol . . . 47
21.6 rdt3.0: Channel with Bit Errors AND Packet Loss . . . 47
21.7 Performance of rdt3.0 (Stop-and-Wait) . . . 47
21.8 Pipelining: Increased Utilization . . . 48
21.9 Go-Back-N (GBN) . . . 48
21.10 Selective Repeat (SR) . . . 48

22 Connection-Oriented Transport: TCP (3.5) . . . 49
22.1 TCP – Overview . . . 49
22.2 TCP Segment Structure . . . 50
22.3 MSS and MTU . . . 50
22.4 TCP Sequence Numbers and ACK . . . 50
22.5 TCP Round-Trip Time and Timeout . . . 51
22.6 TCP Reliable Data Transfer . . . 51
22.7 TCP Flow Control . . . 52
22.8 TCP Connection Management . . . 52

23 Principles of Congestion Control (3.6) . . . 53
23.1 What is Congestion? . . . 53
23.2 Costs of Congestion . . . 54
23.3 Approaches to Congestion Control . . . 54

24 TCP Congestion Control (3.7) . . . 54
24.1 Overview: CWND and RWND . . . 54
24.2 TCP Slow Start . . . 55
24.3 Congestion Avoidance – AIMD . . . 55
24.4 Loss Events and ssthresh . . . 56
24.5 The Three Mechanisms in TCP Congestion Control . . . 56
24.6 TCP Tahoe vs. TCP Reno . . . 57
24.7 TCP CUBIC . . . 57
24.8 TCP Fairness . . . 57
24.9 The Bottleneck Link . . . 58

25 Evolution of Transport Layer Functionality (3.8) . . . 58
25.1 Challenges with TCP . . . 58
25.2 QUIC – Quick UDP Internet Connections . . . 58
25.3 Historical Development of TCP . . . 59

26 Summary . . . 59

Chapter 4: The Network Layer – Data Plane . . . 61

27 The Network Layer – Overview (4.1) . . . 62
27.1 The Role of the Network Layer . . . 62
27.2 Forwarding vs. Routing . . . 62
27.3 The Data Plane vs. The Control Plane . . . 62
27.4 Per-router Control Plane vs. SDN . . . 63
27.5 The Network Service Model . . . 63

28 What is Inside a Router? (4.2) . . . 63
28.1 Router Architecture – Overview . . . 63
28.2 Input Port Functions . . . 64
28.3 Longest Prefix Match . . . 64
28.4 Switching Fabric . . . 65

28.5 Queues and Delays in the Router . . . 65
28.6 Buffer Management . . . 66
28.7 Packet Scheduling . . . 67
29 Internet Protocol (IP) (4.3) . . . 67
29.1 IPv4 Datagram Format . . . 67
29.2 IP Addressing . . . 68
29.3 Subnets . . . 68
29.4 IP Addresses: Binary Conversion . . . 69
29.5 Subnet Allocation from a CIDR Block . . . 69
29.6 Hierarchical Addressing and Route Aggregation . . . 69
29.7 How Does a Host Get Its IP Address? . . . 70
29.8 DHCP – Dynamic Host Configuration Protocol . . . 70
29.9 NAT – Network Address Translation . . . 71
29.10 IPv6 . . . 71
29.11 Transition from IPv4 to IPv6: Tunneling . . . 72
30 Architectural Principles (4.5) . . . 72
30.1 The IP Hourglass Architecture . . . 72
30.2 Middleboxes . . . 73
30.3 The End-to-End Argument . . . 73
31 IP Addressing: ICANN and Address Allocation . . . 73
32 Summary and Exam Tips . . . 74
Chapter 5: Network Layer – Control Plane . . . 75
33 Introduction to the Control Plane (5.1) . . . 76
33.1 Network-Layer Functions Recap . . . 76
33.2 Two Approaches to the Control Plane . . . 76
34 Routing Algorithms (5.2) . . . 76
34.1 Goal and Overview . . . 76
34.2 Graph Abstraction . . . 77
34.3 Routing Algorithm Classification . . . 77
35 Link-State Routing Algorithm: Dijkstra's Algorithm (5.2.1) . . . 77
35.1 Overview . . . 77
35.2 Dijkstra's Algorithm . . . 78
35.3 Oscillations in Link-State Routing . . . 78
36 Distance Vector Routing Algorithm (5.2.2) . . . 79
36.1 Bellman-Ford Equation . . . 79
36.2 Distance Vector Algorithm . . . 79
36.3 Count-to-Infinity Problem . . . 80
37 Comparison: Link State vs. Distance Vector (5.2.3) . . . 80
38 Intra-AS Routing: OSPF (5.3) . . . 80
38.1 Autonomous Systems and Scalability . . . 81
38.2 OSPF – Open Shortest Path First . . . 81
38.3 Hierarchical OSPF . . . 82
39 Inter-AS Routing: BGP (5.4) . . . 82
39.1 BGP Overview . . . 82
39.2 BGP Basics: Path Attributes and Routes . . . 82

39.3 BGP Route Selection . . . 83
39.4 Why Different Intra-AS and Inter-AS Routing? . . . 83
40 ICMP – Internet Control Message Protocol (5.6) . . . 83
40.1 ICMP Overview . . . 84
40.2 ICMP Message Types . . . 84
40.3 Traceroute and ICMP . . . 84
41 SDN – Software-Defined Networking (5.5) . . . 85
41.1 SDN Overview . . . 85
41.2 SDN Architecture . . . 85
41.3 OpenFlow Protocol . . . 86
42 Network Management and SNMP (5.7) . . . 86
42.1 Network Management Overview . . . 86
42.2 SNMP – Simple Network Management Protocol . . . 86
43 Summary and Exam Tips . . . 87
Chapter 6: The Link Layer and LANs . . . 89
44 Introduction to the Link Layer (6.1) . . . 90
44.1 Terminology and Context . . . 90
44.2 Link Layer Services . . . 90
44.3 Where is the Link Layer Implemented? . . . 90
45 Error Detection and Correction (6.2) . . . 91
45.1 Overview . . . 91
45.2 Parity Checking . . . 91
45.3 Internet Checksum . . . 92
45.4 Cyclic Redundancy Check (CRC) . . . 92
46 Multiple Access Protocols (6.3) . . . 93
46.1 The Multiple Access Problem . . . 93
46.2 Taxonomy of MAC Protocols . . . 93
46.3 Channel Partitioning Protocols . . . 94
46.4 Random Access Protocols . . . 94
46.5 Taking Turns Protocols . . . 96
46.6 Cable Access Network: DOCSIS . . . 97
47 LANs – Addressing and ARP (6.4) . . . 97
47.1 MAC Addresses . . . 97
47.2 ARP – Address Resolution Protocol . . . 97
47.3 Routing to Another Subnet . . . 98
48 Ethernet (6.4) . . . 98
48.1 Ethernet Overview . . . 99
48.2 Ethernet Frame Structure . . . 99
48.3 Ethernet Properties . . . 99
48.4 Ethernet Topologies . . . 99
48.5 802.3 Ethernet Standards . . . 100
49 Link-Layer Switches (6.4) . . . 100
49.1 Switch Basics . . . 100
49.2 Switch Self-Learning . . . 100
49.3 Switch Forwarding . . . 101
49.4 Switches vs. Routers . . . 101

50 A Day in the Life of a Web Request (6.7) . . . 101
51 Summary of Key Formulas and Facts . . . 102
52 Additional Exam Questions from Past Papers . . . 103
Chapter 7: Wireless and Mobile Networks . . . 105
53 Introduction to Wireless Networks (7.1) . . . 106
53.1 Elements of a Wireless Network . . . 106
53.2 Infrastructure Mode vs. Ad Hoc Mode . . . 106
53.3 Wireless Network Taxonomy . . . 106
54 Wireless Link Characteristics (7.2) . . . 107
54.1 Challenges of the Wireless Channel . . . 107
54.2 SNR vs. BER – The Fundamental Tradeoff . . . 107
54.3 Hidden Terminal Problem . . . 109
55 IEEE 802.11 – WiFi Wireless LANs (7.3) . . . 109
55.1 802.11 Standards Overview . . . 109
55.2 BSS Architecture and Association . . . 109
55.3 CSMA/CA – Multiple Access in 802.11 . . . 110
55.4 RTS/CTS Mechanism . . . 111
55.5 802.11 Frame Structure . . . 113
55.6 Mobility Within the Same Subnet . . . 114
55.7 Rate Adaptation and Power Management . . . 114
56 Cellular Networks – 4G and 5G (7.4) . . . 114
56.1 4G LTE Architecture . . . 115
56.2 LTE Data Plane – Tunneling . . . 115
56.3 LTE Link Layer . . . 116
56.4 BS Association and Sleep Modes . . . 116
56.5 4G vs. 5G . . . 117
57 Putting It All Together – Wireless Summary . . . 118
Chapter 8: Security . . . 120
58 What Is Network Security? . . . 121
58.1 The Language of Cryptography . . . 121
59 Symmetric Key Cryptography . . . 121
59.1 Substitution Ciphers . . . 121
59.2 DES – Data Encryption Standard . . . 122
59.3 AES – Advanced Encryption Standard . . . 122
60 Public Key Cryptography . . . 122
60.1 RSA – Rivest, Shamir, Adleman . . . 123
61 Authentication and Message Integrity . . . 123
61.1 Message Integrity and Hash Functions . . . 123
61.2 Message Authentication Code – HMAC . . . 124
61.3 Digital Signatures . . . 124
61.4 Certification Authorities – CAs . . . 124
62 Secure E-mail . . . 125
63 TLS – Transport Layer Security . . . 125
63.1 TLS Handshake . . . 126
63.2 TLS Records and TLS 1.3 . . . 126

64 Network Layer Security – IPsec and VPNs . . . **126**
64.1 VPN – Virtual Private Network . . . 127
64.2 IPsec . . . 127
65 Security in Wireless and Mobile Networks . . . **127**
65.1 802.11 WiFi Security – WPA2/WPA3 . . . 127
65.2 4G LTE Authentication and Encryption . . . 128
66 Operational Security – Firewalls . . . **128**
66.1 Three Goals of Firewalls . . . 129
66.2 Stateless Packet Filtering . . . 129
66.3 Stateful Packet Filtering . . . 130
66.4 Application Gateways . . . 130
66.5 Limitations of Firewalls and Gateways . . . 130
67 Summary – Where Security Is Applied . . . **130**
68 Exam Questions . . . **131**
Chapter 9: Multimedia Networking . . . **135**
69 9.1 – Multimedia Applications and Properties . . . **136**
70 9.2 – Streaming Stored Video . . . **137**
71 9.3 – Voice-over-IP – VoIP . . . **138**
72 9.4.1 – RTP – Real-Time Transport Protocol . . . **140**
73 Summary – Key Concepts . . . **142**
74 Exam Questions . . . **142**

TTM4100 – KTN

Chapter 1: Introduction

Computer Networking: A Top-Down Approach, 8th ed.
Kurose & Ross – NTNU Spring 2026

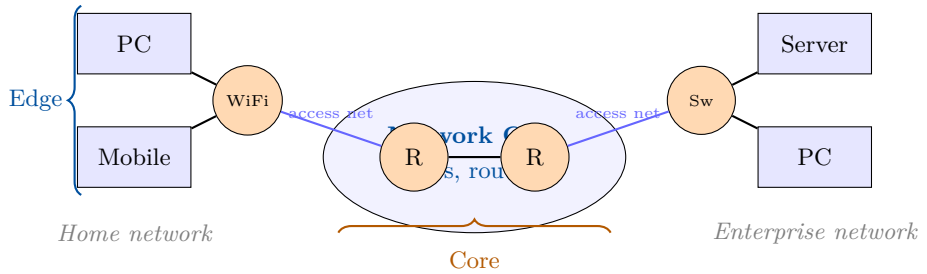
1 What is the Internet?

The Internet can be described from two perspectives: **"nuts and bolts"** (the components) and **"service"** (the services).

1.1 "Nuts and Bolts" – The Component Perspective

The Internet is a "network of networks"

The Internet consists of billions of connected devices that communicate via **packet switches** (routers and switches) over **communication links** (fiber, copper, radio, satellite).



Key components:

- **Hosts / end systems** – devices that run network applications (PCs, mobiles, servers, IoT)
- **Packet switches** – forward packets: *routers* (network layer) and *switches* (link layer)
- **Communication links** – fiber, copper, radio, satellite. Transmission rate = *bandwidth*
- **Networks** – collection of devices, routers, and links managed by an organization
- **ISP** (Internet Service Provider) – provides access to the Internet

Internet Standards

- **RFC** (Request for Comments) – documents that describe the standards
- **IETF** (Internet Engineering Task Force) – develops the standards

1.2 "Service" – The Service Perspective

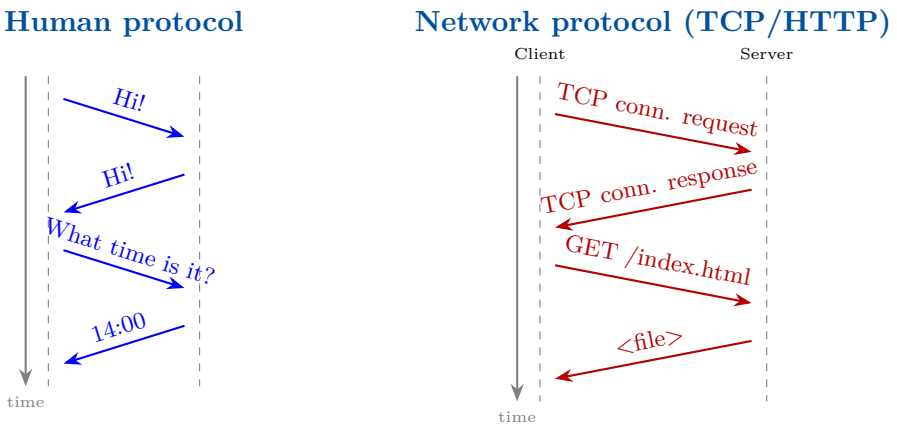
The Internet is an **infrastructure that delivers services to applications**: web, video, e-mail, games, e-commerce, social media, IoT, etc.

The Internet offers a **programming interface** (socket API) to distributed applications – "hooks" that allow apps to connect and use transport services.

2 What is a protocol?

Definition: Protocol

A **protocol** defines the **format**, the **order** of messages sent and received between network devices, and the **actions** taken when messages are sent or received.



All communication activity on the Internet is governed by protocols.

3 The Network Edge

3.1 Hosts: Clients and Servers

- **Hosts** (end systems) reside at the network "edge"
- Two types: **clients** (request services) and **servers** (provide services)
- Servers are often in **data centers**

3.2 Access Networks

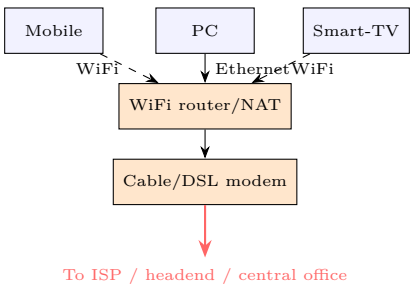
The access network connects end systems to the first router ("edge router").

DSL Telephone line Down: 24–52 Mbps Up: 3.5–16 Mbps **Dedicated**	**Cable (HFC)** TV cable Down: 40 Mbps–1.2 Gbps Up: 30–100 Mbps **Shared**	**Fiber (FTTH)** Fiber optic Up to 10 Gbps **Dedicated**
WiFi (WLAN) 802.11b/g/n 11, 54, 450 Mbps Range: ∼30m	**Mobile network** 4G/5G 10–100+ Mbps Range: ∼10 km	**Enterprise** Ethernet + WiFi 100 Mbps – 10 Gbps

Shared vs. dedicated access

- **DSL**: Dedicated line to the central office (DSLAM) – no sharing with neighbors
- **Cable**: Shared medium – everyone in the neighborhood shares the bandwidth to the "cable headend"

3.3 Home Network



3.4 Hosts send packets

Transmission Delay

A host divides application messages into **packets** of length L bits and sends them at transmission rate R bits/sec:

$$d_{\text{trans}} = \frac{L}{R}$$

3.5 Physical Media

Type	Category	Characteristics
Twisted pair (TP)	Guided	Two insulated copper conductors, Cat6: 10 Gbps
Coaxial cable	Guided	Two concentric conductors, broadband
Fiber optic	Guided	Light pulses in glass, high speed, low error
Wireless radio	Unguided	EM waves, affected by reflection/noise

- **Guided media**: Signal follows physical medium (copper, fiber, coax)
- **Unguided media**: Signal travels freely (radio, WiFi, satellite)
- Radio: can be **half-duplex** (one direction at a time)

4 The Network Core

The network core is a network of interconnected routers – a "network of networks".

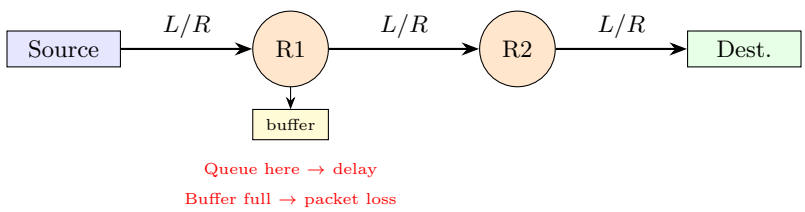
4.1 Packet Switching

Store-and-forward

The entire packet must arrive at a router before it can be forwarded to the next link.
Delay for one packet over one link: $d = L/R$

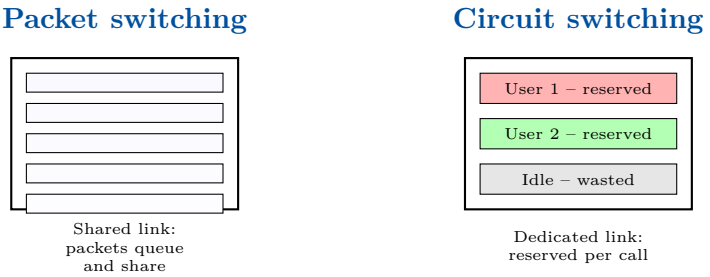
Two key functions:

1. **Forwarding**: Moving a packet from an input link to the correct output link
2. **Routing**: Determining the route the packet should take from source to destination



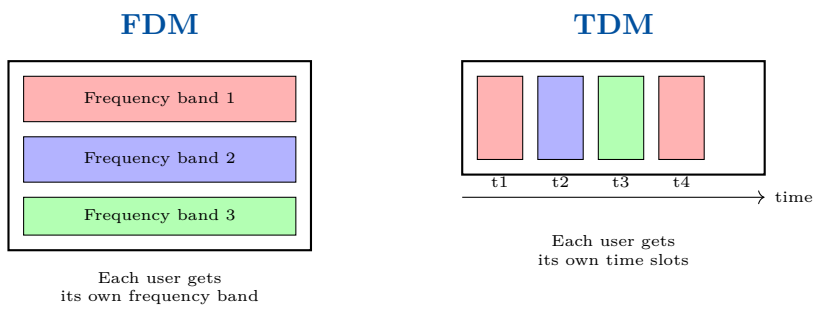
Queuing and packet loss: If packets arrive faster than they can be forwarded, they are placed in a **buffer** (queue). If the buffer is full, packets are **dropped** (lost).

4.2 Circuit Switching

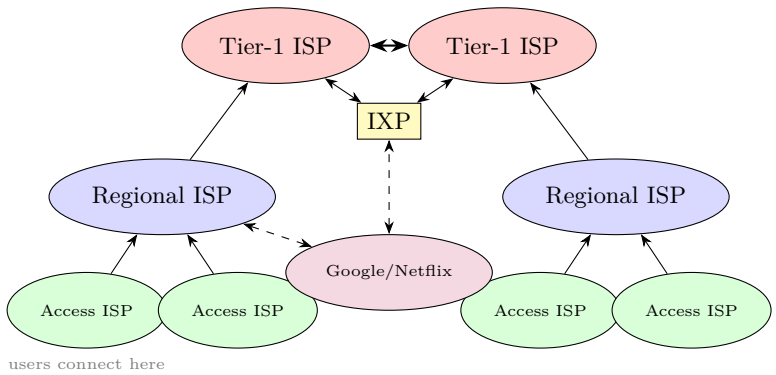


	Packet switching	Circuit switching
Resource sharing	Yes (statistical multiplexing)	No (reserved)
Guarantee	None	Yes (fixed bandwidth)
Efficiency	High (utilizes idle capacity)	Low (idle = wasted)
Queuing/loss	Yes	No
Best for	Bursty traffic (web, e-mail)	Constant traffic (telephone)

4.2.1 FDM and TDM



4.3 Internet Structure



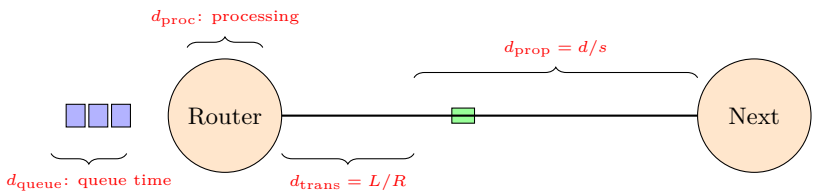
- **Access ISP**: Provides users with access (Telenor, Altibox, etc.)
- **Regional ISP**: Connects access ISPs together
- **Tier-1 ISP**: Large, global networks (e.g. Telenor, AT&T)
- **IXP** (Internet Exchange Point): Connects ISPs directly
- **Content provider networks** (Google, Netflix): Build their own networks

5 Performance: Delay, Loss, and Throughput

5.1 Four types of delay

Total nodal delay

$$d_{\text{nodal}} = d_{\text{proc}} + d_{\text{queue}} + d_{\text{trans}} + d_{\text{prop}}$$



Delay	Formula	Depends on	Typical
d_{proc} – Processing	–	Bit error check, choosing link	< 1 ms
d_{queue} – Queuing	–	Traffic intensity	varies
d_{trans} – Transmission	L/R	Packet size, link rate	μs–ms
d_{prop} – Propagation	d/s	Distance, signal speed	μs–ms

Important: Transmission delay $\neq$ Propagation delay!

- $d_{\text{trans}} = L/R$ – time to **push all bits out onto the link**
- $d_{\text{prop}} = d/s$ – time for the signal to **travel through the link**
- $s \approx 2 \times 10^8$ m/s in fiber (speed of light / refractive index 1.47)

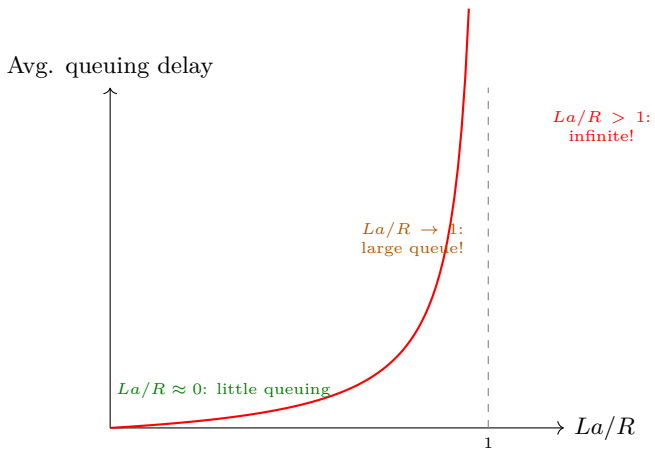
Example: Trondheim–Oslo (540 km), fiber:
$d_{\text{prop}} = 540\,000/200\,000\,000 = 2{,}7$ ms
Packet 1500 bytes, link 1 Gbps:
$d_{\text{trans}} = 12\,000/1\,000\,000\,000 = 0{,}012$ ms

5.2 Traffic intensity and queuing delay

Traffic Intensity

$$\text{Traffic intensity} = \frac{L \cdot a}{R}$$

where L = packet size (bits), a = average arrival rate (packets/s), R = link rate (bps).



5.3 Packet Loss

When a packet arrives at a router with a **full buffer**, the packet is **dropped** (lost). Lost packets may be retransmitted by the previous node, the source end system, or not at all.

5.4 Throughput

Throughput and bottleneck

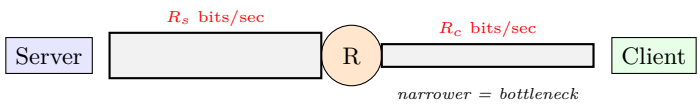
Throughput: Rate (bits/time unit) at which bits are actually transferred from sender to receiver.

- **Instantaneous throughput**: Rate at a given point in time
- **Average throughput**: Rate over a longer period
- **Bottleneck link**: The link with the lowest capacity determines end-to-end throughput

$$\text{Throughput} = \min(R_s, R_c)$$

With 10 simultaneous connections sharing a backbone R:

$$\text{Per-connection throughput} = \min(R_c, R_s, R/10)$$



5.5 Traceroute

Traceroute measures delay along the entire route from source to destination:

- Sends 3 packets to each router i along the way (with TTL $= i$)
- Router i returns the packet, and the sender measures the time
- Also shows packet loss ("* * *" = no response)

6 Security

The Internet was originally designed **without** much security – "a group of mutually trusting users."

Threat	Description
Malware	Viruses (require user action), worms (self-replicating), spyware, botnets
DoS/DDoS	Overwhelms resources with fake traffic from many compromised machines
Packet sniffing	Reads all packets on a shared medium (WiFi, Ethernet) with promiscuous NIC
IP spoofing	Sends packets with a fake source address

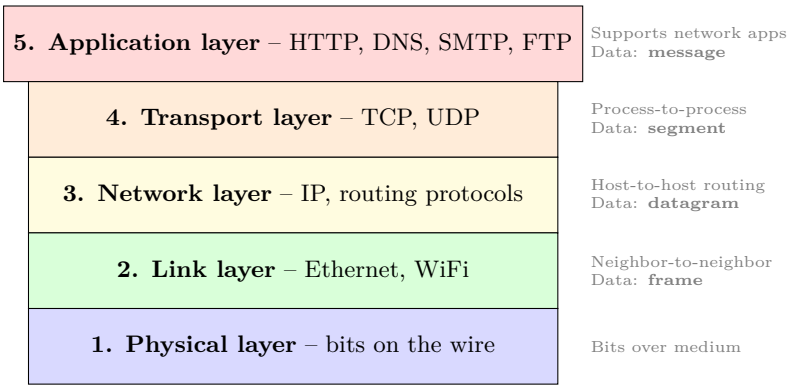
Defenses

- **Authentication** – protects against spoofing
- **Encryption** – protects against sniffing (confidentiality)

- **Digital signatures** – ensures integrity
- **Firewalls** – blocks unauthorized traffic
- **VPN** – restricts access to resources

7 Protocol Layers and Encapsulation

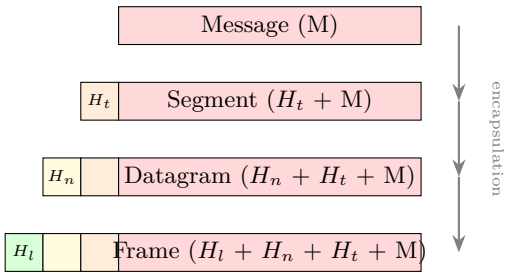
7.1 The Internet's 5-layer protocol stack



The ISO/OSI model has 7 layers

OSI adds **Presentation** and **Session** between application and transport. These are in practice included in the application layer in the Internet model.

7.2 Encapsulation



Each layer adds its own **header** (H) around the data from the layer above. The receiver removes headers in reverse order.

Why layering?

- Provides **clear structure** – easier to design and maintain
- **Modularization** – can change one layer without affecting others
- Separates complex systems into manageable parts

8 Exam Questions – Chapter 1

Here are the most important questions from the curriculum, organized by topic.

8.1 1.1 Nuts and bolts vs. Service view

Exam questions

Question	Answer
What is the Internet according to "nuts and bolts"?	A global network connecting billions of end systems.
What is an ISP?	An Internet service provider that gives end systems access to the Internet.
Which protocols govern the Internet?	TCP/IP.
Who develops Internet standards?	IETF (Internet Engineering Task Force).
What are the documents that describe the standards called?	RFCs (Requests for Comments).
What is the Internet according to "services view"?	A platform that delivers services to applications.
What is a protocol?	A set of rules that defines the format, order, and actions for messages between communicating devices.

8.2 1.2 Network Edge

Exam questions

Question	Answer
What two types of hosts exist?	Clients and servers.
What does DSL stand for?	Digital Subscriber Line.
Is DSL symmetric or asymmetric?	Asymmetric (higher download speed).
What is HFC?	Hybrid Fiber Coax – cable network with fiber + coax.
What is the difference between shared and dedicated access?	DSL is dedicated, cable is shared.
What is the formula for packet transmission delay?	$D = L/R$.
What is guided vs. unguided media?	Guided: signal in physical medium (copper, fiber). Unguided: signal freely (radio).
What are the advantages of fiber?	High speed, low error rate.

8.3 1.3 Network Core

Exam questions

Question	Answer
What is "store-and-forward"?	The entire packet must arrive before it can be forwarded.
What happens when the buffer is full?	Packets are dropped (lost).
What is the difference between packet switching and circuit switching?	Circuit switching reserves a fixed connection; packet switching sends data in small pieces that share the line.
What is FDM?	Frequency Division Multiplexing – each user gets its own frequency band.
What is TDM?	Time Division Multiplexing – each user gets its own time slots.
When is packet switching best?	For bursty traffic – sometimes active, sometimes not.
What is a tier-1 ISP?	Large, global networks (e.g. Telenor, AT&T).
What is an IXP?	Internet Exchange Point – connects ISPs directly.

8.4 1.4 Performance

Exam questions

Question	Answer
What are the four types of delay?	Processing, queueing, transmission (L/R), propagation (d/s).
What is traffic intensity?	$(L \times a)/R$ – the ratio of arrival rate to service rate.
What happens when $La/R > 1$?	More work arrives than can be handled; delay $\to \infty$.
What is throughput?	Rate at which bits are actually transferred from sender to receiver.
What is the bottleneck link?	The link with the lowest capacity – determines end-to-end throughput.

8.5 1.5 Layering and 1.6 Security

Exam questions

Question	Answer
How many layers are in the Internet protocol stack?	Five: application, transport, network, link, physical.
What is encapsulation?	Wrapping data with headers from each layer.
What is packet sniffing?	Reading all packets on a shared network.
What is IP spoofing?	Sending packets with a fake source address.
What is a DoS attack?	Making resources unavailable by overwhelming them with fake traffic.
What protects against sniffing?	Encryption.
What protects against spoofing?	Authentication.

9 Summary – Key Formulas

Formulas you must know

$$d_{\text{trans}} = \frac{L}{R}$$ Transmission delay (packet size / link rate) (1)

$$d_{\text{prop}} = \frac{d}{s}$$ Propagation delay (distance / signal speed) (2)

$$d_{\text{nodal}} = d_{\text{proc}} + d_{\text{queue}} + d_{\text{trans}} + d_{\text{prop}}$$ Total delay per node (3)

$$\text{Traffic intensity} = \frac{L \cdot a}{R}$$ Arrival rate vs. service rate (4)

$$\text{Throughput} = \min(R_s, R_c)$$ Determined by the bottleneck (5)

Values to remember:

- Signal speed in fiber: $s \approx 2 \times 10^8$ m/s
- Speed of light: $c = 3 \times 10^8$ m/s (fiber has refractive index $\approx 1{,}47$)
- 1 byte = 8 bits

TTM4100 – KTN

Chapter 2: The Application Layer

Computer Networking: A Top-Down Approach, 8th ed.

Kurose & Ross – NTNU Spring 2026

10 Principles of Network Applications (2.1)

10.1 Application Architectures

Two dominant architectures for network applications:

Client-Server vs. Peer-to-Peer (P2P)

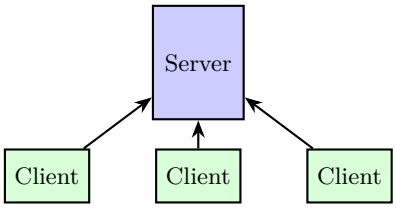
Client-Server:

- The server is **always on** with a fixed (permanent) IP address
- Clients contact the server, can have dynamic IP
- Clients do **not communicate directly** with each other
- Examples: HTTP, IMAP, FTP
- Scaling: multiple servers in data centers

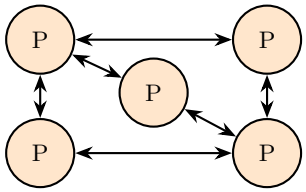
Peer-to-Peer (P2P):

- **No** dedicated server – end systems communicate directly
- Peers provide services to each other
- **Self-scaling**: new peers bring both capacity and demand
- Challenge: peers are intermittently connected, change IP
- Examples: BitTorrent, Skype, KanKan

Client-Server



Peer-to-Peer



10.2 Processes and Sockets

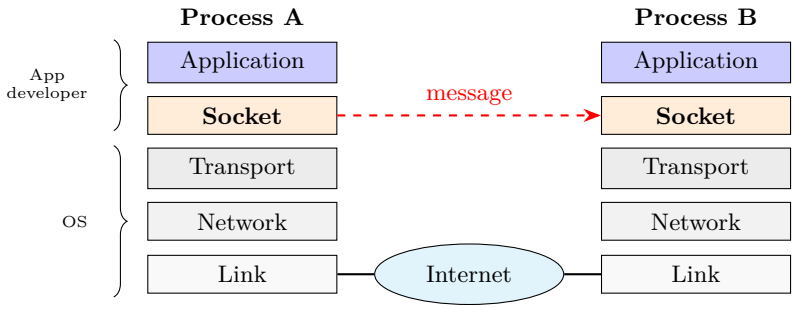
Processes and Communication

- A **process** is a program running on a host
- On the **same** host: inter-process communication (IPC)
- On **different** hosts: exchange of **messages**
- **Client process**: initiates communication
- **Server process**: waits to be contacted

Socket – The "Door" between Application and Transport

A **socket** is the interface between the application layer and the transport layer. The process sends/receives messages through its socket.

- Two sockets involved: one on each side
- Application developer controls everything **above** the socket
- OS controls everything **below** the socket (transport, network, link, physical)



10.3 Addressing Processes

IP address alone is not enough!

A process is uniquely identified by **IP address + port number**.

- The IP address identifies the **host**
- The port number identifies the **process** on the host
- Well-known ports: HTTP = **80**, Mail (SMTP) = **25**, DNS = **53**

10.4 Application-Layer Protocols

An application-layer protocol defines:

1. **Message types**: request and response
2. **Syntax**: the fields in the message and their format
3. **Semantics**: the meaning of the fields
4. **Rules**: when processes send/respond

Open protocols: defined in RFCs, available to everyone (HTTP, SMTP).
Proprietary protocols: private (Skype, Zoom).

10.5 Transport Service Requirements

Four dimensions of transport service

Data integrity (reliability)

Timing requirements (timing)

Throughput (throughput)

Security (security)

TCP vs. UDP – What do they offer?

Property	TCP	UDP
Reliable data transfer	✓	×
Flow control	✓	×
Congestion control	✓	×
Connection setup	✓	×
Timing requirements	×	×
Min. throughput	×	×
Security (built-in)	×	×

TLS provides encrypted TCP connections, data integrity, and authentication.

Exam Questions – 2.1 Principles

Question	Answer
What is a server in the client-server paradigm?	An always-on host with a permanent IP, often in data centers.
What is a client in the client-server paradigm?	A device that contacts servers, may be intermittently connected and have dynamic IPs.
Can clients communicate directly in client-server?	No.
What is peer-to-peer architecture?	A model where end systems (peers) communicate directly without a central server.
What is a socket?	An interface for sending and receiving process messages, connecting an app on one computer to an app on another.
Why is an IP address alone not enough?	Because multiple processes can run on the same host.
What does an application-layer protocol define?	Message types, syntax, semantics, and communication rules.
What does TCP provide?	Reliable transport, flow control, congestion control, connection setup.
What does UDP provide?	Unreliable data transfer with no added features.
What does TLS provide?	Encrypted TCP connections, data integrity, and end-point authentication.

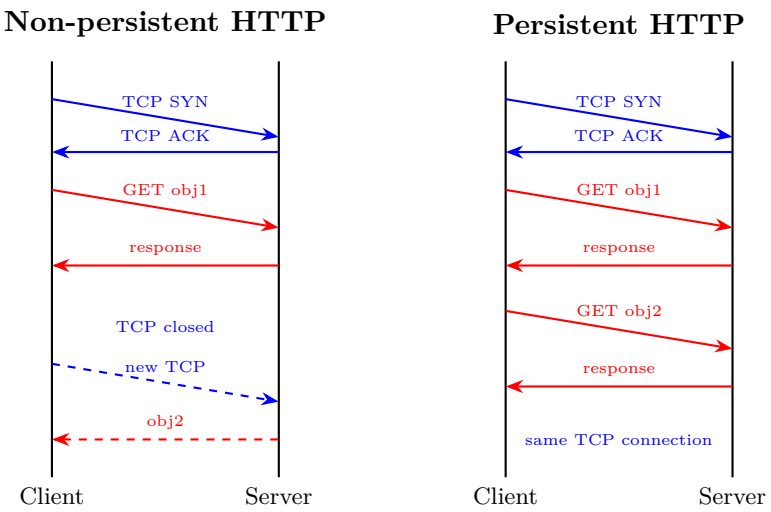
11 Web and HTTP (2.2)

11.1 HTTP Basics

HTTP – HyperText Transfer Protocol

- A **web page** consists of a base HTML file and referenced objects (images, scripts, etc.)
- HTTP follows the **client-server** model
- Uses **TCP** (port **80**)
- HTTP is **stateless** – the server does not store information about previous requests

11.2 Persistent vs. Non-persistent HTTP



Non-persistent HTTP: Time per object

Each object requires: **2 RTT + file transfer time**

- 1 RTT for TCP connection
- 1 RTT for HTTP request/response
- Problems: high RTT cost, OS overhead for many TCP connections

Response time for Non-persistent HTTP

$$T_{\text{non-pers}} = 2 \cdot RTT + T_{\text{fil}}$$

For a web page with base HTML + n objects:

$$T_{\text{total}} = (n+1) \times (2 \cdot RTT + T_{\text{fil}})$$

11.3 HTTP Messages

Two types: Request and Response.

HTTP Request – important methods

Method	Description
`GET`	Retrieves a resource; data in URL after "?"
`POST`	Sends data in the message body (entity body)
`HEAD`	Like GET, but returns headers only
`PUT`	Uploads / replaces a file on the server

HTTP Response – status codes

Code	Meaning
`200 OK`	The request was successful
`301 Moved Permanently`	The object has a new permanent URL
`400 Bad Request`	The server did not understand the message
`404 Not Found`	The document does not exist on the server

11.4 Cookies

Cookies – State information in stateless HTTP

Cookies are used to maintain state across HTTP requests.
Four components:

1. Cookie header in HTTP **response** (Set-Cookie)
2. Cookie header in HTTP **request** (Cookie)
3. Cookie file on the **client** (stored by the browser)
4. **Database** on the server

Use cases: authorization, shopping carts, recommendations, session state.

11.5 Web Caching (Proxy Server)

Web Cache – Goal: satisfy requests without origin server

- The browser sends **all** HTTP requests to the cache
- Object found? → cache returns it directly
- Not found? → cache fetches from origin server, caches it, returns it
- The cache acts both as a **server** (for the client) and **client** (for the origin server)

Benefits of caching:

1. Reduces response time for the client
2. Reduces traffic on the institution's access link
3. Cheaper than upgrading the access link

Conditional GET – Avoid unnecessary transfer

- Request header: `If-Modified-Since: <date>`
- Response if unchanged: `304 Not Modified` (no data sent)
- Response if changed: `200 OK` + updated object

11.6 HTTP/2 and HTTP/3

HTTP/2 – Reduce latency

- Goal: reduce delay in multi-object requests
- **Priority-based transmission**: important objects first
- **Server push**: the server can send objects the client has not requested
- **Framing**: objects are divided into frames that are **interleaved** → reduces HOL-blocking

HOL-blocking (Head-of-Line)

In HTTP/1.1, objects must be sent sequentially. A large object blocks small objects behind it in the queue. HTTP/2 solves this with frame interleaving.

HTTP/3 – Solves TCP limitations

- Uses **UDP** (via QUIC) instead of TCP
- Per-object error and congestion control
- Built-in security
- Solves the problem of *packet loss on TCP affecting all objects*

Exam Questions – 2.2 Web and HTTP

Question	Answer
What is HTTP?	The application layer protocol used by the web to transfer data.
Which protocol does HTTP use underneath?	TCP.
Is HTTP stateful or stateless?	Stateless – the server does not store past requests.
How many RTTs per object in non-persistent HTTP?	2 RTTs + file transmission time.
What does the POST method do?	Sends user input in the entity body of the request.
What does 404 mean?	Not Found – the requested document doesn't exist.
What are the four components of cookies?	1) Cookie in response, 2) Cookie in request, 3) File on client, 4) Database on server.
What is the goal of web caching?	To satisfy client requests without involving the origin server.
What is a Conditional GET used for?	To avoid sending an object if the cached version is still up-to-date.

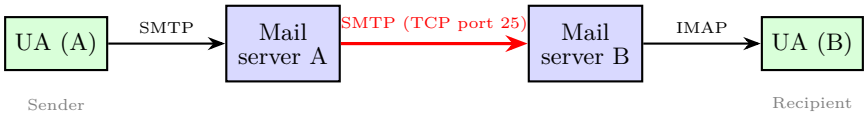
What is the main goal of HTTP/2?	To decrease delay in multi-object HTTP requests.
How does HTTP/2 reduce HOL blocking?	By dividing objects into frames and interleaving their transmission.
How does HTTP/3 address HTTP/2 limitations?	Uses UDP for per-object error and congestion control, includes security.

12 Email: SMTP and IMAP (2.3)

12.1 Components of the Email System

Three main components of the email system

1. **User agent (UA)**: compose, edit, and read email
2. **Mail server**: stores incoming messages (mailbox) and outgoing messages (message queue)
3. **SMTP**: the protocol for sending email between mail servers



12.2 The SMTP Protocol

SMTP – Simple Mail Transfer Protocol

- Port **25**, uses **TCP**
- **Push**-based (sender pushes data to receiver)
- Three phases: **handshaking**, **transfer**, **closure**
- Command/response interaction (ASCII text + status codes)
- Requires **7-bit ASCII** for the message body
- End of message: `CRLF.CRLF`

Differences between SMTP and HTTP

	SMTP	**HTTP**
Direction	**Push** (sender initiates)	**Pull** (receiver initiates)
Objects	Multipart in *one* message	Each object separately
Encoding	7-bit ASCII	Binary data allowed
Port	25	80

12.3 Email Format and IMAP

Message Format

- **Header**: To, From, Subject (part of the message content)
- **Body**: the actual message text
- Do not confuse with SMTP commands (MAIL FROM, RCPT TO) – they are part of the protocol, not the message

IMAP – Internet Message Access Protocol

- Used to **retrieve, organize, and delete** messages stored on the server
- Gmail/Hotmail use a web interface with SMTP (sending) and IMAP (retrieval)

The steps when A sends an email to B:

1. A composes email in their UA

2. UA sends to A's mail server
3. A's mail server opens TCP (port 25) to B's mail server
4. The message is sent via SMTP
5. The message is stored in B's mailbox
6. B reads the message via their UA (IMAP)

Exam Questions – 2.3 Email

Question	Answer
What are the three major components of email systems?	User agents, mail servers, and SMTP.
What is SMTP used for?	To send email messages between mail servers.
Which port does SMTP use?	Port 25.
What are the three phases of SMTP transfer?	Handshaking, transfer of messages, and closure.
List two differences between SMTP and HTTP.	SMTP is push-based; HTTP is pull-based. SMTP sends multipart messages; HTTP sends each object separately.
What encoding does SMTP require?	7-bit ASCII.
What is the difference between email headers and SMTP commands?	Headers are part of the message content; commands like MAIL FROM and RCPT TO are part of the SMTP protocol.
What is IMAP used for?	Retrieving, organizing, and deleting messages stored on the server.

13 DNS – Domain Name System (2.4)

13.1 What is DNS?

DNS – The Internet's Directory

- Translates **hostnames** (e.g. `www.google.com`) to **IP addresses**
- Implemented as a **distributed, hierarchical database**
- Is an **application-layer protocol** – hosts and DNS servers communicate directly to resolve names

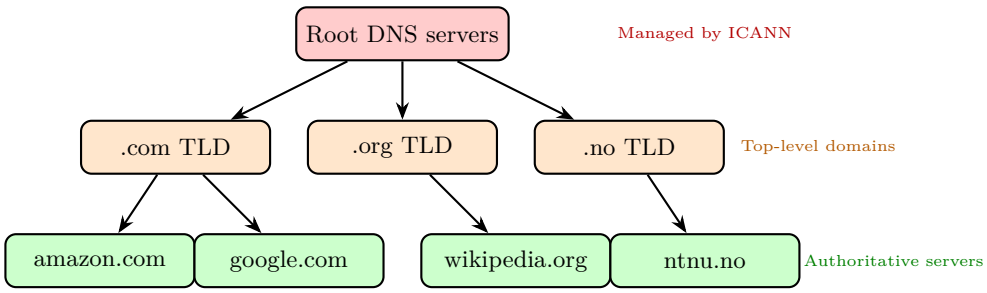
DNS services:

1. Hostname → IP translation
2. Host aliasing (canonical name vs. alias)
3. Mail server aliasing
4. Load distribution (load distribution)

Why is DNS distributed?

A centralized DNS would create: single point of failure, too much traffic, long distance, maintenance problems. DNS is "highly distributed" – millions of organizations manage their own DNS records.

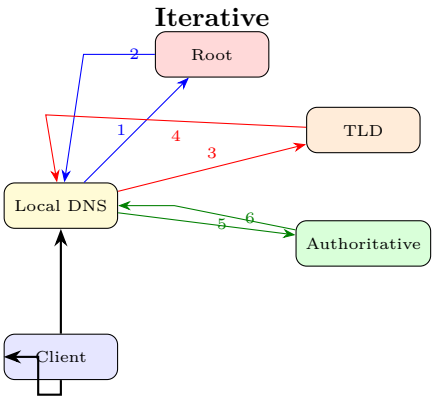
13.2 The DNS Hierarchy

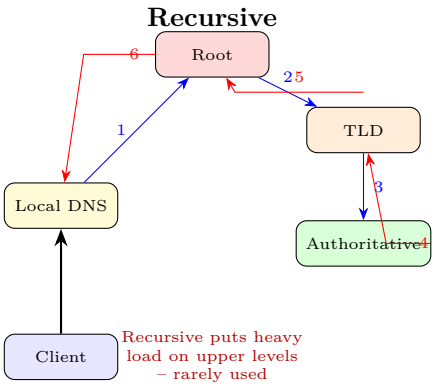


DNS server types

- **Root**: last resort when other servers cannot resolve; managed by ICANN
- **TLD (Top-Level Domain)**: handles .com, .org, .net, country domains (.no)
- **Authoritative**: provides official hostname→IP mappings for an organization
- **Local DNS server**: not part of the hierarchy; first stop for DNS queries, answers from cache or forwards

13.3 DNS Queries: Iterative vs. Recursive





13.4 DNS Caching and Records

DNS Caching

- Servers **cache** responses to get faster lookups
- Records have a **TTL** (Time To Live) – expire after a certain time
- Drawback: cached records can be **outdated** before TTL expires

DNS Resource Records (RR)

Format: `(name, value, type, TTL)`

Type	Name	Value	Description
A	hostname	IP address	Direct mapping
NS	domain	hostname for DNS server	Name server
CNAME	alias	canonical name	Alias
MX	domain	mail server hostname	Mail server

DNS Security

- DDoS protection: traffic filtering, caching in local DNS servers
- **DNSSEC**: authentication and message integrity

Exam Questions – 2.4 DNS

Question	Answer
What does DNS stand for and what is its main role?	Domain Name System; translates hostnames to IP addresses.
How is DNS implemented?	As a distributed, hierarchical database with many name servers.
List the hierarchy of DNS servers.	Root → TLD → Authoritative → Local DNS server.
What are TLD servers responsible for?	Managing top-level domains like .com, .org, .net, and country domains.

What do authoritative DNS servers do?	They provide the official hostname-to-IP mappings for an organization.
Explain iterated DNS query.	The contacted server replies with the next server to ask, not the final answer.
Explain recursive DNS query.	The contacted server handles the full resolution and replies with the answer.
What types of DNS records exist?	A (IP address), NS (name server), CNAME (alias), MX (mail server).
How does caching improve DNS?	By storing mappings to speed up responses and reduce traffic.
What protocol provides DNS security?	DNSSEC, which authenticates and ensures message integrity.

14 P2P File Distribution (2.5)

14.1 Client-Server vs. P2P Distribution

Distribution time – The core question

How long does it take to distribute a file (size F) from *one* server to N peers?
Variables:

- u_s = server upload capacity
- d_i = peer i's download capacity
- u_i = peer i's upload capacity
- $d_{\min}$ = minimum download rate among all peers

Client-Server distribution time

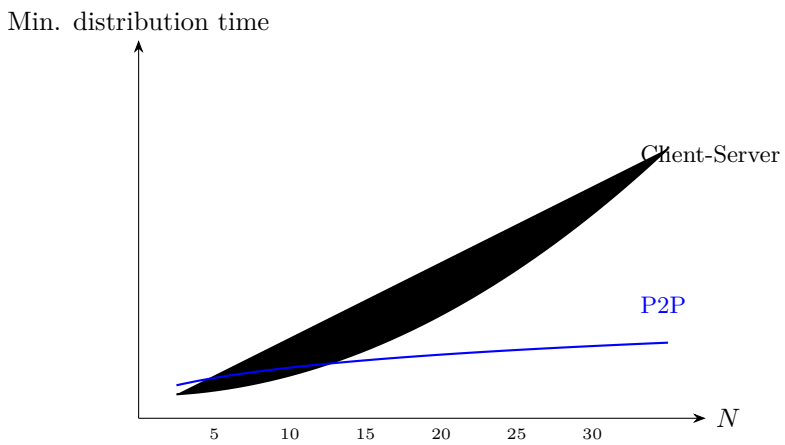
$$D_{CS} \geq \max\left\{\frac{NF}{u_s}, \frac{F}{d_{\min}}\right\}$$

- NF/u_s: the server must send N copies sequentially
- $F/d_{\min}$: the slowest client determines the minimum time
- **Grows linearly with** N

P2P distribution time

$$D_{P2P} \geq \max\left\{\frac{F}{u_s}, \frac{F}{d_{\min}}, \frac{NF}{u_s + \sum u_i}\right\}$$

- F/u_s: the server must upload *at least* one copy
- $F/d_{\min}$: the slowest client cannot receive the file faster
- $NF/(u_s + \sum u_i)$: total load shared by **all** upload capacities
- **Self-scaling**: $\sum u_i$ grows with N!



Example: $u_s = 10u$, $F/u = 1$ **hour,** $d_{\min} \geq u_s$

Client-Server: $D_{CS} = NF/u_s = NF/(10u) = N/10$ (linear)
P2P: $D_{P2P} = NF/(u_s + Nu) = NF/((10+N)u) = N/(10+N)$ (flattens out!)
When $N = 30$: CS takes 3 hours, P2P takes 0.75 hours.

Exam Questions – 2.5 P2P

Question	Answer
What is the main difference between P2P and client-server?	P2P relies on direct peer communication; client-server depends on always-on servers.
What is a major drawback of client-server file distribution?	Server must send a full copy to each peer, overloading it.
How does P2P reduce server load?	Peers redistribute parts of the file they have received.
What makes P2P self-scalable?	As more peers join, they also contribute upload bandwidth.
What is the most popular P2P protocol (2020)?	BitTorrent.
List key differences between CS and P2P distribution.	CS: server sends full file to each peer. P2P: peers share parts, reducing server load and increasing scalability.

15 Video Streaming and CDN (2.6)

15.1 Multimedia: Video

Video and encoding

- Video = sequence of images displayed at a constant rate (e.g. 24 fps)
- Digital image = array of pixels, each represented with bits
- **Encoding** exploits redundancy to reduce bits:
  - **Spatial coding**: redundancy *within* one image (e.g. many identical pixels)
  - **Temporal coding**: redundancy *between* images (send only differences)

CBR vs. VBR

	CBR	VBR
Full form	Constant Bit Rate	Variable Bit Rate
Encoding	Fixed rate	Rate changes with content

Examples: MPEG1 (1.5 Mbps), MPEG2 (3–6 Mbps), MPEG4 (64 Kbps – 12 Mbps)

15.2 Streaming of Stored Video

Challenges:

- Bandwidth varies over time (congestion)
- Packet loss and delay cause poor quality

Continuous Playout Constraint

Once playback starts, it must match the original timing. But network delays vary (**jitter**), so the client needs a **buffer** and **playout delay** to compensate.

15.3 DASH – Dynamic Adaptive Streaming over HTTP

DASH

Server:

- Divides video into **chunks** (segments)
- Each chunk encoded at **multiple quality levels** (bit rates)
- **Manifest file**: list of URLs for different chunks/qualities

Client ("intelligence" at client):

- Periodically measures server-to-client bandwidth
- Decides **when** to fetch a chunk (avoid buffer starvation/overflow)
- Decides **which bit rate** (higher quality when more bandwidth)
- Decides **where** to fetch chunk from (nearest server)

Streaming video = encoding + DASH + playout buffering

15.4 CDN – Content Distribution Networks

Why not one mega-server?

- Single point of failure
- Traffic bottleneck (congestion)
- Long distance to remote clients
- Multiple copies of video over the same link
- **The solution does not scale!**

CDN – Two strategies

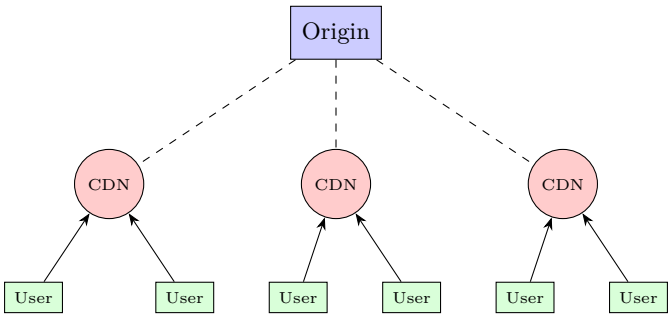
"Enter Deep":

- Place CDN servers **deep inside** access networks, close to users
- Many small server clusters

"Bring Home":

- Fewer, **larger** CDN clusters near access networks (not inside them)

The client gets the manifest from the CDN, fetches chunks from the **best suited server** at the appropriate bit rate.



Users fetch content from the nearest CDN node

OTT – Over-the-Top

Services like Netflix and YouTube run "over" the existing IP infrastructure. They do not control the network, but use CDNs to deliver content efficiently.

Exam Questions – 2.6 Video and CDN

Question	Answer
How much of ISP traffic is video streaming?	About 80%.
What is spatial and temporal coding?	Spatial: within one image. Temporal: between images.
What is DASH?	Dynamic Adaptive Streaming over HTTP.
What does the server do in DASH?	Splits video into chunks, multiple qualities, provides manifest file.

What does the client control in DASH?	When, which bitrate, and from where to request chunks.
What is a CDN?	Content Delivery Network – stores videos near users.
Why not use one mega-server?	Bottleneck, single point of failure, long distance, doesn't scale.
What is the "Enter Deep" strategy?	Place CDN servers deep inside access networks, close to users.
What is the "Bring Home" strategy?	Fewer, larger CDN clusters near access networks.
What is an OTT service?	Over-the-top service (e.g. Netflix), runs on top of IP infrastructure.

16 Socket Programming (2.7)

16.1 Socket Basics

Socket

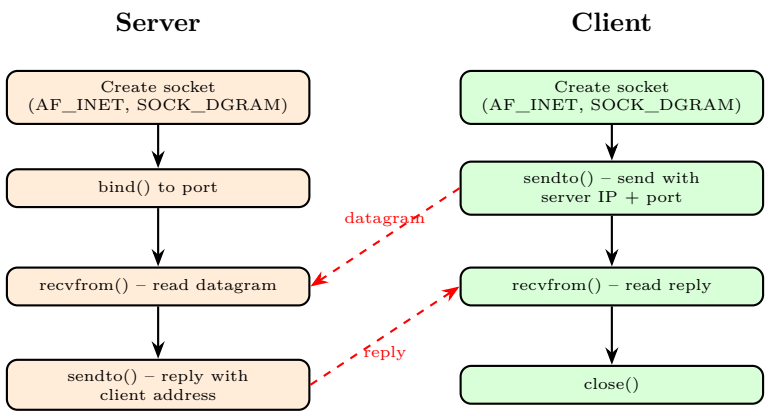
Two types of sockets for two transport services:

- **UDP socket**: unreliable datagram (`SOCK_DGRAM`)
- **TCP socket**: reliable, byte stream-oriented (`SOCK_STREAM`)

16.2 Socket Programming with UDP

UDP – No connection

- **No handshaking** before data transfer
- Sender attaches **IP address and port number** to each packet
- Receiver retrieves sender's IP and port from the packet
- Data can be **lost** or arrive in **wrong order**



Python UDP client:

```
from socket import *
serverName = 'hostname'
serverPort = 12000
clientSocket = socket(AF_INET, SOCK_DGRAM)
message = input('Input lowercase sentence:')
clientSocket.sendto(message.encode(), (serverName, serverPort))
modifiedMessage, serverAddress = clientSocket.recvfrom(2048)
print(modifiedMessage.decode())
clientSocket.close()
```

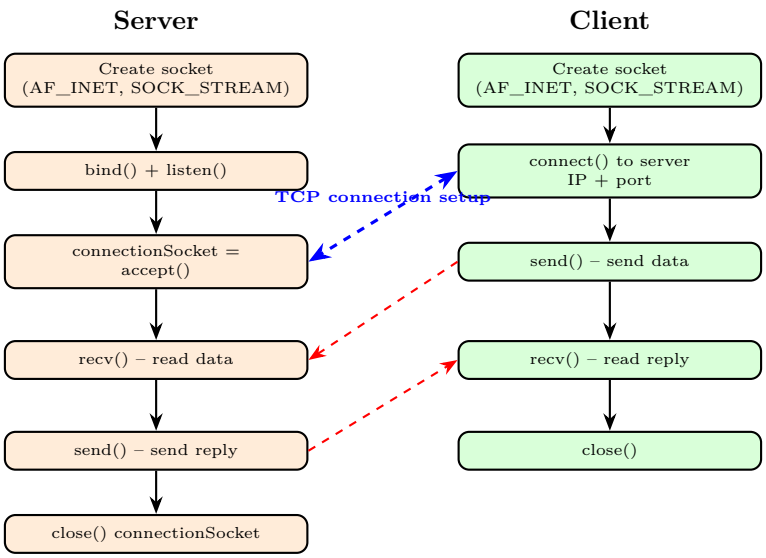
Python UDP server:

```
from socket import *
serverPort = 12000
serverSocket = socket(AF_INET, SOCK_DGRAM)
serverSocket.bind(('', serverPort))
print("The server is ready to receive")
while True:
    message, clientAddress = serverSocket.recvfrom(2048)
    modifiedMessage = message.decode().upper()
    serverSocket.sendto(modifiedMessage.encode(), clientAddress)
```

16.3 Socket Programming with TCP

TCP – Connection-oriented

- The server must run first and have created a **"welcoming socket"**
- The client creates a TCP socket with the server's IP + port → **TCP connection** is established
- The server creates a **new socket** (`connectionSocket`) for each client
- Uses **source port** to distinguish clients
- TCP provides reliable, ordered **byte-stream** transfer



Python TCP client:

```
from socket import *
serverName = 'servername'
serverPort = 12000
clientSocket = socket(AF_INET, SOCK_STREAM)
clientSocket.connect((serverName, serverPort))
sentence = input('Input lowercase sentence:')
clientSocket.send(sentence.encode())
modifiedSentence = clientSocket.recv(1024)
print('From Server:', modifiedSentence.decode())
clientSocket.close()
```

Python TCP server:

```
from socket import *
serverPort = 12000
serverSocket = socket(AF_INET, SOCK_STREAM)
serverSocket.bind(('', serverPort))
serverSocket.listen(1)
print('The server is ready to receive')
while True:
    connectionSocket, addr = serverSocket.accept()
    sentence = connectionSocket.recv(1024).decode()
    capitalizedSentence = sentence.upper()
    connectionSocket.send(capitalizedSentence.encode())
    connectionSocket.close()
```

16.4 Key Differences: UDP vs. TCP Sockets

Comparison UDP vs. TCP

Property	UDP	TCP
Socket type	`SOCK_DGRAM`	`SOCK_STREAM`
Connection	None	Connection setup
Reliability	Unreliable	Reliable, ordered
Data unit	Datagram (packet)	Byte stream
Addressing	Explicit per packet	Automatic (via connect)
Send method	`sendto()`	`send()`
Receive method	`recvfrom()`	`recv()`
Server new socket?	No	Yes (`accept()`)

Exam Questions – 2.7 Socket Programming

Question	Answer
What is a socket?	A door between an application process and the end-end transport protocol.
What are the two types of sockets?	UDP: unreliable datagram. TCP: reliable, byte stream-oriented.
How does the client send data in UDP?	It attaches IP address and port number to each packet.
What must happen before a TCP client contacts the server?	The server must be running and have created a welcoming socket.
Why does the server create a new socket for each client in TCP?	To allow communication with multiple clients simultaneously.
What does TCP use to distinguish different clients?	Source port numbers.
What does TCP provide from the app's view?	Reliable, in-order byte-stream transfer between client and server.
Why might an app choose UDP over TCP?	When low overhead and speed are more important than reliability.
Why might an app choose TCP over UDP?	When reliable and ordered delivery of data is needed.

17 Summary – Chapter 2

Chapter 2: Main Topics

- **Application architectures**: client-server and P2P
- **Service requirements**: reliability, bandwidth, delay
- **Transport model**: TCP (reliable, connection-oriented) vs. UDP (unreliable, datagram)
- **Protocols**: HTTP, SMTP, IMAP, DNS, BitTorrent
- **Video streaming**: DASH, CDN
- **Socket programming**: TCP and UDP sockets in Python

Important topics to understand

- **Centralized vs. decentralized** (DNS, CDN, P2P)
- **Stateless vs. stateful** (HTTP is stateless, cookies provide state)
- **Scalability** (P2P vs. client-server distribution time)
- **Reliable vs. unreliable** message transfer (TCP vs. UDP)
- **"Complexity at the network edge"** – intelligence at the endpoints

Formula Summary

Non-persistent HTTP response time:

$$T = 2 \cdot RTT + T_{\text{fil}}$$

Client-Server distribution time:

$$D_{CS} \geq \max\left\{\frac{NF}{u_s}, \frac{F}{d_{\min}}\right\}$$

P2P distribution time:

$$D_{P2P} \geq \max\left\{\frac{F}{u_s}, \frac{F}{d_{\min}}, \frac{NF}{u_s + \sum u_i}\right\}$$

Port numbers to remember

Protocol	Port
HTTP	80
HTTPS	443
SMTP	25
DNS	53

TTM4100 – KTN

Chapter 3: The Transport Layer

Computer Networking: A Top-Down Approach, 8th ed.
Kurose & Ross – NTNU Spring 2026

18 Transport Layer Services and Protocols (3.1)

18.1 The Role of the Transport Layer

The Transport Layer – Overview

The transport layer provides **logical communication** between *application processes* on different hosts.

- Sender side: breaks application messages into **segments**, sends them to the network layer
- Receiver side: reassembles segments into messages, delivers to the application layer
- Two main protocols: **TCP** and **UDP**

18.2 Transport Layer vs. Network Layer

Important Difference

- **Network layer (IP):** Logical communication between *hosts*
- **Transport layer (TCP/UDP):** Logical communication between *processes*
- The transport layer extends the network layer's host-to-host delivery to process-to-process delivery

Household Analogy

Think of two houses with 12 children in each house sending letters to each other:

- **Processes** = children
- **Application messages** = letters in envelopes
- **Hosts** = houses
- **Transport protocol** = Ann and Bill (the children who collect/sort mail)
- **Network layer protocol** = the postal service

Ann and Bill work within the house – they cannot control what happens out in the postal system.

18.3 Internet Transport Protocols

TCP vs. UDP – Overview

TCP (Transmission Control Protocol):

- Reliable, ordered delivery (*reliable, in-order delivery*)
- Congestion control (*congestion control*)
- Flow control (*flow control*)
- Connection setup (*connection setup*)

UDP (User Datagram Protocol):

- Unreliable, unordered delivery (*unreliable, unordered delivery*)
- No congestion control, flow control, or connection setup
- "Best effort" – no guarantee of delivery

Services NOT provided: Delay or bandwidth guarantees.

19 Multiplexing and Demultiplexing (3.2)

19.1 Basic Concept

Multiplexing and Demultiplexing

Multiplexing at sender:

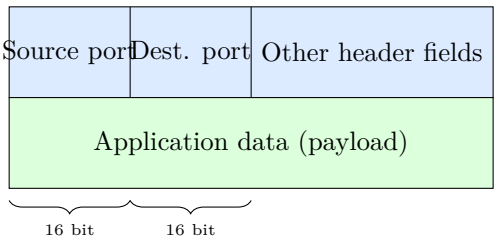
- Handles data from multiple sockets
- Adds transport header (with port number) to identify socket

Demultiplexing at receiver:

- Uses header information (port number) to deliver segments to the correct socket

19.2 How Demultiplexing Works

- The host receives IP datagrams – each datagram has source IP, destination IP, and one transport layer segment
- Each segment has a **source port number** and **destination port number** (16 bits each)
- The host uses IP addresses and port numbers to direct the segment to the correct socket



19.3 Connectionless Demultiplexing (UDP)

UDP socket identified by 2-tuple

UDP socket identified by: **(destination IP, destination port)**

- Segments with the *same* destination IP and destination port, but *different* source IP/port, are directed to the **same socket**
- Source port is used as a "return address"

19.4 Connection-Oriented Demultiplexing (TCP)

TCP socket identified by 4-tuple

TCP socket identified by: **(source IP, source port, destination IP, destination port)**

- Receiver uses *all four values* to direct the segment to the correct socket
- The server host can support many simultaneous TCP sockets – each identified by its own 4-tuple
- Web servers have different sockets for each client (different source IP and/or source port)

Important Difference UDP vs. TCP demux

- **UDP:** Only destination port determines which socket receives the segment
- **TCP:** Both source and destination (IP + port) determine socket – two clients with different source IP/port get *separate* sockets

20 Connectionless Transport: UDP (3.3)

20.1 UDP – Overview

UDP – User Datagram Protocol (RFC 768)

- "Bare bones" transport protocol – "no frills"
- **Best effort**: Segments can be lost or delivered in wrong order
- **Connectionless:** No handshake between sender and receiver, each segment handled independently

Why use UDP?

- No connection setup (which adds delay)
- Simple: no state at sender/receiver
- Small header (8 bytes vs. 20 bytes for TCP)
- No congestion control – can send as fast as desired / the application allows

20.2 Use Cases for UDP

- Multimedia streaming (tolerates loss, rate-sensitive)
- DNS (fast, short queries)
- SNMP (network management)

- HTTP/3 (uses QUIC over UDP)

UDP and Reliability

If reliable transfer is necessary over UDP, it must be added at the **application layer**. Example: HTTP/3 / QUIC adds reliability and congestion control over UDP.

20.3 UDP Segment Format

Header 8 bytes	Source port #	Dest. port #
	Length	Checksum
	Application data (payload)	

20.4 UDP Checksum

Checksum – Error Detection

Goal: Detect "errors" (e.g., flipped bits) in the transmitted segment.
Sender:

- Treats the segment contents (including header fields) as a sequence of 16-bit words
- Adds all 16-bit words using **1's complement addition**
- Takes the **1's complement** of the result = checksum
- Places the checksum in the UDP header's checksum field

Receiver:

- Computes the checksum of the received segment (including the checksum field)
- If the result is *not* `1111111111111111` ⇒ error detected
- If the result is all 1s ⇒ no error detected (but there may still be errors!)

Checksum Example

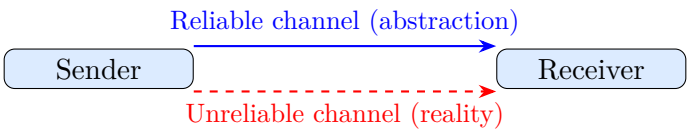
Add two 16-bit words:

$$\begin{aligned} & \texttt{1110011001100110} \\ + & \texttt{1101010101010101} \\ = & \texttt{1011101110111011} \quad \text{(with wraparound carry)} \\ \text{Checksum} = & \texttt{0100010001000100} \quad \text{(1's complement)} \end{aligned}$$

21 Principles of Reliable Data Transfer (3.4)

21.1 Problem Statement

Reliable data transfer is one of the most important problems in networking. The challenge is to build a **reliable channel** over an **unreliable channel** (which can lose packets, flip bits, etc.).



21.2 rdt1.0: Transfer over a Perfectly Reliable Channel

rdt1.0 – Simplest Case

Assumption: Underlying channel is *perfectly reliable* (no bit errors, no packet loss).
Sender: Just send data down into the channel.
Receiver: Just read data from the channel.
No error handling necessary – trivial!

21.3 rdt2.0: Channel with Bit Errors

rdt2.0 – Stop-and-Wait with ACK/NAK

Assumption: The channel may have *bit errors* (but no packet loss).
New mechanisms:

- **Checksum:** Detects bit errors
- **ACK (acknowledgement):** Receiver says "OK, received correctly"
- **NAK (negative acknowledgement):** Receiver says "error, send again"
- **Retransmission:** Sender retransmits on NAK

The sender sends one packet and **waits** for ACK/NAK before the next packet is sent (*stop-and-wait*).

Fatal Problem with rdt2.0!

What if ACK/NAK itself has bit errors? The sender does not know if the packet was received correctly!

- Cannot simply retransmit – causes possible duplicates
- Solution → rdt2.1

21.4 rdt2.1: Sequence Numbers

rdt2.1 – Solution: Sequence Numbers

Sender adds a **sequence number** (0 or 1) to each packet:

- If ACK/NAK is corrupt → sender retransmits current packet
- Receiver discards duplicates using the sequence number
- Two sequence numbers (0 and 1) are sufficient for stop-and-wait

21.5 rdt2.2: NAK-Free Protocol

rdt2.2 – ACK Only (no NAK)

Same functionality as rdt2.1, but *without NAK*:

- Receiver sends **ACK with the sequence number** of the last correctly received packet
- Duplicate ACK at sender acts as NAK (means: "the last packet was wrong")
- Sender retransmits when it receives a duplicate ACK

21.6 rdt3.0: Channel with Bit Errors AND Packet Loss

rdt3.0 – Timer / Countdown Timer

New assumption: The channel can also *lose packets* (data or ACKs).
New mechanism: Countdown timer

- Sender waits a "reasonable" time for ACK
- Retransmits if no ACK is received before timeout
- If the packet (or ACK) is just delayed (not lost): retransmission causes a duplicate, but sequence numbers handle this

rdt3.0 is correct, but performance is poor on high-capacity links!

21.7 Performance of rdt3.0 (Stop-and-Wait)

Utilization for Stop-and-Wait

$$U_{\text{sender}} = \frac{L/R}{RTT + L/R}$$

where:

- L = packet size (bits)
- R = link capacity (bps)
- RTT = round-trip time
- L/R = transmission time for one packet

Example: Poor Utilization

Given: $L = 8000$ bits, $R = 1$ Gbps, $RTT = 30$ ms:

$$U_{\text{sender}} = \frac{8000/10^9}{0{,}030 + 8000/10^9} = \frac{0{,}000008}{0{,}030008} = 0{,}00027 = 0{,}027\%$$

Only 0.027% utilization! The sender is active only 0,008 ms out of 30,008 ms.

21.8 Pipelining: Increased Utilization

Pipelining – Send Multiple Packets Without Waiting

Solution to poor utilization: **pipelining** – the sender allows multiple packets to be "in flight" (sent but not yet acknowledged).

- Requires a larger range of sequence numbers
- Buffering at sender and/or receiver
- Two main variants: **Go-Back-N** and **Selective Repeat**

With pipelining (3 packets):

$$U_{\text{sender}} = \frac{3 \cdot L/R}{RTT + L/R}$$

3x better utilization than stop-and-wait!

21.9 Go-Back-N (GBN)

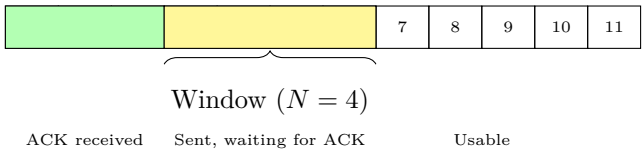
Go-Back-N Protocol

Sender:

- "Window" of up to N unacknowledged packets in the pipeline
- **Cumulative ACK:** ACK(n) acknowledges all packets up to and including sequence number n
- Timer for the oldest unacknowledged packet
- On timeout: retransmit *all* packets in the window (from the oldest unacknowledged)

Receiver:

- Sends ACK only for the highest correctly received packet *in order*
- Can have only one expected sequence number
- Discards out-of-order packets (no buffering) – re-sends ACK for last in-order packet



21.10 Selective Repeat (SR)

Selective Repeat Protocol

Sender:

- Window of up to N unacknowledged packets
- **Individual ACK:** Each packet has its own ACK
- Timer for *each* unacknowledged packet
- On timeout: retransmit only the *specific* packet that timed out

Receiver:

- Acknowledges *each* correctly received packet individually

- **Buffers** out-of-order received packets
- Delivers to the application when the "gap" is filled in

SR Dilemma: Window Size vs. Sequence Number Space

The window size N must be **at most half the sequence number space**:

$$N \leq \frac{k}{2} \quad \text{where } k \text{ is the number of possible sequence numbers}$$

Otherwise the receiver cannot distinguish between new packets and retransmissions!
Example: With sequence numbers $\{0, 1, 2, 3\}$ (4 possible), $N \leq 2$.

Property	Go-Back-N	Selective Repeat
ACK type	Cumulative	Individual
Receiver buffering	No	Yes
On timeout	Retransmit all	Retransmit one
Timer	One (oldest)	Per packet
Complexity	Lower	Higher
Efficiency	Lower (many retransm.)	Higher

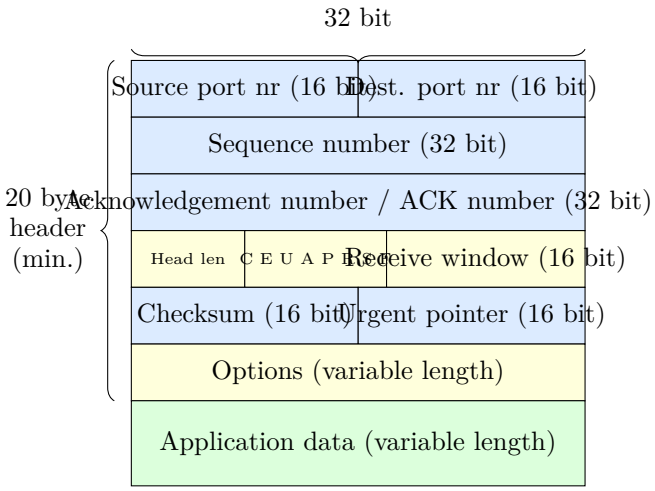
22 Connection-Oriented Transport: TCP (3.5)

22.1 TCP – Overview

TCP – Properties

- **Point-to-point:** One sender, one receiver
- **Reliable, ordered byte stream:** No message boundaries
- **Full duplex:** Data can flow in both directions simultaneously (bi-directional data flow)
- **Cumulative ACK**
- **Pipelining:** Window-controlled segments (congestion & flow control set window)
- **Connection-oriented:** Handshake before data exchange
- **Flow and congestion control:** Sender does not overwhelm receiver or network

22.2 TCP Segment Structure



Important Fields in the TCP Header

- **Sequence number:** Byte number of the first byte in the segment's data
- **ACK number:** Next byte the receiver expects (cumulative ACK)
- **Receive window (rwnd):** Number of bytes the receiver is willing to accept (flow control)
- **Flags:** SYN, FIN, ACK, RST, PSH, URG, CWR, ECE
- **Header length:** Variable due to options (normally 20 bytes without options)

22.3 MSS and MTU

MSS and MTU

- **MTU (Maximum Transmission Unit):** Largest link layer frame (typically **1500 bytes** for Ethernet)
- **MSS (Maximum Segment Size):** Max application data in a segment
- MSS = MTU − IP header − TCP header = 1500 − 20 − 20 = **1460 bytes**
- MSS refers to application data, *not* including the TCP header

22.4 TCP Sequence Numbers and ACK

Sequence Numbers and ACK in TCP

Sequence numbers:

- TCP numbers *bytes* (not segments!)
- The sequence number is the byte number of the first data byte in the segment
- Example: file of 500 000 bytes, MSS = 1000 bytes ⇒ segment 0 has seq=0, segment 1 has seq=1000, segment 2 has seq=2000, ...

ACK numbers:

- The ACK number is the sequence number of the **next byte the receiver expects**
- **Cumulative ACK:** Acknowledges all bytes up to ACK number − 1
- Question: What does the receiver do with out-of-order packets? TCP spec says nothing – up to implementation (in practice: buffer them)

Example: Telnet Session

Host A sends 'C' (1 byte) to Host B, both start with seq=42 (A) and seq=79 (B):

1. A → B: Seq=42, ACK=79, data='C'
2. B → A: Seq=79, ACK=43, data='C' (echo)
3. A → B: Seq=43, ACK=80

22.5 TCP Round-Trip Time and Timeout

RTT Estimation and Timeout

SampleRTT: Measured time from segment sent to ACK received (only for segments sent once).
Estimated RTT (exponentially weighted moving average – EWMA):

$$\text{EstimatedRTT} = (1-\alpha)\cdot\text{EstimatedRTT} + \alpha\cdot\text{SampleRTT}$$

Typical: $\alpha = 0{,}125$ (i.e. 1/8)
Deviation measure (Safety margin):

$$\text{DevRTT} = (1-\beta)\cdot\text{DevRTT} + \beta\cdot|\text{SampleRTT} - \text{EstimatedRTT}|$$

Typical: $\beta = 0{,}25$ (i.e. 1/4)
Timeout interval:

$$\text{TimeoutInterval} = \text{EstimatedRTT} + 4\cdot\text{DevRTT}$$

Important about SampleRTT

SampleRTT is **not** measured for retransmitted segments (Karn's algorithm) – one cannot know whether the ACK is for the original or the retransmission.

22.6 TCP Reliable Data Transfer

TCP Sender – Simplified

TCP uses **one retransmission timer** per connection (not per segment).
Events at sender:

1. **Data from application:** Create segment with sequence number, start timer if not already running
2. **Timeout:** Retransmit the segment with the lowest sequence number that is not acknowledged. Restart timer
3. **ACK received:** If ACK acknowledges previously unacknowledged segments – update what is acknowledged. Restart timer if there are still unacknowledged segments

TCP Fast Retransmit

If sender receives **3 duplicate ACKs** (i.e. 4 ACKs for the same segment): retransmit the segment *before* timeout expires.
Rationale: Duplicate ACKs indicate that a segment has been lost, but subsequent segments have arrived.

22.7 TCP Flow Control

Flow Control

Goal: Sender should not overflow the receiver's buffer.
Mechanism:

- Receiver advertises available buffer space in the TCP header via the **rwnd** field (receive window)
- rwnd = RcvBuffer − (LastByteRcvd − LastByteRead)
- Sender limits unacknowledged data: LastByteSent − LastByteAcked ≤ rwnd
- Default RcvBuffer = 4096 bytes (can be increased via operating system)

What when rwnd

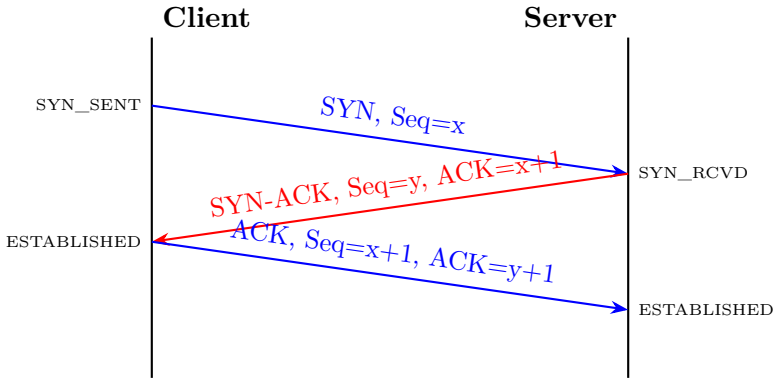
When the receiver advertises rwnd = 0, the sender still sends small **probe segments** (1 byte) to get an updated rwnd value and avoid deadlock.

22.8 TCP Connection Management

22.8.1 Three-Way Handshake

TCP Connection Setup – 3-Way Handshake

1. **SYN:** Client sends SYN segment (SYN=1, chooses random sequence number `client_isn`)
2. **SYN-ACK:** Server responds with SYNACK (SYN=1, ACK=1, chooses `server_isn`, acks `client_isn+1`)
3. **ACK:** Client sends ACK (ACK=1, acks `server_isn+1`). Can contain data!



Why Not a Two-Way Handshake?

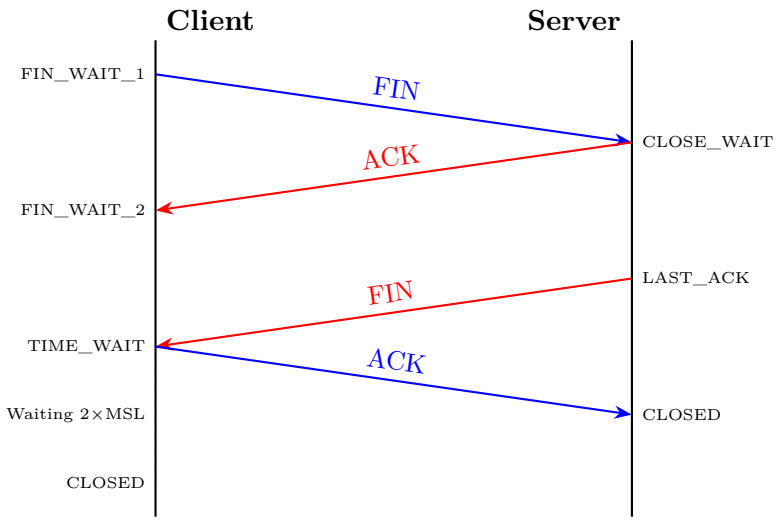
A two-way handshake fails because:

- Variable delays – retransmitted messages can arrive after the connection is closed
- Messages can arrive in wrong order
- Cannot "see" the other side – **half-open connections** arise

22.8.2 Connection Closing

TCP Connection Closing – 4-Step Process

1. Client sends **FIN** segment
2. Server responds with **ACK** (half-closed – server can still send data)
3. Server sends **FIN** segment
4. Client responds with **ACK**, enters **TIME_WAIT** (waits typically 2 × MSL)



22.8.3 TCP States

State	Description
CLOSED	No connection
LISTEN	Server waiting for incoming SYN
SYN_SENT	Client has sent SYN, waiting for SYN-ACK
SYN_RCVD	Server has received SYN, sent SYN-ACK
ESTABLISHED	Connection established, data can be transferred
FIN_WAIT_1	Has sent FIN, waiting for ACK
FIN_WAIT_2	FIN acknowledged, waiting for FIN from other side
CLOSE_WAIT	Received FIN, waiting for application to close
LAST_ACK	Sent FIN, waiting for final ACK
TIME_WAIT	Waiting 2×MSL to ensure last ACK is received

23 Principles of Congestion Control (3.6)

23.1 What is Congestion?

Congestion

Congestion occurs when too many sources send too much data too fast, such that **the network** cannot handle it all.
Symptoms:

- Long queueing delays (queues in router buffers)
- Packet loss (buffer overflow in routers)

Difference from flow control:

- **Flow control:** Prevents sender from overwhelming the *receiver*
- **Congestion control:** Prevents sender from overwhelming the *network*

23.2 Costs of Congestion

Costs of Congestion

- Queueing delays increase as arrival rate approaches link capacity
- Retransmissions due to packet loss – "wasted" work
- Unnecessary retransmissions (premature timeout) – the sender sends *copies* of packets already delivered
- When a packet is dropped along a path, all upstream transmission capacity used on that packet is wasted

23.3 Approaches to Congestion Control

Two Main Approaches

1. **End-to-end congestion control:**
   - No explicit feedback from the network
   - Congestion inferred from observed loss and delay
   - TCP uses this approach
2. **Network-assisted congestion control:**
   - Routers provide feedback to the sender
   - Can be *direct* (explicit rate) or *indirect* (ECN bit in header)
   - ECN (Explicit Congestion Notification): 2 bits in the IP header

24 TCP Congestion Control (3.7)

24.1 Overview: CWND and RWND

The Two "Brakes" in TCP

The TCP sender limits the transmission rate with two windows:

- **cwnd (Congestion Window):** Determined by the *sender* based on observed congestion. *Not* visible in the header (internal variable)
- **rwnd (Receive Window):** Determined by the *receiver*, advertised in the TCP header

The golden rule:

$$\text{LastByteSent} - \text{LastByteAcked} \leq \min(\text{cwnd}, \text{rwnd})$$

TCP send rate:

$$\text{rate} \approx \frac{\text{cwnd}}{\text{RTT}} \text{ bytes/sec}$$

Bandwidth With and Without Congestion Control

Without congestion control (pre-Tahoe):

$$BW_i = \frac{\text{TCP Window}}{\text{RTT}} = \frac{\text{RWND} \cdot \text{MSS}}{\text{RTT}}$$

With congestion control:

$$BW_i = \frac{\text{TCP Window}}{\text{RTT}} = \frac{\min(\text{CWND}, \text{RWND}) \cdot \text{MSS}}{\text{RTT}}$$

24.2 TCP Slow Start

Slow Start – Exponential Growth

At **connection start**:

- cwnd = 1 MSS
- Initial rate = MSS/RTT
- For **each ACK received**: cwnd = cwnd + MSS
- Results in **doubling** of cwnd per RTT (exponential growth)

Example (MSS = 500 bytes, RTT = 200 ms):
Initial rate = 500/0,2 = 20 kbps, but doubles every RTT.

Slow Start Calculation

If cwnd = 4500 and MSS = 1500:
3 segments are sent (4500/1500 = 3), and 3 ACKs arrive this RTT:

- ACK 1: cwnd = 4500 + 1500 = 6000
- ACK 2: cwnd = 6000 + 1500 = 7500
- ACK 3: cwnd = 7500 + 1500 = 9000

I.e. cwnd **doubled** from 4500 to 9000 in one RTT!

24.3 Congestion Avoidance – AIMD

Congestion Avoidance – Linear Growth

When cwnd ≥ ssthresh (slow start threshold):

- **Additive Increase:** For each ACK received:

  $$\text{cwnd} = \text{cwnd} + \text{MSS} \cdot \frac{\text{MSS}}{\text{cwnd}}$$

  Results in an increase of **1 MSS per RTT** (linear growth)
- **Multiplicative Decrease:** On packet loss: halve cwnd

Gives a "sawtooth" pattern (*saw tooth behavior*) – continually probing for available bandwidth.

Congestion Avoidance Calculation

If cwnd = 3000 and MSS = 1500 (in Congestion Avoidance):

- cwnd0 = 3000 (start of RTT)
- 3000/1500 = 2 segments sent ⇒ 2 ACKs
- ACK 1: cwnd = 3000 + 1500·(1500/3000) = 3000 + 750 = 3750
- ACK 2: cwnd = 3750 + 1500·(1500/3750) = 3750 + 600 = 4350

I.e. cwnd increased by approx. **1 MSS** to the next RTT (linear growth). (Exactly 1 MSS at the start of the next RTT period.)

24.4 Loss Events and ssthresh

Reaction to Loss – ssthresh

The variable **ssthresh** (slow start threshold) determines when to switch from Slow Start to Congestion Avoidance.
On loss event:

- ssthresh = cwnd/2

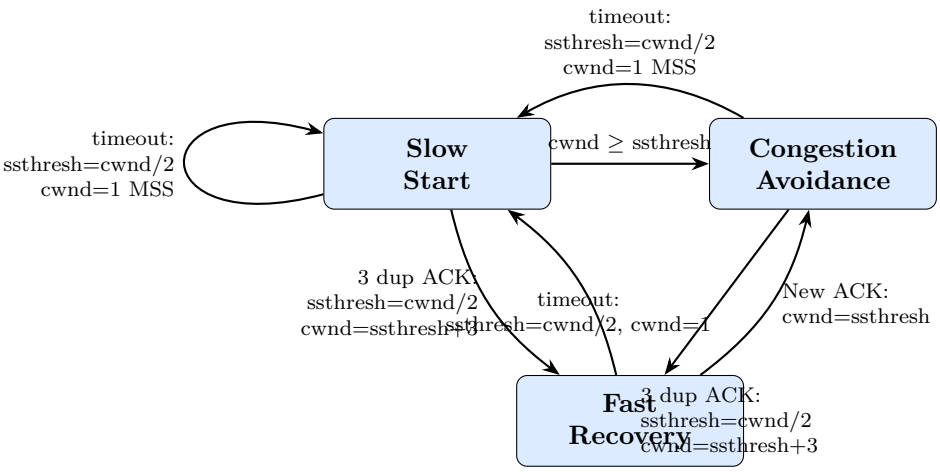
Two types of loss – different responses:
1. Timeout:

- ssthresh = cwnd/2
- cwnd = 1 MSS
- Go back to **Slow Start**

2. Triple duplicate ACK (3 dup ACK):

- ssthresh = cwnd/2
- cwnd = ssthresh + 3 MSS
- Enter **Fast Recovery**

24.5 The Three Mechanisms in TCP Congestion Control



24.6 TCP Tahoe vs. TCP Reno

TCP Tahoe vs. TCP Reno

TCP Tahoe (1988):

- On *all* loss events (both timeout and 3 dup ACK): cwnd = 1 MSS, go to Slow Start

TCP Reno (1997, RFC 2001/2581):

- On **timeout**: cwnd = 1 MSS, go to Slow Start (like Tahoe)
- On **3 dup ACK**: cwnd = ssthresh + 3, go to Fast Recovery (milder reaction)
- Fast Recovery gives better performance – packet loss via dup-ACK means the network is still delivering some packets

24.7 TCP CUBIC

TCP CUBIC – Modern Congestion Control

TCP CUBIC is the **default in Linux** and the most widely used TCP variant for web servers.
Motivation: Classic AIMD (linear increase) is too slow when bandwidth is high.
Idea:

- $W_{\max}$ = send rate when congestion was last detected
- After halving cwnd: increase quickly back toward $W_{\max}$, but slow down near $W_{\max}$
- K = point in time when the window reaches $W_{\max}$ (configurable)
- Increase of W as a function of the **cube** of the distance between current time and K
- Larger increases when far from K, smaller increases near K (cautious)

Advantages:

- Independent of RTT (uses wall-clock time)
- Fair across different RTTs
- Higher throughput than classic TCP on high-speed links

24.8 TCP Fairness

TCP Fairness

Goal: If K TCP sessions share a bottleneck link with capacity R, each should get an average rate of R/K.
Why AIMD gives fairness:

- Additive Increase: both sessions increase with slope 1 (parallel to the equal-share line)
- Multiplicative Decrease: both halve (moves toward the equal-share line)
- Over time the sessions converge toward an equal share of the bandwidth

Note: This is a very theoretical result – in practice, RTT, number of connections per application, and UDP traffic (which has no congestion control) affect fairness.

24.9 The Bottleneck Link

Bottleneck Link and TCP Behavior

TCP (both classic and CUBIC) increases the send rate until packet loss occurs at a router's output – the **bottleneck link**.
Key insights:

- Increased TCP send rate **will not** increase end-to-end throughput when the bottleneck is saturated
- Increased send rate **will** increase measured RTT (due to queues)
- Goal: "Keep the end-to-end pipe exactly full, but not fuller"

25 Evolution of Transport Layer Functionality (3.8)

25.1 Challenges with TCP

Different Scenarios – Different Challenges

Scenario	Challenge
Long, fat pipes (large data transfers)	Many packets "in flight"; loss stops pipeline
Wireless networks	Loss due to noise interpreted as congestion
Long delays	Extremely long RTT
Datacenter networks	Latency-sensitive
Background traffic	Low-priority TCP flows

25.2 QUIC – Quick UDP Internet Connections

QUIC and HTTP/3

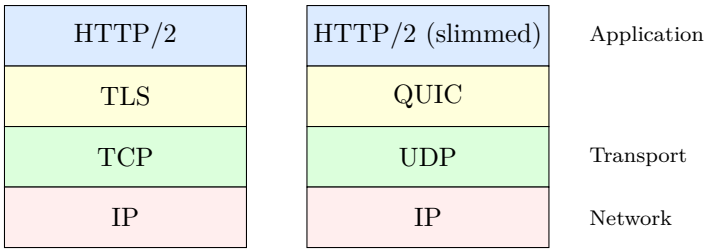
QUIC is an application layer protocol built **over UDP** that provides:

- Reliable data transfer
- Congestion control
- Authentication and encryption
- Connection setup in **1 RTT** (vs. TCP+TLS = 2 serial handshakes)

HTTP/3 = HTTP/2 (slimmed) over QUIC over UDP
Advantages over TCP:

- Multiple application-level "streams" multiplexed over one QUIC connection
- Streams are **independent** – avoids TCP's head-of-line blocking
- Separate reliable transfer and security per stream
- Shared congestion control across streams

HTTP/2 over TCP HTTP/3 over QUIC



25.3 Historical Development of TCP

Year	Milestone	Comment
1980–81	RFC 761 / RFC 793	Original TCP (Connection + Transfer only)
1988	TCP Tahoe (BSD)	Jacobson: Slow Start + Congestion Avoidance
1997	TCP Reno (RFC 2001/2581)	Fast Retransmit + Fast Recovery
2005	BIC TCP	Binary search for optimal window size
2018	TCP CUBIC	Cubic function, default in Linux

26 Summary

Chapter 3 – Key Points

- **Transport layer** provides logical communication between *processes* (not hosts)
- **Multiplexing/demultiplexing:** Port numbers direct to the correct socket (UDP: 2-tuple, TCP: 4-tuple)
- **UDP:** Simple, connectionless, "best effort" – just mux/demux + checksum
- **Reliable data transfer:** Checksum, sequence numbers, ACK, timer, retransmission
- **Pipelining:** GBN (cumulative ACK, retransmit all) vs. SR (individual ACK, buffer out-of-order)
- **TCP:** Reliable, ordered byte stream with flow control, congestion control and connection management
- **TCP congestion control:** Slow Start (exponential), Congestion Avoidance (AIMD), Fast Recovery
- **TCP CUBIC:** Cubic window increase, default in Linux, RTT-independent
- **QUIC/HTTP/3:** Modern alternative over UDP with 1-RTT connection and independent streams

Formula Summary

All Important Formulas – Chapter 3

UDP checksum: 1's complement of the sum of all 16-bit words.
Stop-and-wait utilization:

$$U_{\text{sender}} = \frac{L/R}{RTT + L/R}$$

Pipelining utilization (n packets):

$$U_{\text{sender}} = \frac{n \cdot L/R}{RTT + L/R}$$

MSS:

$$\text{MSS} = \text{MTU} - \text{IP header} - \text{TCP header} = 1500 - 20 - 20 = 1460 \text{ bytes}$$

RTT estimation:

$$\text{EstimatedRTT} = (1-\alpha) \cdot \text{EstimatedRTT} + \alpha \cdot \text{SampleRTT} \qquad (\alpha = 0{,}125)$$
$$\text{DevRTT} = (1-\beta) \cdot \text{DevRTT} + \beta \cdot |\text{SampleRTT} - \text{EstimatedRTT}| \qquad (\beta = 0{,}25)$$

Timeout:

$$\text{TimeoutInterval} = \text{EstimatedRTT} + 4 \cdot \text{DevRTT}$$

TCP flow control:

$$\text{rwnd} = \text{RcvBuffer} - (\text{LastByteRcvd} - \text{LastByteRead})$$

TCP send limit:

$$\text{LastByteSent} - \text{LastByteAcked} \leq \min(\text{cwnd}, \text{rwnd})$$

TCP send rate:

$$\text{rate} \approx \frac{\min(\text{cwnd}, \text{rwnd})}{\text{RTT}} \text{ bytes/sec}$$

Slow Start (per ACK): cwnd = cwnd + MSS (doubling per RTT)
Congestion Avoidance (per ACK): cwnd = cwnd + MSS · (MSS/cwnd) (+1 MSS per RTT)
On timeout: ssthresh = cwnd/2, cwnd = 1 MSS
On 3 dup ACK: ssthresh = cwnd/2, cwnd = ssthresh + 3 MSS
TCP fairness: K sessions, capacity $R \Rightarrow$ each gets R/K
SR window size: $N \leq k/2$ where k = number of sequence numbers

TTM4100 – KTN

Chapter 4: The Network Layer – Data Plane

Computer Networking: A Top-Down Approach, 8th ed.
Kurose & Ross – NTNU Spring 2026

27 The Network Layer – Overview (4.1)

27.1 The Role of the Network Layer

The Network Layer – overview

The network layer transports **segments** from the sending host to the receiving host.

- Sender side: encapsulates segments into **datagrams** (packets)
- Receiver side: delivers segments to the transport layer
- Network layer protocols exist in *all* hosts and routers
- The router examines header fields in all IP datagrams that pass through it

27.2 Forwarding vs. Routing

Exam-relevant: Forwarding vs. Routing (V2025

Forwarding and **routing** are two central concepts that are often confused on the exam:

- **Forwarding:** Moving packets from the router's input port to the correct output port. This is a *local* action in *each individual router* (data plane).
- **Routing:** Determining the route packets should take from source to destination. This is a *global*, *network-wide* process (control plane).

Analogy: Forwarding = taking the correct exit at an intersection. Routing = planning the entire travel route from Oslo to Trondheim.

From exam V2025: "What is the difference between forwarding and routing?"

- Forwarding: move packets from router's input to appropriate router output (local action, data plane)
- Routing: determine route taken by packets from source to destination (global action, control plane)

27.3 The Data Plane vs. The Control Plane

Two planes in the network layer

- **Data plane:**
  - Local, per-router function
  - Determines how datagrams arriving at the input port are forwarded to the output port
  - Forwarding function – operates on a **nanosecond** timescale (hardware)
- **Control plane:**
  - Network-wide logic
  - Determines how datagrams are routed between routers along end-to-end paths
  - Operates on a **millisecond** timescale (software)
  - Two approaches: traditional routing algorithms (per-router) or SDN (Software-Defined Networking)

27.4 Per-router Control Plane vs. SDN

Two control plane approaches

Traditional (per-router control plane):

- Each router has its own routing algorithm
- Routers communicate with each other to build forwarding tables
- The routing algorithm runs in *each* router

SDN (Software-Defined Networking):

- An external (remote) controller computes and distributes forwarding tables
- Routers/switches receive their flow tables from the central controller
- Physically separates the control plane from the data plane

27.5 The Network Service Model

Best Effort service

The Internet's network layer offers only **best effort** service:

- **No** guarantee for bandwidth
- **No** guarantee against packet loss
- **No** guarantee for ordering
- **No** guarantee for delay (timing)
- **No** guarantee for minimal delay between packets (jitter)

Comparison with other network architectures

Other network architectures (such as ATM) offered stronger guarantees:

- **ATM CBR** (Constant Bit Rate): guaranteed bandwidth, ordering, timing, no loss
- **ATM ABR** (Available Bit Rate): guaranteed minimum bandwidth, no loss

The Internet's "best effort" has nevertheless won because its simplicity allows innovation at the edges.

28 What is Inside a Router? (4.2)

28.1 Router Architecture – Overview

Generic router architecture

A router consists of four main components:

1. **Input ports:** Physical reception, link layer protocol, lookup in forwarding table, queue
2. **Switching fabric:** Connects input ports to output ports
3. **Output ports:** Buffering, queue discipline, link layer protocol, physical transmission
4. **Routing processor:** Runs routing protocols, management (control plane)

Important distinction:

- Control plane (routing processor): software, milliseconds
- Data plane (input ports, switching fabric, output ports): hardware, nanoseconds

28.2 Input Port Functions

The three steps of the input port

1. **Line termination:** Physical layer – bit-level reception
2. **Link layer protocol:** E.g. Ethernet (receive frame)
3. **Lookup and forwarding:** Use header values to look up the output port in the forwarding table ("match plus action")

Decentralized switching:

- The forwarding table is copied to the input port's memory
- Goal: complete input port processing at **line speed**
- **Input port queuing:** Occurs when datagrams arrive faster than the switching speed

28.2.1 Destination-based vs. Generalized Forwarding

- **Destination-based forwarding:** Forwarding based only on the destination IP address (traditional)
- **Generalized forwarding:** Forwarding based on an arbitrary set of header values (SDN/OpenFlow)

28.3 Longest Prefix Match

Exam-relevant: Longest Prefix Match (V2023 Q4)

Longest prefix match: When looking up in the forwarding table for a given destination address, use the entry with the *longest* address prefix that matches the destination address.

Example from lecture:

Destination address range	Link Interface
`11001000 00010111 00010***`	0
`11001000 00010111 00011000`	1
`11001000 00010111 00011***`	2
otherwise	3

For the address `11001000 00010111 00011000 10101010`:

- Matches both interface 1 (full match on 24 bits) and interface 2 (21 bits)
- Selects interface **1** because it has the longest prefix match

From exam V2023 (Q4): The problem gave a forwarding table with 5 CIDR entries and asked which output port 5 different IP addresses would be forwarded to. The solution requires binary conversion and longest prefix match.

Implementation: Often implemented with **TCAM** (Ternary Content Addressable Memory) – provides lookup in *one* clock cycle, regardless of table size.

28.4 Switching Fabric

Three types of switching fabric

The switching fabric transfers packets from the input port to the correct output port.
Switching rate: The speed at which packets can be transferred from input to output.

- Often measured as a multiple of the input/output line speed
- With N input ports: desirable switching rate $= N \times R$ (line speed)

Three main types:

1. **Via memory:** First generation. Packet is copied to system memory via CPU. Limited by memory bandwidth (2 bus crossings per datagram).
2. **Via bus:** Datagram from input port memory to output port memory via a shared bus. *Bus contention* – speed limited by bus bandwidth. E.g. Cisco 5600: 32 Gbps.
3. **Via interconnection network:** Crossbar, Clos network. Can transfer multiple packets in parallel. Fragments datagrams into cells, switches the cells, reassembles. Scales with parallel switching "planes" (e.g. Cisco CRS: 100s Tbps).

28.5 Queues and Delays in the Router

28.5.1 Input port queuing

HOL blocking

If the switching fabric is slower than the sum of the input ports, queues form at the input ports.

- Queue delay and packet loss due to buffer overflow
- **Head-of-the-Line (HOL) blocking:** A datagram at the front of the queue blocks other datagrams in the same queue from advancing, even if their output port is free

28.5.2 Output port queuing

Output port queuing – important!

Buffering is required when datagrams arrive from the switching fabric faster than the line transmission speed.

- Queue delay and packet loss due to buffer overflow
- **Drop policy:** Which packets are dropped when the buffer is full?
- **Scheduling discipline:** Selects among queued datagrams for transmission

28.5.3 Buffer Sizing

Rule of thumb for buffer size

RFC 3439:

$$B = RTT \times C$$

where $RTT \approx 250$ ms (typical) and C = link capacity.

E.g. $C = 10$ Gbps $\Rightarrow$ buffer = 2,5 Gbit.

More recent recommendation: With N concurrent flows:

$$B = \frac{RTT \times C}{\sqrt{N}}$$

Note: Too much buffering can also be problematic – causes long RTTs ("bufferbloat"), poor performance for real-time applications, and sluggish TCP response.

28.6 Buffer Management

Buffer management – drop and marking

Drop policy (when the buffer is full):

- **Tail drop:** Drop the arriving packet (simplest)
- **Priority drop:** Drop based on priority

Marking (congestion signal):

- ECN (Explicit Congestion Notification)
- RED (Random Early Detection)

28.7 Packet Scheduling

Exam-relevant: Queue disciplines (V2023 Q1.3.5)

Four common scheduling methods:

1. **FCFS / FIFO:** First in, first out. Simplest method.
2. **Priority scheduling:** Traffic is classified, queued by class. Always send from the highest priority queue first. FCFS within each class.
3. **Round Robin (RR):** Traffic is classified. The server cyclically scans the class queues, sending *one* packet from each class at a time.
4. **Weighted Fair Queuing (WFQ):** Generalized Round Robin. Each class i has weight w_i, and receives a share of:

$$\frac{w_i}{\sum_j w_j}$$

   Provides a minimum bandwidth guarantee per traffic class.

From exam V2023 (Q1.3.5): "A FIFO queue holds k packets when a new packet arrives. Considering queuing delay only, how many packets must be transmitted before the newly arrived packet can be transmitted?"
Answer: k packets (alternative c). The new packet must wait until all k packets ahead in the queue have been sent.

29 Internet Protocol (IP) (4.3)

29.1 IPv4 Datagram Format

The IPv4 datagram header

The IPv4 header is normally **20 bytes** (without options):

Field	Description
Version (4 bit)	IP version (4 for IPv4)
Header length (4 bit)	Header length in 32-bit words
Type of service (8 bit)	Differentiated service (DSCP, ECN)
Total length (16 bit)	Total datagram length in bytes
Identifier, flags, offset	Used for fragmentation/reassembly
TTL (8 bit)	Time to live – max number of hops
Upper layer protocol (8 bit)	Protocol in payload (6=TCP, 17=UDP)
Header checksum (16 bit)	Checksum for the header
Source IP (32 bit)	Sender's IP address
Destination IP (32 bit)	Receiver's IP address
Options (variable)	Timestamp, record route, etc.

Overhead

Minimum overhead for TCP over IP:

$$\text{Overhead} = \underbrace{20 \text{ bytes}}_{\text{IP header}} + \underbrace{20 \text{ bytes}}_{\text{TCP header}} = 40 \text{ bytes}$$

This means that small application data incurs a large relative overhead.

29.2 IP Addressing

IP addresses – basics

- An IP address is **32 bits** = 4 bytes, written in "dotted decimal" notation
- Example: `223.1.1.1` = `11011111 00000001 00000001 00000001`
- IP addresses belong to **interfaces**, not hosts
- A router typically has multiple interfaces, and thus multiple IP addresses
- A host normally has one or two interfaces (wired and/or WiFi)

29.3 Subnets

Exam-relevant: Subnets and CIDR (V2025

Subnet: A group of devices that can reach each other without going through a router.
CIDR (Classless InterDomain Routing):

- Address format: `a.b.c.d/x`, where `x` is the number of bits in the network part (subnet prefix)
- The first `x` bits = network part (subnet part)
- The remaining $32 - x$ bits = host part

Calculating the number of addresses and hosts:

- Total number of addresses in a subnet: 2^{32-x}
- **Usable host addresses:** $2^{32-x} - 2$ (subtract network address and broadcast address)

Broadcast address: All host bits set to 1.

From exam V2025:

- "An organization is assigned the address block `208.27.1.0/22` and divides it into 12 subnets. How many usable host addresses?"
- Solution: /22 gives $2^{10} = 1024$ addresses. 12 subnets require $\lceil \log_2 12 \rceil = 4$ bits, so each subnet gets $2^{10-4} = 2^6 = 64$ addresses, i.e. $64 - 2 = \mathbf{62}$ usable host addresses.
- "What is the broadcast address for `172.18.16.0/21`?"
- Solution: /21 means 21 network bits, 11 host bits. Set all host bits to 1: `172.18.23.255`

How to calculate the broadcast address

Step by step:

1. Write the network address in binary

2. Identify the host part (the last $32 - x$ bits)
3. Set *all* host bits to 1
4. Convert back to decimal

Example: `172.18.16.0/21`

- Binary: `10101100.00010010.00010000.00000000`
- Host part (11 bits): the last 11 bits
- Set to 1: `10101100.00010010.00010111.11111111`
- Decimal: `172.18.23.255`

29.4 IP Addresses: Binary Conversion

Exam-relevant: Binary conversion (V2023 Q1.3.1)

From exam V2023: "What is `193.32.216.9` in binary?"
Solution: Convert each octet separately:

- $193 = 128 + 64 + 1 =$ `11000001`
- $32 =$ `00100000`
- $216 = 128 + 64 + 16 + 8 =$ `11011000`
- $9 = 8 + 1 =$ `00001001`

Answer: `11000001.00100000.11011000.00001001`

29.5 Subnet Allocation from a CIDR Block

Exam-relevant: Subnet allocation (V2023 Q1.3.2)

From exam V2023: "An organization is granted the address block `193.32.216.0/22`. Which of the following addresses belong to this block?"
Solution method:

1. Determine the network part: the first 22 bits
2. All addresses with the same 22-bit prefix belong to the block
3. The address range is `193.32.216.0` – `193.32.219.255`

29.6 Hierarchical Addressing and Route Aggregation

Route aggregation

Hierarchical addressing enables efficient announcement of routing information:

- An ISP is assigned a large address block (e.g. `200.23.16.0/20`)
- The ISP divides the block into smaller parts for organizations (e.g. 8 blocks of /23)
- To the rest of the Internet, the ISP announces only *one* aggregated route (`200.23.16.0/20`)
- This dramatically reduces the size of routing tables

More specific routes: If an organization switches ISPs, the new ISP can announce a more specific route (e.g. /23). Due to longest prefix match, traffic will go to the correct ISP.

29.7 How Does a Host Get Its IP Address?

Two methods

- **Manual configuration:** A system administrator configures the address directly
- **DHCP (Dynamic Host Configuration Protocol):** Automatic assignment from a server

29.8 DHCP – Dynamic Host Configuration Protocol

Exam-relevant: DHCP (V2025

DHCP is a "plug-and-play" protocol that lets a host automatically obtain an IP address.
DHCP uses UDP: Client on port 68, server on port 67.
Four-step process (DORA):

1. **DHCP Discover:** The client sends a broadcast (`255.255.255.255`) to find DHCP servers.
   - SIP=`0.0.0.0`, DIP=`255.255.255.255`
   - Ethernet DA=`FF:FF:FF:FF:FF:FF`
2. **DHCP Offer:** Server(s) respond with an offered IP address, subnet mask, DNS, router, lease time.
3. **DHCP Request:** The client selects an offer and broadcasts a request with the server ID.
   - SIP=`0.0.0.0` (still), DIP=`255.255.255.255` (broadcast)
   - Contains `DHCP Server Identifier` for the chosen server
   - Broadcast ensures that other servers know they were *not* selected
4. **DHCP ACK:** The chosen server confirms the assignment with all configuration parameters.

DHCP provides more than just an IP address:

- Address of the first-hop router (default gateway)
- Name and IP address of the DNS server
- Network mask (subnet mask)

Protocol stack: DHCP message is encapsulated in UDP → IP → Ethernet.

From exam V2023 (Q1.3.3): "Which of these are related to DHCP?"

- a) Provides an IP address to a host – **correct**
- b) Uses UDP – **correct**
- d) Address of default gateway is provided – **correct**

29.9 NAT – Network Address Translation

NAT – overview

NAT allows multiple devices in a local network to share *one* public IP address.
How it works:

- All datagrams leaving the local network get the *same* source IP (the NAT router's public address), but *different* source port numbers
- The NAT router has a **translation table** (NAT translation table) that maps: (source IP, source port) ↔ (NAT IP, NAT port)
- Incoming packets are translated back using the table

Private address ranges (RFC 1918):

- `10.0.0.0/8` – 16.7 million addresses
- `172.16.0.0/12` – approx. 1 million addresses
- `192.168.0.0/16` – 65 536 addresses

16-bit port numbers allow for approximately 65 000 simultaneous connections with a single IP address.

Exam-relevant: NAT and private addresses (V2025)

From exam V2025 (true/false):

- "IP addresses are always globally unique" – **FALSE**. Private addresses (10.x.x.x, 172.16.x.x, 192.168.x.x) are reused in many networks, and are only unique within their local network. NAT is the reason this works.

NAT controversy

NAT is controversial because:

- Routers should only operate up to layer 3 – NAT manipulates port numbers (layer 4)
- Violates the end-to-end principle
- Makes it difficult for external hosts to initiate contact with devices behind NAT
- IPv6 was intended to make NAT unnecessary (enough addresses for everyone)

29.10 IPv6

IPv6 – main differences from IPv4

Motivation: The IPv4 address space (32 bits = approx. 4.3 billion) is exhausted. ICANN allocated the last block in 2011.
IPv6 datagram format:

- **128-bit** addresses (vs. 32-bit in IPv4)
- Fixed header size of **40 bytes** (vs. variable in IPv4)
- **No fragmentation** by routers – only end systems can fragment
- **No header checksum** – faster processing
- **New field: Flow Label** – identifies packets in the same flow for special treatment
- `Next Header` field replaces both "Protocol" and "Options" from IPv4

- ICMPv6: new version of ICMP with additional message types

Exam-relevant: IPv4 vs. IPv6 (V2023 Q1.3.4)

From exam V2023: "Which of these are correct about IPv4/IPv6?"

- a) IPv6 has a larger address space – **correct** (128 vs. 32 bit)
- b) IPv6 has a fixed-length header – **correct** (40 bytes)
- d) IPv6 does not allow fragmentation by routers – **correct**
- e) IPv6 removes the header checksum field – **correct**

From exam V2025 (true/false):

- "IP guarantees that a packet will be delivered in order" – **FALSE**. IP is "best effort" and provides no guarantee for ordering. That is TCP's job.

Fields removed in IPv6 (compared to IPv4)

- **Header checksum:** Removed for faster processing (checksum is already computed in the transport and link layers)
- **Fragmentation/reassembly:** Routers do not fragment – the sender must use Path MTU Discovery
- **Options:** Replaced by the "Next Header" mechanism
- **Header Length:** Unnecessary with a fixed header size

29.11 Transition from IPv4 to IPv6: Tunneling

Tunneling – IPv6 in IPv4

Since not all routers can be upgraded simultaneously, **tunneling** is used:

- The IPv6 datagram is encapsulated as *payload* in an IPv4 datagram
- IPv4 routers along the tunnel see it as a regular IPv4 packet
- At the end of the tunnel, the IPv6 datagram is unpacked again

Adoption: Approx. 40–50% of traffic uses IPv6 (as of approx. 2023).

30 Architectural Principles (4.5)

30.1 The IP Hourglass Architecture

IP as the "narrow waist"

The Internet architecture has an hourglass shape:

- **Above IP:** Many applications and transport protocols (HTTP, SMTP, TCP, UDP, ...)
- **IP:** The only network layer – "narrow waist" that everything must pass through
- **Below IP:** Many link layer technologies (Ethernet, WiFi, fiber, ...)

All devices on the Internet must support IP – it is what binds everything together.

30.2 Middleboxes

Middleboxes

Devices on the network path that perform functions beyond standard IP forwarding:

- **NAT boxes:** Address translation
- **Firewalls:** Traffic filtering
- **Load balancers:** Distribute traffic across multiple servers
- **Caches/proxies:** Content caching

Middleboxes are controversial because they potentially violate the **end-to-end principle**.

30.3 The End-to-End Argument

End-to-End Argument

The end-to-end principle: Functionality should be implemented in the *end systems* (hosts), not in the network.

- The network should be kept **simple** – only forwarding packets
- Complexity (reliability, security, etc.) should reside in the end systems
- Rationale: End systems have the most context about what the application needs

Historical development:

- 20th century (telephone network): Intelligence in the network's switches
- Internet (pre-2005): Intelligence at the edges (end systems)
- Internet (post-2005): Programmable network devices + massive infrastructure at the edges

31 IP Addressing: ICANN and Address Allocation

How does an ISP get addresses?

ICANN (Internet Corporation for Assigned Names and Numbers):

- Allocates IP address blocks through 5 **regional registries** (RIRs)
- Administers the DNS root zone
- Delegates TLD administration (.com, .no, .edu, etc.)

Address allocation hierarchy:

1. ICANN → Regional registries (RIPE NCC for Europe)
2. Regional registries → ISPs
3. ISPs → Organizations/customers (via CIDR blocks)
4. Organizations → Individual hosts (via DHCP or manually)

32 Summary and Exam Tips

Frequently tested exam topics – Chapter 4

Based on exams from V2023 and V2025:

1. **Forwarding vs. Routing:** Clear definition and distinction between local/global, data plane/control plane
2. **Subnet calculation:** Being able to calculate the number of hosts, broadcast address, check whether an address belongs to a subnet (CIDR)
3. **Binary conversion:** Converting between decimal and binary IP notation
4. **Longest prefix match:** Looking up in a forwarding table with CIDR entries
5. **DHCP:** The 4-step process (DORA), use of broadcast, UDP ports, what DHCP provides
6. **IPv4 vs. IPv6:** Differences in header, address length, fragmentation, checksum
7. **NAT:** How it works, private addresses, consequences for "globally unique" addresses
8. **Queue disciplines:** FIFO, priority, round robin, WFQ – especially FIFO calculations
9. **Best effort:** IP provides no guarantees (ordering, delivery, delay)

Common pitfalls on the exam

- **"IP guarantees ordering"** – **Wrong!** IP is best effort, no ordering guarantee
- **"IP addresses are always globally unique"** – **Wrong!** Private addresses (NAT) are reused
- **Forwarding ≠ Routing** – Do not confuse these terms
- **Don't forget to subtract 2** when calculating usable host addresses (network address + broadcast)
- **Broadcast address:** Remember to set *all* host bits to 1, not just the last octet
- **DHCP Request is broadcast**, not unicast (so other servers know they were not selected)

Important formulas – Chapter 4

$$\text{Number of addresses in subnet } /x = 2^{32-x} \tag{6}$$

$$\text{Usable host addresses} = 2^{32-x} - 2 \tag{7}$$

$$\text{Number of subnet bits for } n \text{ subnets} = \lceil \log_2 n \rceil \tag{8}$$

$$\text{Buffer size (RFC 3439)} = RTT \times C \tag{9}$$

$$\text{Buffer size (with } N \text{ flows)} = \frac{RTT \times C}{\sqrt{N}} \tag{10}$$

$$\text{WFQ share for class } i = \frac{w_i}{\sum_j w_j} \tag{11}$$

$$\text{IP+TCP overhead} = 20 + 20 = 40 \text{ bytes} \tag{12}$$

TTM4100 – KTN

Chapter 5: Network Layer – Control Plane

Computer Networking: A Top-Down Approach, 8th ed.
Kurose & Ross – NTNU Spring 2026

33 Introduction to the Control Plane (5.1)

33.1 Network-Layer Functions Recap

Forwarding vs. Routing – Two Key Functions

The network layer has two fundamental functions:

- **Forwarding:** Move packets from a router's input port to the appropriate output port. This is a *local*, per-router action belonging to the **data plane**.
- **Routing:** Determine the route (path) that packets take from source to destination. This is a *network-wide*, global action belonging to the **control plane**.

Analogy: Forwarding = taking the correct exit at a roundabout. Routing = planning the entire trip from city A to city B.

33.2 Two Approaches to the Control Plane

Per-Router Control vs. SDN

There are two main approaches to structuring the network control plane:

1. Per-Router Control (Traditional):

- Individual routing algorithm components run in *each and every router*
- Routers interact with each other in the control plane to compute forwarding tables
- Each router independently runs a routing algorithm (e.g., OSPF, BGP)

2. Logically Centralized Control (SDN):

- A remote controller computes and distributes forwarding tables to routers
- Routers/switches receive their flow tables from the central controller
- Physically separates the control plane from the data plane

34 Routing Algorithms (5.2)

34.1 Goal and Overview

Routing Protocol Goal

The goal of a routing protocol is to determine "good" paths (routes) from sending hosts to receiving hosts through the network of routers.

- A **path** is a sequence of routers that packets traverse from source to destination
- "Good" typically means: least cost, fastest, or least congested
- Routing is considered a "top-10" networking challenge

34.2 Graph Abstraction

Network as a Graph

A network can be modelled as a graph $G = (N, E)$:

- N = set of routers (nodes): $\{u, v, w, x, y, z\}$
- E = set of links (edges): $\{(u, v), (u, x), (v, x), \ldots\}$
- Each edge has a **cost** $c(x, y)$ (e.g., bandwidth, delay, monetary cost, congestion)
- $c(x, y) = \infty$ if (x, y) is not a direct link
- The cost of a path $(x_1, x_2, \ldots, x_p) = c(x_1, x_2) + c(x_2, x_3) + \cdots + c(x_{p-1}, x_p)$

Key question: What is the least-cost path between two nodes?

34.3 Routing Algorithm Classification

Exam-relevant: Classification of Routing Algorithms

Routing algorithms can be classified along two axes:
Axis 1 – Information scope:

- **Global (link state):** All routers have *complete* topology and link cost information. Each router computes shortest paths independently using global knowledge.
- **Decentralized (distance vector):** Iterative process of computation and exchange of information with neighbors. Routers initially only know link costs to their directly attached neighbors.

Axis 2 – How fast do routes change?

- **Static:** Routes change slowly over time (manual configuration)
- **Dynamic:** Routes change more quickly – periodic updates or in response to link cost changes

35 Link-State Routing Algorithm: Dijkstra's Algorithm (5.2.1)

35.1 Overview

Link-State Algorithm – Key Properties

In a **link-state** algorithm:

- All routers have *complete* knowledge of the network topology and all link costs
- This is achieved by **link-state broadcast** – each router broadcasts its link information to *all* other routers in the network (flooding)
- Each router then independently runs **Dijkstra's algorithm** to compute shortest paths from itself to all other nodes
- The result is a **shortest-path tree** rooted at the computing router, which determines the forwarding table

35.2 Dijkstra's Algorithm

Dijkstra's Algorithm – Notation and Procedure

Notation:

- $c(x, y)$: direct link cost from node x to node y; $= \infty$ if not direct neighbors
- $D(v)$: current estimate of the least-cost path from source to v
- $p(v)$: predecessor node along the current least-cost path to v
- N': set of nodes whose least-cost path is definitively known

Algorithm:

1. **Initialize:** $N' = \{u\}$ (source node). For all neighbors v of u: $D(v) = c(u, v)$. For all non-neighbors: $D(v) = \infty$.
2. **Loop:** Find node $w \notin N'$ such that $D(w)$ is minimum. Add w to N'.
3. **Update:** For each neighbor v of w that is not in N':

$$D(v) = \min\left(D(v),\ D(w) + c(w, v)\right)$$

   If the path through w is better, update $p(v) = w$.
4. **Repeat** step 2–3 until all nodes are in N'.

Complexity: $O(n^2)$ for n nodes (can be improved to $O(n \log n)$ with a priority queue).

Exam-relevant: Dijkstra's Algorithm Walkthrough

Example: Given the following network (from textbook Figure 5.3):

Step	N'	$D(v), p(v)$	$D(w), p(w)$	$D(x), p(x)$	$D(y), p(y)$	$D(z), p(z)$
0	$\{u\}$	$2, u$	$5, u$	$1, u$	∞	∞
1	$\{u, x\}$	$2, u$	$4, x$	–	$2, x$	∞
2	$\{u, x, v\}$	–	$4, x$	–	$2, x$	∞
3	$\{u, x, v, y\}$	–	$4, x$	–	–	$4, y$
4	$\{u, x, v, y, w\}$	–	–	–	–	$4, y$
5	$\{u, x, v, y, w, z\}$	–	–	–	–	–

At each step, the node with the smallest $D(\cdot)$ not yet in N' is added, and all neighbor distances are updated.

Tip: On the exam, you may be asked to run Dijkstra's algorithm on a given graph. Practice filling out a table like the one above step by step.

35.3 Oscillations in Link-State Routing

Oscillation Problem

When link costs are **traffic-dependent** (e.g., proportional to the traffic load on a link), Dijkstra's algorithm can cause **oscillations**:

- All routers compute new routes based on current traffic
- Traffic shifts to the new "best" path, making it congested
- In the next iteration, the old path becomes least-cost again

- Routes keep oscillating back and forth

Mitigation: Routers can randomize the timing of their link-state broadcasts so they don't all recompute routes simultaneously.

36 Distance Vector Routing Algorithm (5.2.2)

36.1 Bellman-Ford Equation

Bellman-Ford Equation (Dynamic Programming)

The Bellman-Ford equation defines the least-cost path recursively:

$$d_x(y) = \min_v \{c(x, v) + d_v(y)\}$$

where:

- $d_x(y)$: cost of the least-cost path from x to y
- $\min_v$: minimum taken over all neighbors v of x
- $c(x, v)$: cost of the direct link from x to neighbor v
- $d_v(y)$: cost of the least-cost path from v to y (as reported by neighbor v)

The neighbor v^* that achieves the minimum becomes the **next hop** in the forwarding table for destination y.

36.2 Distance Vector Algorithm

DV Algorithm – How It Works

The distance vector algorithm is:

- **Iterative:** Continues until no more information is exchanged
- **Asynchronous:** Nodes do not need to operate in lockstep
- **Distributed:** Each node only communicates with its direct neighbors

Process:

1. Each node x maintains a **distance vector** $\mathbf{D}_x = [D_x(y) : y \in N]$ – its estimate of the least cost to every other node
2. Each node x maintains its neighbors' distance vectors: $\mathbf{D}_v$ for each neighbor v
3. Periodically, or when its local DV changes, each node sends its DV to its neighbors
4. When node x receives a new DV from a neighbor v, it updates its own DV using Bellman-Ford:

$$D_x(y) = \min_v \{c(x, v) + D_v(y)\} \quad \text{for each node } y$$

5. If $D_x(y)$ changes, x notifies its neighbors

The algorithm converges to the correct least-cost paths. Each node only needs to know its own link costs and its neighbors' distance vectors.

36.3 Count-to-Infinity Problem

Exam-relevant: Count-to-Infinity Problem

A major issue with distance vector routing is the **count-to-infinity problem**:
Scenario: Suppose a link cost increases dramatically (e.g., y–x cost goes from 4 to 60):

- Node y sees the cost to x via direct link is now 60
- But y thinks it can reach x via z for cost 5 (because z's old DV says $D_z(x) = 5$)
- So y sets $D_y(x) = c(y, z) + D_z(x) = 1 + 5 = 6$
- When z gets y's update, z updates $D_z(x) = c(z, y) + D_y(x) = 1 + 6 = 7$
- This continues: $D_y(x) = 8$, $D_z(x) = 9$, $D_y(x) = 10$, ...
- It takes **44 iterations** to converge to the correct value of 60!

This is called "bad news travels slowly." Good news (a cost decrease) propagates in one iteration.

Mitigation – Poisoned Reverse:

- If z routes to x through y, then z tells y that $D_z(x) = \infty$
- This prevents y from routing to x through z (avoiding the loop)
- **Limitation:** Poisoned reverse only solves loops involving two nodes. Loops involving three or more nodes are *not* detected.

37 Comparison: Link State vs. Distance Vector (5.2.3)

Exam-relevant: LS vs. DV Comparison

Property	Link State (LS)	Distance Vector (DV)
Message complexity	$O(n \cdot E)$ messages sent (each of n nodes broadcasts to E neighbors)	Exchange only between neighbors; convergence time varies
Speed of convergence	$O(n^2)$ algorithm; may have oscillations with traffic-dependent costs	Convergence time varies; count-to-infinity problem; routing loops possible
Robustness (what if a router malfunctions?)	A router can advertise *incorrect link cost*; each node computes its own table – errors are somewhat contained	A router can advertise *incorrect path cost*; errors propagate through the network ("rumors")

Key takeaway:

- LS: each node computes independently based on global information – errors are localized
- DV: nodes rely on neighbors' information – a single incorrect value can propagate everywhere

38 Intra-AS Routing: OSPF (5.3)

38.1 Autonomous Systems and Scalability

Why Hierarchical Routing?

A flat routing scheme where all routers run the same algorithm does not scale because:

- **Scale:** Billions of destinations – storing all destinations in routing tables is infeasible; routing message exchange would overwhelm links
- **Administrative autonomy:** Each network admin wants to control routing within their own network

Solution: Autonomous Systems (AS)

- The Internet is divided into **autonomous systems** (ASes)
- An AS is a collection of routers under the *same administrative control* (e.g., an ISP, a university, a company)
- **Intra-AS routing:** Routing *within* an AS (all routers run the same intra-AS protocol)
- **Inter-AS routing:** Routing *between* ASes (handled by a separate protocol – BGP)
- **Gateway routers:** Routers at the "edge" of an AS that connect to routers in other ASes

38.2 OSPF – Open Shortest Path First

OSPF – Overview

OSPF is the most widely used **intra-AS** (interior gateway) routing protocol. It uses a **link-state** algorithm.
Key characteristics:

- Uses **Dijkstra's algorithm** to compute shortest-path tree
- Each router floods **link-state advertisements (LSAs)** to *all* other routers in the AS
- LSAs contain: one entry per directly attached neighbor, with the link cost
- OSPF messages are carried directly over **IP** (protocol number 89) – not TCP or UDP
- "Open" means the protocol specification is publicly available

Advanced features of OSPF:

- **Security:** All OSPF messages can be authenticated (prevents malicious intrusion)
- **Multiple same-cost paths:** When multiple paths have equal cost, OSPF allows traffic to be spread across them (ECMP – Equal-Cost Multi-Path)
- **Integrated unicast and multicast:** Multicast OSPF (MOSPF) uses the same topology database
- **Hierarchical OSPF:** Support for large ASes through *areas*

38.3 Hierarchical OSPF

Two-Level Hierarchy in OSPF

For large ASes, OSPF supports a two-level hierarchy:

- **Local areas:** Each area runs its own OSPF link-state algorithm. Routers only flood LSAs *within* their area.
- **Backbone area (Area 0):** Connects all local areas. Used to route traffic *between* areas.
- **Area border routers:** Belong to both a local area and the backbone. They summarize routing information from their area and advertise it to the backbone.
- **Backbone routers:** Run OSPF routing limited to the backbone area.
- **Boundary routers:** Connect to other ASes (run inter-AS routing as well).

39 Inter-AS Routing: BGP (5.4)

39.1 BGP Overview

BGP – Border Gateway Protocol

BGP is the **inter-AS** (inter-domain) routing protocol – the "glue that holds the Internet together."
BGP provides each AS with a means to:

- **eBGP** (external BGP): Obtain subnet reachability information from neighboring ASes
- **iBGP** (internal BGP): Propagate reachability information to all routers *within* the AS
- Determine "good" routes to other networks based on reachability information and **policy**

Key facts:

- BGP allows each subnet to advertise its existence to the rest of the Internet: "I am here, and here is how to reach me"
- BGP runs over **TCP** (port 179) – reliable message exchange
- BGP sessions are called **BGP connections** (semi-permanent TCP connections)

39.2 BGP Basics: Path Attributes and Routes

BGP Route Advertisements

When a router advertises a prefix via BGP, it includes **BGP attributes**. A prefix together with its attributes is called a **route**.
Two important attributes:

- **AS-PATH:** The list of ASes through which the advertisement has passed. Example: AS2 AS17 AS67. Used for loop detection – if a router sees its own AS in the path, it rejects the advertisement.
- **NEXT-HOP:** The IP address of the router interface that begins the AS-PATH. Indicates the next-hop router to reach the destination.

Policy-based routing:

- A gateway receiving a route advertisement uses its **import policy** to decide whether to accept or filter the route
- An AS uses its **export policy** to decide whether to advertise routes to neighboring ASes

39.3 BGP Route Selection

Exam-relevant: BGP Route Selection Rules

When a router learns about multiple routes to the same prefix, it selects the best route using the following criteria (in order of priority):

1. **Local preference value** (policy decision): Highest local preference wins. Set by the AS administrator.
2. **Shortest AS-PATH:** Fewest number of ASes traversed.
3. **Closest NEXT-HOP router** (hot potato routing): Choose the route whose NEXT-HOP router has the lowest intra-AS cost from the current router.
4. **Additional criteria:** BGP identifiers and other tie-breakers.

Hot Potato Routing: The router chooses the gateway that has the *smallest least cost* (determined by the intra-AS algorithm) to reach, regardless of the inter-AS cost. The idea: "get rid of the packet as quickly as possible" (within the AS).

39.4 Why Different Intra-AS and Inter-AS Routing?

Intra-AS vs. Inter-AS: Different Goals

Policy:

- **Inter-AS (BGP):** Policy is dominant. An admin wants control over how traffic is routed and who routes through their network. BGP carries path attributes and allows filtering.
- **Intra-AS (OSPF):** Single admin, so policy is less of an issue. Focus on performance.

Scale:

- Hierarchical routing saves table size and reduces update traffic
- Intra-AS can focus on performance; inter-AS may sacrifice performance for policy

Performance:

- Intra-AS routing can focus on finding optimal paths
- Inter-AS routing is constrained by policies (may not use shortest path)

40 ICMP – Internet Control Message Protocol (5.6)

40.1 ICMP Overview

ICMP – Purpose and Position

ICMP is used by hosts and routers to communicate **network-level information**:

- **Error reporting:** Unreachable host, network, port, or protocol
- **Echo request/reply:** Used by `ping`
- ICMP is a network-layer protocol but sits "above" IP:
  - ICMP messages are carried as the **payload** of IP datagrams
- Each ICMP message has a **type** and a **code** field, plus the first 8 bytes of the IP datagram that caused the error

40.2 ICMP Message Types

Exam-relevant: ICMP Message Types and Codes

Type	Code	Description
0	0	Echo reply (ping response)
3	0	Destination network unreachable
3	1	Destination host unreachable
3	2	Destination protocol unreachable
3	3	Destination port unreachable
3	6	Destination network unknown
3	7	Destination host unknown
4	0	Source quench (congestion control – not used)
8	0	Echo request (ping)
9	0	Route advertisement
10	0	Router discovery
11	0	TTL expired
12	0	Bad IP header

Key types to remember:

- Type 8/0: Echo request and reply (`ping`)
- Type 11: TTL expired (used by `traceroute`)
- Type 3: Destination unreachable (various codes for network, host, port, protocol)

40.3 Traceroute and ICMP

Exam-relevant: How Traceroute Works

`Traceroute` uses ICMP to discover the path packets take through the network:
How it works:

1. The source sends sets of **UDP segments** to the destination
2. The 1st set has **TTL = 1**, the 2nd set has **TTL = 2**, etc. (typically 3 probes per TTL value)
3. When a datagram arrives at the nth router, its TTL expires (reaches 0)
4. The router **discards** the datagram and sends back an **ICMP "TTL expired" message** (type 11, code 0) to the source

5. The ICMP message includes the router's name and IP address
6. When the ICMP message arrives at the source, it records the **RTT** (round-trip time)

Stopping criteria:

- Eventually a UDP segment reaches the destination host
- The destination returns an ICMP **"port unreachable"** message (type 3, code 3) because the destination port is unlikely to be in use
- The source receives this message and knows the destination has been reached – it stops

Output: For each TTL value, traceroute displays the router IP address and three RTT measurements.

41 SDN – Software-Defined Networking (5.5)

41.1 SDN Overview

SDN Control Plane

In Software-Defined Networking, the control plane is **logically centralized**:

- A remote **SDN controller** computes and installs forwarding tables (flow tables) in each router/switch
- The controller has a **global view** of the network (topology, traffic, link states)
- Routers/switches become simple forwarding devices ("dumb" data plane)
- The controller is implemented as a distributed system for reliability and scalability

Why SDN?

- Easier network management – one central point of configuration
- Table-based forwarding allows "programming" of routers via an open API (e.g., OpenFlow)
- Enables new routing algorithms without changing router hardware/software
- Centralized control makes it easier to implement complex policies, load balancing, and traffic engineering

41.2 SDN Architecture

SDN Layers

The SDN architecture has three main layers:

1. **Data plane (SDN-controlled switches):**
   - Fast, simple forwarding devices
   - Flow table computed and installed by the controller
   - API for communication with controller (e.g., OpenFlow)
2. **SDN Controller (network OS):**
   - Maintains network state information
   - Interacts with switches below (southbound API) and applications above (northbound API)

- Implemented as a distributed system (e.g., ONOS, OpenDaylight)

3. **Network-control applications:**
   - "Brain" of the control – routing, access control, load balancing
   - Implemented as programs on top of the controller using the northbound API
   - Can be provided by third parties (separate from controller and switches)

41.3 OpenFlow Protocol

OpenFlow – Controller-to-Switch Communication

OpenFlow is the protocol used between the SDN controller and the switches:

- Operates over **TCP** (with optional encryption)
- **Controller-to-switch messages:** configure, modify-state (flow table entries), read-state (statistics), send-packet
- **Switch-to-controller messages:** packet-in (forward a packet to controller), flow-removed (flow table entry removed), port-status (port change notification)

Generalized forwarding: OpenFlow flow table entries have the form:

$$\text{match} + \text{action} + \text{priority} + \text{counters}$$

- **Match:** Pattern on packet header fields (source/dest IP, port, protocol, VLAN, etc.)
- **Action:** Forward to port(s), drop, modify fields, send to controller
- This generalizes traditional destination-based forwarding

42 Network Management and SNMP (5.7)

42.1 Network Management Overview

What is Network Management?

An autonomous system ("network") consists of 1000s of interacting hardware/software components. Network management involves:

- **Deployment, integration, and coordination** of hardware, software, and human elements
- **Monitor, test, poll, configure, analyze, evaluate, and control** the network
- Goal: meet real-time operational performance and Quality of Service requirements at a reasonable cost

42.2 SNMP – Simple Network Management Protocol

SNMP

SNMP is the protocol used for network management:

- **Management server** (managing entity) queries and controls managed devices
- **Managed devices** (routers, switches, hosts) contain a **Management Information Base (MIB)** – a collection of objects/variables representing the device's state

- SNMP has two modes:
  - **Request/response:** Manager queries or sets MIB values on a managed device
  - **Trap messages:** Managed device sends an unsolicited notification to the manager (e.g., link failure, threshold exceeded)
- SNMP runs over **UDP**
- SNMP message types: GetRequest, SetRequest, GetBulkRequest, InformRequest, Response, Trap

43 Summary and Exam Tips

Frequently Tested Topics – Chapter 5

Based on past exams and lecture emphasis:

1. **Dijkstra's algorithm:** Be able to run it step-by-step on a given graph and fill in the table. Know the formula: $D(v) = \min(D(v), D(w) + c(w, v))$.
2. **Bellman-Ford equation:** $d_x(y) = \min_v\{c(x, v) + d_v(y)\}$. Understand what each term means.
3. **LS vs. DV:** Know the differences in message complexity, convergence speed, and robustness.
4. **Count-to-infinity:** Understand why it happens, how poisoned reverse helps, and its limitation.
5. **OSPF:** Link-state protocol, runs Dijkstra's, floods LSAs, runs directly over IP (protocol 89), supports hierarchical areas.
6. **BGP:** Inter-AS routing, uses TCP port 179, eBGP vs. iBGP, AS-PATH and NEXT-HOP attributes, route selection rules, hot potato routing.
7. **ICMP:** Know the main message types (echo request/reply, TTL expired, dest unreachable). Understand how `traceroute` uses ICMP.
8. **SDN:** Logically centralized control plane, OpenFlow, match+action paradigm, three-layer architecture.
9. **Intra-AS vs. Inter-AS:** Why they are different (policy, scale, performance).

Common Exam Pitfalls

- **Link State ≠ Distance Vector** – Don't confuse which algorithm uses global vs. local information. LS = global/Dijkstra, DV = local/Bellman-Ford.
- **OSPF runs over IP**, not TCP or UDP. BGP runs over **TCP**.
- **Traceroute uses UDP segments**, not ICMP echo requests. The ICMP messages (TTL expired, port unreachable) are the *responses*.
- **Poisoned reverse does not solve all loops** – only prevents two-node loops. Three-or-more-node loops can still occur.
- **BGP is policy-based**, not purely shortest-path. Local preference can override shortest AS-PATH.
- **Hot potato routing** minimizes *intra-AS* cost to the exit point, not the total end-to-end cost.
- **ICMP messages are carried inside IP datagrams** – ICMP is "above" IP but is considered a network-layer protocol.
- **eBGP vs. iBGP:** eBGP is between routers in *different* ASes; iBGP is between routers in the *same* AS. Don't mix them up.

Key Formulas and Equations – Chapter 5

$$\text{Dijkstra's update:} \quad D(v) = \min\left(D(v),\ D(w) + c(w,v)\right) \tag{13}$$

$$\text{Bellman-Ford:} \quad d_x(y) = \min_v \left\{c(x,v) + d_v(y)\right\} \tag{14}$$

$$\text{Dijkstra complexity:} \quad O(n^2) \text{ (or } O(n \log n) \text{ with priority queue)} \tag{15}$$

$$\text{LS message complexity:} \quad O(n \cdot E) \text{ messages} \tag{16}$$

TTM4100 – KTN

Chapter 6: The Link Layer and LANs

Computer Networking: A Top-Down Approach, 8th ed.
Kurose & Ross – NTNU Spring 2026

44 Introduction to the Link Layer (6.1)

44.1 Terminology and Context

Link Layer – Basic Terminology

- **Nodes:** Hosts and routers (any device running a link-layer protocol).
- **Links:** Communication channels that connect adjacent nodes along a communication path. Links can be wired (e.g., Ethernet cable), wireless (e.g., WiFi), or LANs.
- **Frame:** The link-layer PDU (protocol data unit). The link layer encapsulates a network-layer datagram into a frame.

The link layer is responsible for transferring a datagram from one node to a **physically adjacent** node over a single link.

Important Distinction

Different link protocols may be used on different links along the same path. For example, a datagram might travel over Ethernet on the first link, then over a point-to-point link, then over WiFi – each hop uses whatever link-layer protocol is appropriate for that link.

44.2 Link Layer Services

Services Provided by the Link Layer

1. **Framing:** Encapsulate datagram into a frame, adding header and trailer.
2. **Link access (MAC protocol):** Controls how frames are transmitted onto the link. Especially important for shared (broadcast) links, where multiple nodes share the same medium.
3. **Reliable delivery:** Guarantees to move each datagram across the link without error (not all link-layer protocols offer this; common on high-error-rate links like wireless).
4. **Flow control:** Pacing between adjacent sending and receiving nodes.
5. **Error detection:** Errors caused by signal attenuation and electromagnetic noise. The receiver detects the presence of errors.
6. **Error correction:** The receiver identifies *and corrects* bit errors without retransmission.
7. **Half-duplex and full-duplex:** Half-duplex means nodes at both ends can transmit, but not at the same time.

44.3 Where is the Link Layer Implemented?

Network Interface Card (NIC)

The link layer is implemented in the **Network Interface Card (NIC)**, also known as the **network adapter**:

- Implements link-layer and physical-layer functionality
- Attaches to the host's system bus (PCIe, USB, etc.)
- Is a combination of hardware, software, and firmware

- The "adaptor" communicates with the rest of the host system

Sending side: Encapsulates datagram in a frame, adds error-checking bits, manages flow control, etc.
Receiving side: Looks for errors, extracts datagram, passes it up to the receiving node's network layer.

45 Error Detection and Correction (6.2)

45.1 Overview

Error Detection and Correction – EDC

Error detection is **not 100% reliable**: the protocol may miss some errors (i.e., undetected bit errors). However, we can make the probability of undetected errors extremely small by using stronger (larger) EDC fields.
The general approach:

- Sender computes EDC bits based on the data D
- Both D and EDC are sent
- Receiver recomputes EDC over received D' and compares with received EDC

45.2 Parity Checking

Single-Bit Parity

Add a single parity bit so that the total number of 1's is even (even parity) or odd (odd parity).

- Can detect **single-bit** errors
- Cannot detect errors where an even number of bits are flipped

Two-Dimensional (2D) Bit Parity

Arrange the data bits in a matrix (rows and columns). Compute parity for each row and each column:

- Can **detect and correct** single-bit errors (identify the exact row and column of the flipped bit)
- Can **detect** (but not correct) any combination of two-bit errors

Exam-Type: 2D Even Parity – V23 Q1.4.2

Question: Which alternatives show correct implementation(s) of a two-dimensional even parity scheme?

You are given 8 matrices (a–h), each with 4 rows of 4 data bits plus a parity column and a parity row. To verify:

1. Count the 1's in each **row** (including the parity bit). Must be even.
2. Count the 1's in each **column** (including the parity bit). Must be even.
3. If all rows AND all columns have even counts ⇒ correct.

Tip: Carefully check *every* row and column. A single mismatch means that alternative is wrong.

45.3 Internet Checksum

Internet Checksum (Review)

Used in the transport layer (TCP/UDP) and IP header:

- Treat data as a sequence of 16-bit integers
- Compute the 1's complement sum of all 16-bit words
- The checksum is the 1's complement of that sum
- Receiver adds all words including checksum; result should be `1111111111111111`

Weakness: Relatively weak – can miss certain patterns of errors. Used at the transport/network layer but *not* typically at the link layer, which uses CRC instead.

45.4 Cyclic Redundancy Check (CRC)

CRC – Key Concepts

CRC is a more powerful error-detection technique used at the link layer:

- Based on **modulo-2 arithmetic** (XOR, no carries/borrows)
- Uses a **generator polynomial** G of degree r (i.e., $r+1$ bits long)
- The sender appends r CRC bits (remainder R) to the data D
- The transmitted bits are: $D \cdot 2^r \oplus R$
- The receiver divides the received bits by G; if the remainder is 0, no error is detected

CRC Computation

Given data bits D and generator G (of $r+1$ bits):

1. Append r zero bits to D: $D \cdot 2^r$
2. Divide $D \cdot 2^r$ by G using modulo-2 division (XOR)
3. The remainder R (exactly r bits) is the CRC
4. Transmit: $D \cdot 2^r \oplus R$ (which is exactly divisible by G)

Key properties:

- Can detect **all burst errors of fewer than $r+1$ bits**
- Can detect **all odd numbers of bit errors** (if G contains $(x+1)$ as a factor)
- Burst errors of length $> r+1$ are detected with probability $1-(1/2)^r$

CRC vs. Checksum

CRC is much stronger than the Internet checksum. CRC is used at the link layer (e.g., Ethernet CRC-32), while the Internet checksum is used at higher layers. CRC can detect all burst errors up to length r, whereas the checksum can miss many error patterns.

Exam-Type: CRC Matching – S2025 Q7

Question: Given data string 101110 and a CRC generator, match the following:

- CRC value → 001
- CRC length → 3
- CRC generator → 1001
- Total bits sent → 9
- CRC generator polynomial → $x^3 + 1$

How to solve: The generator 1001 has 4 bits, so $r = 3$. Append 3 zeros to data: 101110**000**. Perform modulo-2 division by 1001 to get remainder 001. Total bits = 6 (data) + 3 (CRC) = 9. The polynomial $1001 = 1 \cdot x^3 + 0 \cdot x^2 + 0 \cdot x^1 + 1 \cdot x^0 = x^3 + 1$.

46 Multiple Access Protocols (6.3)

46.1 The Multiple Access Problem

The Shared Channel Problem

On a **broadcast link** (shared medium), when two or more nodes transmit simultaneously, a **collision** occurs – the signals interfere and the transmitted frames are corrupted.
Goal of MAC (Medium Access Control) protocols: Determine how nodes share the channel, i.e., determine when a node can transmit.

Ideal properties for a MAC protocol (channel rate R):

1. When one node wants to transmit, it can send at rate R
2. When M nodes want to transmit, each gets average rate R/M
3. Fully decentralized (no master node, no clock synchronization)
4. Simple

46.2 Taxonomy of MAC Protocols

Three Categories of MAC Protocols

1. **Channel Partitioning:** Divide channel into smaller "pieces" (time slots, frequency bands, codes). Allocate a piece to each node for exclusive use.
   - TDMA, FDMA, CDMA
2. **Random Access:** Channel not divided; allow collisions, then "recover" from them.
   - ALOHA, Slotted ALOHA, CSMA, CSMA/CD, CSMA/CA
3. **Taking Turns:** Nodes take turns, but nodes with more to send can take longer turns.
   - Polling, Token passing

46.3 Channel Partitioning Protocols

TDMA – Time Division Multiple Access

- Access to the channel is divided into "rounds" of fixed-length time slots
- Each station gets a fixed-length slot in every round
- Unused slots go idle (wasted capacity)
- **Pros:** No collisions, fair
- **Cons:** Wasted capacity when nodes have nothing to send; each node limited to R/N even if it's the only one transmitting

FDMA – Frequency Division Multiple Access

- Channel spectrum divided into frequency bands
- Each station gets a fixed frequency band
- Unused bands are idle (wasted capacity)
- Same pros/cons as TDMA

46.4 Random Access Protocols

46.4.1 Slotted ALOHA

Slotted ALOHA

Assumptions:

- All frames are exactly the same size
- Time is divided into equal-size slots (= time to transmit 1 frame)
- Nodes start to transmit only at the beginning of a slot
- Nodes are synchronized (they all know when slots begin)
- If 2 or more nodes transmit in the same slot, all nodes detect the collision

Operation:

- When a node has a frame to send, it transmits in the next slot
- If no collision: success!
- If collision: retransmit in each subsequent slot with probability p, until success

Slotted ALOHA Efficiency

Maximum efficiency $= 1/e \approx 0.37$ (37%)
This means: even under best conditions, the channel is productive only 37% of the time. The remaining 63% is wasted on collisions and empty slots.

Derivation: With N active nodes, each transmitting with probability p, the probability of a successful slot is $N \cdot p \cdot (1-p)^{N-1}$. Optimizing over p and letting $N \to \infty$ gives $\lim_{N\to\infty} N \cdot \frac{1}{N} \cdot (1-\frac{1}{N})^{N-1} = 1/e$.

46.4.2 Pure (Unslotted) ALOHA

Pure ALOHA

- No synchronization, no slots
- A node transmits immediately when it has a frame
- Collision risk: if any other node transmits during the *entire* vulnerable period (2 frame times), there's a collision
- **Maximum efficiency** $= 1/(2e) \approx 0.18$ **(18%)** – half that of slotted ALOHA

Exam-Type: ALOHA Properties – V24 Q1.4.5

Question: Which statements about pure ALOHA and slotted ALOHA are true?
Key facts:

- Pure ALOHA has lower efficiency than slotted ALOHA ✓
- Max efficiency of slotted ALOHA $(1/e)$ is **two times** that of pure ALOHA $(1/2e)$ ✓
- Slotted ALOHA reduces collisions compared to pure ALOHA ✓
- In slotted ALOHA, stations must wait for the beginning of the next time slot ⇒ they **cannot** send at any time without synchronization
- In pure ALOHA, stations can transmit at any time (no waiting for slots)

46.4.3 CSMA – Carrier Sense Multiple Access

CSMA – "Listen Before Transmit"

CSMA adds **carrier sensing**: if the channel is sensed busy, defer transmission.

Why do collisions still occur? Because of **propagation delay**: a node may not hear another node's transmission that has already started (the signal hasn't arrived yet). The longer the propagation delay, the higher the chance of collision.

46.4.4 CSMA/CD – CSMA with Collision Detection

CSMA/CD (used in Ethernet)

CSMA/CD adds **collision detection**: if a collision is detected, the transmission is **aborted immediately**, reducing channel waste.
Algorithm:

1. NIC receives datagram from network layer, creates frame
2. If NIC senses channel idle, start transmitting. If busy, wait until idle, then transmit.
3. If entire frame transmitted without detecting collision ⇒ done!
4. If collision detected, abort and send a **jam signal**
5. After aborting, enter **binary (exponential) backoff**

Binary Exponential Backoff

After the mth collision (for the same frame):

- Choose K randomly from $\{0, 1, 2, \ldots, 2^m - 1\}$
- Wait $K \times 512$ bit times (for Ethernet)

- After 10 collisions, freeze the range at $\{0, 1, \ldots, 1023\}$
- After 16 collisions, give up and report an error

The idea: adapt the retransmission delay to the estimated number of colliding stations. More collisions $\Rightarrow$ longer expected backoff.

CSMA/CD Efficiency

$$\text{Efficiency} = \frac{1}{1 + 5 \cdot t_{\text{prop}}/t_{\text{trans}}}$$

where:

- t_{prop} = maximum propagation delay between any 2 nodes
- t_{trans} = time to transmit a maximum-size frame

Key insight: Efficiency $\to 1$ as $t_{\text{prop}} \to 0$ or $t_{\text{trans}} \to \infty$.
Better than ALOHA, and is decentralized, simple, and cheap.

46.5 Taking Turns Protocols

Polling and Token Passing

Polling:

- A master node "invites" slave nodes to transmit in turn
- **Concerns:** Polling overhead, latency, single point of failure (master)
- Example: Bluetooth

Token Passing:

- A special "token" frame is passed from node to node sequentially
- A node can transmit only when it holds the token
- **Concerns:** Token overhead, latency, single point of failure (token loss)
- Example: FDDI (Fiber Distributed Data Interface)

Exam-Type: Classify MAC Protocols – S2025 Q6

Question: Categorize these protocols into Channel Partitioning, Random Access, or Taking Turns:

Protocol	Category
Bluetooth	Taking Turns (polling)
Ethernet CSMA/CD	Random Access
CDMA	Channel Partitioning
Slotted ALOHA	Random Access
FDDI	Taking Turns (token passing)
FDMA	Channel Partitioning

46.6 Cable Access Network: DOCSIS

DOCSIS – Data Over Cable Service Interface Specification

Cable internet uses a shared medium (coaxial cable + fiber):

- **Downstream:** FDM (Frequency Division Multiplexing) – multiple channels, broadcast to all homes
- **Upstream:** FDM + TDM + Random Access
- Upstream channels are divided into time slots; some slots are assigned (for data), some are for contention (random access using mini-slots)
- MAP messages from CMTS (Cable Modem Termination System) assign upstream slots
- Random access slots use a form of ALOHA with binary backoff for requesting bandwidth

47 LANs – Addressing and ARP (6.4)

47.1 MAC Addresses

MAC (Physical/LAN) Addresses

- **48-bit** address (6 bytes), written in hexadecimal (e.g., `1A-2F-BB-76-09-AD`)
- **Burned into** the NIC's ROM (or set by software)
- Administered by IEEE: manufacturers buy blocks of addresses
- **Flat address space:** portable – you can move a NIC from one LAN to another without changing its MAC address
- **Broadcast address:** `FF-FF-FF-FF-FF-FF` – delivered to all adapters on the LAN

MAC vs. IP Addresses

	MAC Address	IP Address
Length	48 bits (6 bytes)	32 bits (IPv4)
Structure	Flat (no hierarchy)	Hierarchical (network + host)
Scope	Link-layer, local	Network-layer, global
Portability	Portable (burned in NIC)	Depends on subnet
Analogy	SSN (identity number)	Postal address (location)

Why both? IP addresses are needed for network-layer routing (hierarchical, location-dependent). MAC addresses are needed for link-layer frame delivery on a single LAN segment.

47.2 ARP – Address Resolution Protocol

ARP – How It Works

ARP maps an IP address to a MAC address *on the same subnet*:

1. Node A wants to send a frame to Node B (knows B's IP, but not B's MAC)
2. A broadcasts an **ARP query** (dest MAC = `FF-FF-FF-FF-FF-FF`) containing B's IP address

3. **All nodes** on the LAN receive the query
4. Node B recognizes its IP and replies with a **unicast ARP response** containing its MAC address
5. A caches the IP→MAC mapping in its **ARP table** (with TTL, typically 20 minutes)

ARP is "plug-and-play": tables are built automatically, no manual configuration needed.

Exam-Type: ARP – V23 Q1.4.1

Question: It is _____'s job to translate between IP addresses and MAC (medium access control) addresses.
Answer: ARP (Address Resolution Protocol).

Common confusion:

- **DNS** translates *hostnames* to *IP addresses* (application layer)
- **ARP** translates *IP addresses* to *MAC addresses* (link layer)

47.3 Routing to Another Subnet

Sending a Datagram to Another Subnet

When host A (on subnet 1) sends to host B (on subnet 2), going through router R:
Key principle: IP addresses stay the same end-to-end; **MAC addresses change at each hop**.

1. A creates IP datagram: src IP = A, dest IP = B
2. A looks up next-hop (router R) using its routing table
3. A uses ARP to find R's MAC address on subnet 1
4. A creates link-layer frame: src MAC = A, dest MAC = R (on subnet 1 interface)
5. Router R receives frame, extracts datagram, looks up next hop for B
6. R uses ARP to find B's MAC address on subnet 2
7. R creates new frame: src MAC = R (on subnet 2 interface), dest MAC = B
8. B receives the frame

Critical Exam Point: MAC Changes – IP Stays

At every hop:

- The **source and destination IP addresses** remain **unchanged** (A → B)
- The **source and destination MAC addresses** are **rewritten** to reflect the current link

This is a very commonly tested concept!

48 Ethernet (6.4)

48.1 Ethernet Overview

Ethernet – The Dominant Wired LAN Technology

Ethernet is the most widely used wired LAN technology:

- First widely used LAN technology
- Kept up with speed demands: 10 Mbps → 10 Gbps → 400 Gbps
- Simple, cheap, and standardized (IEEE 802.3)

48.2 Ethernet Frame Structure

Ethernet Frame Format

Preamble	Dest MAC	Src MAC	Type	Data (Payload)	CRC
8 bytes	6 bytes	6 bytes	2 bytes	46–1500 bytes	4 bytes

- **Preamble** (8 bytes): 7 bytes of `10101010` followed by 1 byte of `10101011` (SFD – Start Frame Delimiter). Used for clock synchronization.
- **Destination MAC** (6 bytes): MAC address of the receiving adapter. If broadcast (`FF-FF-FF-FF-FF-FF`), all adapters process the frame.
- **Source MAC** (6 bytes): MAC address of the sending adapter.
- **Type** (2 bytes): Indicates the higher-layer protocol (e.g., `0x0800` = IPv4, `0x0806` = ARP). Acts as a demultiplexing key.
- **Data/Payload** (46–1500 bytes): The network-layer datagram. Minimum 46 bytes (padded if necessary). Maximum 1500 bytes = **MTU** (Maximum Transmission Unit).
- **CRC** (4 bytes): Cyclic Redundancy Check for error detection. If CRC check fails, the frame is **dropped** (no NAK/retransmission at link layer).

48.3 Ethernet Properties

Ethernet – Key Properties

- **Connectionless:** No handshaking between sending and receiving NICs.
- **Unreliable:** Receiving NIC does **not** send ACKs or NAKs. Dropped frames are only recovered if the sender uses a reliable higher-layer protocol (e.g., TCP).
- **MAC protocol:** Uses unslotted CSMA/CD with binary exponential backoff (in half-duplex mode).

48.4 Ethernet Topologies

Bus vs. Star Topology

Bus topology (legacy):

- All nodes share a single coaxial cable (broadcast medium)
- Collisions are common; uses CSMA/CD
- Popular in the 1990s

Star topology (modern):

- Each node has a dedicated connection to a central **switch**

- The switch operates in full-duplex mode – **no collisions!**
- Each link is its own collision domain
- Dominant topology today

48.5 802.3 Ethernet Standards

Common Ethernet Standards

Standard	Speed	Medium
100BASE-TX	100 Mbps	Copper (Cat5)
1000BASE-T	1 Gbps	Copper (Cat5e/6)
10GBASE-T	10 Gbps	Copper (Cat6a)
100GBASE-SR	100 Gbps	Fiber
400GBASE	400 Gbps	Fiber

All share the same frame format and use CSMA/CD (at lower speeds) or full-duplex switched (at higher speeds).

49 Link-Layer Switches (6.4)

49.1 Switch Basics

Ethernet Switch – Key Properties

An Ethernet switch is a **link-layer device**:

- **Store-and-forward:** Examines incoming frame's destination MAC, selectively forwards to one or more outgoing links
- **Transparent:** Hosts are **unaware** of the switch's presence
- **Plug-and-play / self-learning:** Switches do **not** need to be manually configured
- Uses a **switch table**: entries of (MAC address, interface, TTL)

49.2 Switch Self-Learning

Self-Learning Algorithm

The switch **learns** which hosts are reachable through which interfaces:

1. When a frame arrives on interface i, the switch records the **source MAC address** in its switch table, along with the interface i and a timestamp
2. If an entry is not refreshed within the **TTL** (aging time), it is deleted

This is completely automatic – no configuration needed!

49.3 Switch Forwarding

Frame Forwarding/Filtering

When a frame arrives at a switch on interface i, with destination MAC address `dest`:

1. Look up `dest` in the switch table
2. **If entry found:**
   - If `dest` is on interface i (same segment) $\Rightarrow$ **drop/filter** (no need to forward)
   - If `dest` is on interface $j \neq i \Rightarrow$ **forward** the frame to interface j
3. **If entry not found: flood** – forward the frame on **all** interfaces except i

49.4 Switches vs. Routers

Switches vs. Routers

	Switch (Layer 2)	Router (Layer 3)
Layer	Data link layer	Network layer
Addresses	MAC addresses	IP addresses
Forwarding	Switch table (self-learned)	Routing table (algorithms)
Broadcast	Floods unknown frames	Does not forward broadcasts
Configuration	Plug-and-play	Needs IP configuration
Scalability	Limited (flat addressing)	Hierarchical (IP subnets)

Exam-Type: Switches vs. Routers – V24 Q1.4.1

Question: What is/are **false** when comparing between switches and routers?
Key facts:

- Both routers and switches are connecting devices in networking ✓
- Routers operate at the **Data link layer** and switches at **Network layer** $\Rightarrow$ **FALSE!** (It's the opposite)
- Switches operate at the Data link layer, routers at the Network layer ✓
- Switches connect devices within a network; routers connect different networks ✓
- Routers rely on IP addresses; switches rely on MAC addresses ✓

50 A Day in the Life of a Web Request (6.7)

Synthesis: All Layers Working Together

When a laptop connects to a network and requests a web page, the following protocols interact across **all five layers**:
Step 1: Connecting to the network (DHCP)

- Laptop needs an IP address, subnet mask, default gateway, and DNS server
- DHCP Discover $\rightarrow$ DHCP Offer $\rightarrow$ DHCP Request $\rightarrow$ DHCP ACK
- DHCP is encapsulated in UDP, then IP, then Ethernet (broadcast)

Step 2: Getting the router's MAC address (ARP)

- Laptop knows the gateway IP (from DHCP) but needs its MAC address

- ARP request (broadcast) → ARP reply (unicast) gives the router's MAC

Step 3: Resolving the web server's IP (DNS)

- Laptop creates DNS query for the web server hostname
- DNS query sent via UDP to DNS server (IP from DHCP)
- DNS response provides the web server's IP address

Step 4: Establishing a connection (TCP)

- TCP SYN to web server → SYN-ACK → ACK (three-way handshake)
- IP datagram routed through network; at each hop, link-layer frames are created

Step 5: Requesting the web page (HTTP)

- HTTP GET request sent inside TCP segment
- Web server responds with HTTP response containing the web page

Exam-Type: Protocols at Each Step – S2025 Part II Q3

Question: A researcher plugs his laptop into the Ethernet port. He sends an email using Outlook. Elaborate the protocols used at each step.
Answer outline:

1. **DHCP** (over UDP/IP/Ethernet broadcast): Obtain IP address, gateway, DNS server
2. **ARP**: Resolve gateway router's MAC address
3. **DNS** (over UDP): Resolve mail server hostname to IP
4. **TCP**: Establish connection to mail server
5. **SMTP**: Send the email (application layer)
6. On the receiving end: **IMAP/POP3/HTTP** to retrieve the email

At every hop along the path, the **link layer** (Ethernet or other) handles frame creation and MAC addressing, while **IP** handles routing.

51 Summary of Key Formulas and Facts

Key Formulas – Chapter 6

Concept	Formula / Value
Slotted ALOHA max efficiency	$1/e \approx 37\%$
Pure ALOHA max efficiency	$1/(2e) \approx 18\%$
CSMA/CD efficiency	$\dfrac{1}{1 + 5 \cdot t_{\text{prop}}/t_{\text{trans}}}$
Binary exponential backoff	After mth collision: $K \in \{0, 1, \ldots, 2^m - 1\}$, wait $K \times 512$ bit times
CRC remainder bits	$r = \deg(G) =$ length of G minus 1
CRC burst detection	Detects all bursts $< r + 1$ bits

MAC address length	48 bits (6 bytes)
ARP table TTL	Typically 20 minutes
Ethernet MTU	1500 bytes
Ethernet min payload	46 bytes

52 Additional Exam Questions from Past Papers

S2025 – CSMA Analysis – Part II Q7

Question: Consider 6 packets arriving at times $t = 0.1, 1.4, 1.8, 3.2, 3.3, 4.1$ at different wireless nodes. Each transmission takes exactly 1 time unit. Propagation delay = 0.2 time units. For CSMA (without collision detection), determine which packets are successfully transmitted.

Approach:

1. Packet 1 starts at $t = 0.1$, finishes at $t = 1.1$. No other node has transmitted yet ⇒ **success**.
2. Packet 2 at $t = 1.4$: channel is free (packet 1 ended at $t = 1.1$). Check if signal from packet 2 reaches other nodes before they transmit.
3. Packet 3 at $t = 1.8$: does it sense packet 2? Packet 2 started at $t = 1.4$, prop delay = 0.2, so signal reaches at $t = 1.6$. At $t = 1.8$, packet 3's node senses busy ⇒ defers.
4. Continue this analysis for each packet...

Key points:

- A node senses the channel *before* transmitting
- Signal takes 0.2 time units to propagate between nodes
- If the channel is sensed busy, the node does *not* transmit
- If collision occurs, both frames are lost (no CD, so full transmission happens)

S2025 – Link-Layer Frame Headers – Q8

Question: A link-layer frame travels from a host to a router. Match headers H_1, H_2, H_3 (outermost to innermost before the message M) to the correct layer.
Answer:

- H_1 (outermost) = **Link layer** (Ethernet header with MAC addresses)
- H_2 = **Network layer** (IP header)
- H_3 (closest to M) = **Transport layer** (TCP/UDP header)

Remember: encapsulation adds headers from top to bottom. At the link layer, the frame wraps everything, so the link-layer header is the outermost.

V23/V24 – Flow Control in Link Layer – V23 Q2.2

Question: Besides Transport Layer, which of Physical Layer, Link Layer, and Network Layer has flow control concepts?
Answer: The **Link Layer** has flow control (pacing between adjacent sending and receiving nodes). This is analogous to TCP flow control but operates at the link level.

The Link Layer also has error detection/correction (like transport layer's checksum), and reliable delivery on some link protocols (like WiFi 802.11).

S2025 – Subnetting and Broadcast Address – Q5 and Q12

Q5: Given 208.27.1.0/22, create 12 subnets. How many usable hosts per subnet?
/22 = 1024 addresses total. To get 12 subnets, need $\lceil \log_2 12 \rceil = 4$ subnet bits $\Rightarrow$ /26. Each /26 subnet has $2^6 - 2 = 62$ usable hosts. **Answer: b) 62**

Q12: What is the broadcast address of prefix 172.18.16.0/21?
/21 means 21 network bits, 11 host bits. $2^{11} = 2048$ addresses. 172.18.16.0 + 2047 = 172.18.23.255. **Answer: b) 172.18.23.255**

TTM4100 – KTN

Chapter 7: Wireless and Mobile Networks

Computer Networking: A Top-Down Approach, 8th ed.
Kurose & Ross – NTNU Spring 2026

53 Introduction to Wireless Networks (7.1)

53.1 Elements of a Wireless Network

Key Components of a Wireless Network

A wireless network consists of several key elements:

- **Wireless hosts:** End-user devices (laptops, smartphones, tablets) running applications. May be stationary or mobile.
- **Base stations:** Fixed infrastructure devices (cell towers, WiFi access points) that relay packets between wired network and wireless hosts within coverage area.
- **Wireless links:** Communication medium connecting hosts to base stations (or hosts to hosts in ad hoc mode). Various link standards differ in range, data rate, and frequency band.
- **Infrastructure (wired) network:** The backbone connecting base stations to each other and to the global Internet.

53.2 Infrastructure Mode vs. Ad Hoc Mode

Infrastructure Mode vs. Ad Hoc Mode

Infrastructure mode:

- Hosts connect *through* a base station (access point or cell tower).
- Base station connects to wired network – provides Internet access.
- Handoff: host moves from one base station's coverage to another (typical in cellular networks, possible in WiFi).

Ad hoc (infrastructure-less) mode:

- No base station. Hosts communicate *directly* with each other.
- Hosts must self-organize, self-route.
- Examples: Bluetooth devices, vehicular networks, emergency scenarios where infrastructure is unavailable.

53.3 Wireless Network Taxonomy

Wireless Network Taxonomy – 2x2 Matrix

Networks are classified along two dimensions:

	Single hop	**Multiple hops**
Infrastructure	WiFi 802.11, 4G/5G LTE	Wireless mesh, relay
No infrastructure	Bluetooth, ad hoc WiFi	MANET, VANET

- **Single-hop, infrastructure:** Host connects to base station in one hop. Most common (home WiFi, cellular).
- **Single-hop, no infrastructure:** Hosts talk directly, no base station (Bluetooth).
- **Multi-hop, infrastructure:** Packets may relay through wireless nodes before reaching a base station (wireless mesh).
- **Multi-hop, no infrastructure:** Packets relay hop-by-hop with no fixed infrastructure (MANET – Mobile Ad-hoc NETwork).

54 Wireless Link Characteristics (7.2)

54.1 Challenges of the Wireless Channel

Why Wireless Is Harder Than Wired

Three fundamental challenges in wireless communications:

1. **Path loss / decreased signal strength:** Signal energy attenuates as it propagates through space (even in free space). Signal strength falls off roughly as $1/d^2$ (free-space) or faster with obstacles.
2. **Interference from other sources:** ISM bands (2.4 GHz, 5 GHz used by WiFi) are unlicensed – shared with microwaves, Bluetooth, cordless phones. Licensed cellular bands still experience inter-cell interference.
3. **Multipath propagation:** Signal reflects off objects (buildings, ground, furniture) and arrives at receiver via multiple paths with different delays. Causes constructive/destructive interference – the channel changes over time (fading).

All three mean: wireless links have *higher* bit error rates than wired links, and the error rate is highly variable.

54.2 SNR vs. BER – The Fundamental Tradeoff

SNR and BER – Signal-to-Noise Ratio vs. Bit Error Rate

SNR (Signal-to-Noise Ratio): Ratio of received signal power to noise power, expressed in dB.

$$\mathrm{SNR}_{dB} = 10 \cdot \log_{10}\left(\frac{P_{\text{signal}}}{P_{\text{noise}}}\right)$$

BER (Bit Error Rate): Fraction of transmitted bits received in error.

Key relationships:

- **Higher SNR ⇒ lower BER.** More signal power relative to noise makes it easier to decode bits correctly.
- **Higher bit-rate modulation ⇒ higher BER** (for the same SNR). E.g., QAM-256 encodes more bits per symbol but requires higher SNR to achieve the same BER

as BPSK.
- As a host moves farther from the base station, SNR decreases and BER increases.

Modulation and Adaptive Rate

Different modulation schemes achieve different tradeoffs:

- **BPSK** (2 states): 1 bit/symbol – robust, low BER at given SNR, but slow.
- **QPSK** (4 states): 2 bits/symbol.
- **QAM-16** (16 states): 4 bits/symbol.
- **QAM-256** (256 states): 8 bits/symbol – fast, but requires high SNR.

Rate adaptation / adaptive modulation and coding (AMC):
Transmitter and receiver dynamically adjust the modulation scheme based on measured channel quality (SNR estimate).

- Good channel (high SNR) → use high-rate modulation (e.g., QAM-64) to maximize throughput.
- Poor channel (low SNR, e.g., host far from AP) → fall back to lower-rate, more robust modulation (e.g., BPSK).
- Both 802.11 WiFi and 4G/5G LTE use AMC.

Exam – SNR/BER/Modulation - V23 Q1.4.3 and V24 Q1.4.3

V23 Q1.4.3 – Which statements about wireless physical layer are true?

- **a) Higher SNR means lower BER. ✓ TRUE**
- b) Wireless links have lower BER than wired links. × FALSE – wireless is generally worse.
- c) Higher modulation bit rate means lower BER at same SNR. × FALSE – higher bit rate means higher BER.
- d) Multipath propagation eliminates interference. × FALSE – it causes fading/interference.
- **e) As SNR decreases, using a lower modulation bit rate can maintain BER. ✓ TRUE** – this is exactly rate adaptation.

V24 Q1.4.3 – SNR/BER/Modulation statements:

- **a) Lower SNR means higher BER. ✓ TRUE** (equivalent to: lower SNR ⇒ worse channel ⇒ more errors).
- b) Lower SNR means lower BER. × FALSE.
- c) Higher modulation rate means lower BER at given SNR. × FALSE.
- **d) Higher modulation bit rate means higher BER at given SNR. ✓ TRUE**
- e) Path loss increases signal strength. × FALSE – path loss *decreases* signal strength.

Rule of thumb for exam: Higher SNR = better; higher bit-rate modulation = more errors per bit for same SNR.

54.3 Hidden Terminal Problem

The Hidden Terminal Problem

In wireless networks, node A may be able to hear node B, but *not* hear node C which is also transmitting to B. From A's perspective, the channel appears idle even though B is experiencing a collision.

```
A    --  > B <--    C    (A and C cannot hear each other)
```

- A transmits to B. C also transmits to B simultaneously (C cannot hear A's transmission, so C thinks channel is idle).
- At B, both signals collide – but A and C don't know this.
- This is the **hidden terminal problem**: C is "hidden" from A.
- CSMA carrier sense does *not* solve this – A hears nothing and proceeds to transmit regardless.
- Solution: **RTS/CTS** mechanism (see Section 55.4).

55 IEEE 802.11 – WiFi Wireless LANs (7.3)

55.1 802.11 Standards Overview

IEEE 802.11 WiFi Standards

Standard	Name	Freq. band	Max rate	Notes
802.11b	WiFi 1	2.4 GHz	11 Mbps	First widely deployed
802.11a	WiFi 2	5 GHz	54 Mbps	Less range than b
802.11g	WiFi 3	2.4 GHz	54 Mbps	Backward compat with b
802.11n	WiFi 4	2.4/5 GHz	600 Mbps	MIMO antennas
802.11ac	WiFi 5	5 GHz	3.47 Gbps	Wide channels, MU-MIMO
802.11ax	WiFi 6/6E	2.4/5/6 GHz	14 Gbps	OFDMA, better dense use
802.11af	White-Fi	TV whitespace	varies	Long range
802.11ah	HaLow	sub-1 GHz	varies	IoT, long range, low power

All 802.11 standards use CSMA/CA for multiple access.
All operate in **BSS (Basic Service Set)** infrastructure mode or IBSS (ad hoc/independent BSS).

55.2 BSS Architecture and Association

BSS Architecture – Basic Service Set

- **BSS:** A group of wireless stations and one **Access Point (AP)**. The AP connects the BSS to the wired distribution system (router/Internet).
- **SSID (Service Set Identifier):** The human-readable network name (e.g., "NTNU-WiFi").
- **Channel:** 802.11 operates on channels within the frequency band. In 2.4 GHz there are 11 overlapping channels (only 1, 6, 11 are non-overlapping). Adjacent APs should use non-overlapping channels.
- **AP beacon frames:** AP periodically broadcasts beacon frames (typically every

100 ms) containing its SSID and MAC address.

Passive vs. Active Scanning

To associate with an AP, a host uses one of two scanning methods:

Passive scanning:

1. Host listens on each channel for beacon frames from APs.
2. Selects AP with best signal (or user preference).
3. Sends association request frame to chosen AP.
4. AP sends association response frame.

Active scanning:

1. Host broadcasts **probe request frame** on each channel.
2. APs that hear it reply with **probe response frame** (SSID, rates, etc.).
3. Host selects AP and sends association request.
4. AP sends association response.

After association: Host typically runs DHCP to obtain IP address; DHCP server often co-located with AP/router.

55.3 CSMA/CA – Multiple Access in 802.11

Why Not CSMA/CD in Wireless?

Wired Ethernet uses **CSMA/CD** (Collision Detection): transmit, detect collision, abort, backoff, retry.

In wireless, collision *detection* is impractical:

- To detect a collision, the transmitter must compare its own transmitted signal with what it receives simultaneously. In wireless, the transmitted signal is much stronger than any incoming signal – the transmitter would drown out the received signal.
- Hidden terminal: two senders may be transmitting to the same receiver but cannot hear each other.

Therefore, 802.11 uses **CSMA/CA** (Collision *Avoidance*) – try to *avoid* collisions rather than detect them.

CSMA/CA Protocol – Step by Step

Sender:

1. Sense channel. If channel idle for **DIFS** (Distributed Inter-Frame Space) duration, transmit entire frame.
2. If channel busy, wait until channel becomes idle. Then wait DIFS. Then choose random backoff (binary exponential backoff). Countdown the backoff timer *only* while channel is idle (freeze timer when channel is busy).
3. When backoff reaches 0, transmit entire frame.
4. **No collision detection** – must transmit the *entire* frame.

Receiver:

1. If frame received correctly, wait **SIFS** (Short Inter-Frame Space – shorter than DIFS) and send **ACK**.

Sender:

2. If no ACK received within timeout → collision assumed → binary exponential back-off and retransmit.

Key timing: SIFS < DIFS. ACK uses SIFS so it gets priority access before any new transmission (which must wait DIFS).

Inter-Frame Spacing Summary

- **SIFS (Short IFS):** Used for ACK, CTS (high priority). Shortest wait.
- **DIFS (Distributed IFS):** Used for data frames and RTS. Longer than SIFS – ensures ACK/CTS sent before new data.
- **EIFS (Extended IFS):** Used after a reception error (frame not decoded). Longest.

Priority: SIFS < DIFS < EIFS. Shorter IFS → higher access priority.

55.4 RTS/CTS Mechanism

RTS/CTS – Request to Send / Clear to Send

Problem solved: Hidden terminal problem. Even with CSMA/CA, two hidden terminals can both sense the channel as idle and transmit simultaneously, causing a collision at the AP.

RTS/CTS mechanism (optional in 802.11):

1. Sender waits DIFS, then sends a short **RTS (Request to Send)** frame to the AP. RTS includes: duration of intended data transmission.
2. AP broadcasts **CTS (Clear to Send)** frame (after SIFS). CTS is heard by *all* stations in AP's range (including hidden terminals).
3. CTS reserves the channel: all stations that hear CTS defer for the duration specified.
4. Sender transmits data frame (after SIFS).
5. AP sends ACK (after SIFS).

Why it helps: Even if A and C are hidden from each other, both hear the CTS from the AP and know to defer. The overhead of RTS/CTS is small compared to large data frames.

Trade-off: RTS/CTS adds overhead (extra frames). Worth it for large frames; typically enabled with a threshold (e.g., RTS/CTS only for frames > 500 bytes).

Important: RTS/CTS *reduces* but does *not* eliminate hidden terminal collisions (two senders can still collide on the RTS itself).

Exam – CSMA/CA and RTS/CTS - V23 Q1.4.4 and V24 Q1.4.4

V23 Q1.4.4 – Which are elements of 802.11 MAC?

- **a) CSMA/CA. ✓ TRUE** – the core MAC protocol.
- b) CSMA/CD. × FALSE – that is Ethernet.
- **c) ACK for every frame. ✓ TRUE** – receiver sends ACK after every successfully received frame (SIFS wait).
- **d) RTS/CTS mechanism. ✓ TRUE** – optional but part of the standard.
- **e) DIFS wait before transmission. ✓ TRUE** – mandatory part of CSMA/CA.
- f) Collision detection. × FALSE – no collision detection in 802.11.

V24 Q1.4.4 – CSMA/CA and RTS/CTS statements:

- **a) Hidden terminal problem can still occur even with CSMA/CA. ✓ TRUE** – CSMA/CA alone does not solve hidden terminal; RTS/CTS is needed.
- b) In RTS/CTS, the sender broadcasts CTS to reserve the channel. × FALSE – the *AP* broadcasts CTS, not the sender.
- **b) In RTS/CTS, sender sends RTS to AP and AP responds with CTS. ✓ TRUE** – correct description of RTS/CTS.
- c) CSMA/CA detects collisions and retransmits. × FALSE – it *avoids* collisions; no detection.
- d) Backoff timer counts down while channel is busy. × FALSE – timer is *frozen* when channel is busy.
- **e) RTS/CTS improves channel access for large frames. ✓ TRUE** – reduces wasted bandwidth from large-frame collisions.

Key distinctions to remember:

- **CA not CD:** Avoid, not detect.
- **AP broadcasts CTS**, not the sender.
- **Backoff timer freezes** when channel is busy (counts down only during idle).
- **SIFS** $<$ **DIFS**: ACK/CTS use SIFS for priority access.

Exam – S2025 Part II Q7 – CSMA/CA Analysis

Scenario: Six wireless nodes share an AP. At time $t = 0$, each node has one packet queued to transmit. Analyze the transmission order and timing under 802.11 CSMA/CA.

Key points for solving such problems:

1. At $t = 0$ all nodes sense channel idle. All wait DIFS, then each draws a random backoff counter (uniformly from $[0, CW - 1]$, initial contention window CW = 15 or 31 depending on standard).
2. The node with the *smallest* backoff counter transmits first.
3. All other nodes freeze their timers when they detect the channel is busy (ongoing transmission).
4. After transmission + ACK + DIFS, remaining nodes resume counting down from where they froze.
5. If two nodes had the same backoff, they collide: both do not receive ACK, double the contention window, draw new random backoff.

Typical exam questions:

- Who transmits first given specific backoff values?
- When does a given node finish transmitting (add DIFS + transmission time + SIFS + ACK time)?
- What happens if two nodes draw the same backoff?

Formula for channel occupancy:

$$T_{\text{total}} = \text{DIFS} + \text{backoff} \cdot T_{\text{slot}} + T_{\text{frame}} + \text{SIFS} + T_{\text{ACK}}$$

55.5 802.11 Frame Structure

802.11 Frame – Four MAC Addresses

Unlike Ethernet (2 MAC addresses), an 802.11 frame has **4 address fields**:

Field	In Infrastructure Mode	Direction
Address 1	MAC of receiver	Destination of this wireless hop
Address 2	MAC of transmitter	Source of this wireless hop
Address 3	MAC of router interface	Gateway router to/from wired network
Address 4	Used only in ad hoc mode	–

Example – host H1 sending to server S1 via AP:

- H1 → AP (wireless frame): Addr1 = AP MAC, Addr2 = H1 MAC, Addr3 = Router MAC
- AP → Router (Ethernet frame): Dest = Router MAC, Src = AP MAC
- (The IP datagram inside always has IP src = H1 IP, IP dst = S1 IP)

Why Address 3? The AP acts as a link-layer relay. It strips the 802.11 frame and creates an Ethernet frame for the wired LAN. Address 3 tells the AP *where to forward* the packet on the wired side.

802.11 Frame Control Field

The **Frame Control** field (2 bytes) contains:

- **Protocol version** (2 bits): Always 0 for current 802.11.
- **Type** (2 bits): Management (00), Control (01), Data (10).
- **Subtype** (4 bits): E.g., beacon, probe request, RTS, CTS, ACK, data.
- **To DS / From DS** (1 bit each): Indicate direction relative to distribution system (infrastructure).
- **More fragments** (1 bit): Whether more fragments follow.
- **Retry** (1 bit): Whether this is a retransmission.
- **Power management** (1 bit): Sender going to sleep after this frame.
- **More data** (1 bit): AP has more frames buffered for this station.
- **WEP/Protected** (1 bit): Frame body is encrypted.
- **Order** (1 bit): Frames must be delivered in order.

55.6 Mobility Within the Same Subnet

Mobility Within Same IP Subnet

Scenario: Host H1 moves from BSS1 (AP1) to BSS2 (AP2). Both APs are in the *same* IP subnet.

What changes and what doesn't:

- **IP address stays the same** – same subnet, no need for new DHCP (short timescale).
- **Association changes:** H1 disassociates from AP1 and associates with AP2.
- **Switch self-learning:** The Ethernet switch between AP1 and AP2 updates its forwarding table via self-learning when H1 sends a frame through AP2. 802.11 may also send a broadcast frame to "wake up" switch learning.
- **No routing change required.**

Cross-subnet mobility (different IP subnet) requires IP-level mobility support (Mobile IP, not covered in 7th ed. detail).

55.7 Rate Adaptation and Power Management

Rate Adaptation

802.11 dynamically adjusts the modulation/coding scheme based on channel quality:

- Good channel (high SNR) → use high-rate modulation (e.g., QAM-64 5/6 coding) → higher throughput.
- Poor channel (low SNR, edge of coverage) → fall back to low-rate, robust modulation (e.g., BPSK 1/2) → maintain connection at lower rate.
- Feedback: receiver sends SNR estimates (or implicit: lack of ACK ⇒ reduce rate).

This is the same AMC concept as in LTE (see Section 56).

802.11 Power Management

Mobile devices want to save battery. 802.11 has a sleep/wake mechanism:

- Host sets **Power Management** bit in frame control = 1 to tell AP it will sleep.
- AP buffers frames destined for sleeping host.
- AP's **beacon frame** includes a list of APs with buffered frames (TIM – Traffic Indication Map).
- Host wakes up before every beacon (every 100 ms), checks TIM, stays awake if it has buffered frames, goes back to sleep otherwise.
- Result: host can sleep for ∼99% of time if no traffic (significant power savings).

56 Cellular Networks – 4G and 5G (7.4)

56.1 4G LTE Architecture

4G LTE Network Architecture

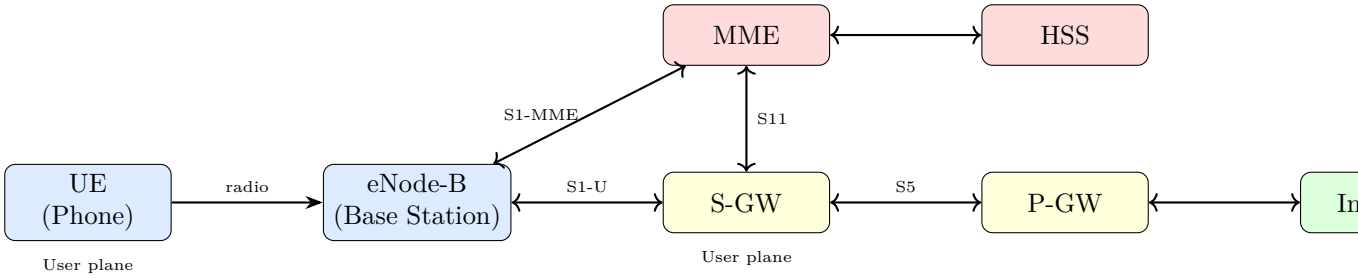
4G LTE (Long-Term Evolution) architecture separates the **user plane** (data) and **control plane** (signaling):

Radio Access Network (RAN):

- **UE (User Equipment):** The mobile device (phone, tablet). Contains a SIM card with a 64-bit **IMSI (International Mobile Subscriber Identity)** that uniquely identifies the subscriber.
- **eNode-B (eNB):** The 4G base station. Handles radio access (scheduling, modulation, etc.) and some control functions. In 5G, called **gNode-B (gNB)**.

Evolved Packet Core (EPC):

- **MME (Mobility Management Entity):** Control plane. Handles authentication, UE tracking, paging, handover decisions. Contacts HSS to verify IMSI.
- **HSS (Home Subscriber Service):** Database of subscriber information (IMSI, service profile, authentication keys).
- **S-GW (Serving Gateway):** User-plane gateway in the visited network. Forwards data between eNB and P-GW. Handles mobility across eNBs.
- **P-GW (PDN Gateway):** User-plane gateway to the Internet (Packet Data Network). Performs NAT, allocates UE IP address, enforces QoS.



56.2 LTE Data Plane – Tunneling

LTE Tunneling – GTP-U Protocol

In LTE, user data is **tunneled** from eNode-B all the way to P-GW using **GTP-U (GPRS Tunneling Protocol – User plane)**:

Tunnel path: UE $\leftrightarrow$ eNB $\xrightarrow{\text{GTP tunnel}}$ S-GW $\xrightarrow{\text{GTP tunnel}}$ P-GW $\leftrightarrow$ Internet

- Each user datagram is encapsulated in a GTP-U header (which adds a Tunnel Endpoint ID – TEID).
- eNB $\rightarrow$ S-GW: one GTP tunnel per UE.
- S-GW $\rightarrow$ P-GW: another GTP tunnel per UE.
- The tunnels allow user traffic to flow transparently regardless of which eNB the UE is attached to.

Why tunneling? Hides radio-network mobility from the IP core. When a UE moves between eNBs (handover), only the tunnel endpoints change – the P-GW IP and the UE's allocated IP remain constant.

56.3 LTE Link Layer

LTE Radio Access – OFDM and Link-Layer Protocols

Physical layer: LTE uses **OFDM (Orthogonal Frequency Division Multiplexing)**:

- Frequency band divided into many narrow orthogonal subcarriers.
- Combined FDM (across subcarriers) + TDM (time slots) = **OFDMA** in downlink.
- Each 1 ms subframe is divided into resource blocks (time $\times$ frequency) allocated to UEs by the base station scheduler.
- Each UE's resource block can use a different modulation/coding scheme (adaptive modulation per UE).

LTE link layer sub-layers (between UE and eNB):

1. **PDCP (Packet Data Convergence Protocol):** IP header compression, ciphering (encryption).
2. **RLC (Radio Link Control):** Fragmentation and reassembly of large packets. Reliable data transfer (ARQ – Automatic Repeat reQuest).
3. **MAC (Medium Access Control):** Scheduling, HARQ (Hybrid ARQ – combines FEC + ARQ).

56.4 BS Association and Sleep Modes

BS Association in LTE

How a UE connects to an eNB:

1. eNBs broadcast **synchronization signals** every 5 ms on a set of well-known frequencies.
2. UE scans frequencies, detects synch signals, measures signal strength from multiple eNBs.
3. UE selects the best eNB (strongest signal / highest SINR).
4. UE sends **Random Access** message to eNB (contention-based RACH or

contention-free).
5. eNB responds with timing alignment and temporary ID.
6. Authentication via MME and HSS: IMSI sent, HSS verifies subscriber, session established.
7. P-GW assigns UE an IP address (via MME → S-GW → P-GW signaling).

LTE Sleep Modes – DRX

To save battery, LTE uses **DRX (Discontinuous Reception)** with two sleep modes:

- **Light sleep:** Wake up every 100s of milliseconds to check for downlink activity. Used when device is active but idle momentarily.
- **Deep sleep:** Sleep for 5–10 seconds. Used for longer idle periods (e.g., app in background). Device must re-establish connection more fully on wakeup.

Tradeoff: Deeper sleep → more battery savings, but longer latency to resume data transfer.

56.5 4G vs. 5G

4G vs. 5G – Key Differences

Feature	4G LTE	5G NR
Peak downlink rate	∼1 Gbps	Up to 20 Gbps
Latency	∼30–50 ms	<1 ms (target)
Frequency bands	Sub-6 GHz	Sub-6 GHz + mmWave
Mobility	Full support	Full support
Architecture	Partial C/U-plane separation	Complete C/U-plane separation
Base station	eNode-B	gNode-B (gNB)
Core network	EPC	5GC (5G Core)

Key facts for exams:

- **Both 4G and 5G support mobility / handover.**
- **5G has higher peak bitrate** than 4G.
- 5G is designed for *complete* control/user-plane separation (4G has partial separation).
- 5G mmWave (millimeter wave, 24–100 GHz) offers very high rates but short range and limited penetration.
- 5G sub-6 GHz offers better range and coverage.

Exam – 4G/5G Statements - V23 Q1.4.5

V23 Q1.4.5 – Which statements about 4G and 5G are true?

- **a) 5G has higher peak bitrate than 4G. ✓ TRUE** – up to 20 Gbps vs. ∼1 Gbps.
- b) Only 5G supports mobility/handover; 4G does not. × FALSE – both support mobility.
- **c) Both 4G and 5G support host mobility. ✓ TRUE**

- d) 4G uses complete control/user-plane separation but 5G does not. × FALSE – 5G is the one designed for *complete* separation.
- e) 5G uses CSMA/CA for medium access. × FALSE – that is 802.11 WiFi. LTE/5G use OFDMA and scheduled access.

Remember: 5G = higher rate + lower latency + complete C/U-plane separation + both support mobility.

57 Putting It All Together – Wireless Summary

Chapter 7 – Big Picture Summary

Why is wireless hard? Three challenges: path loss, interference, multipath. These cause higher and variable BER compared to wired links.

SNR/BER tradeoff: Higher SNR ⇒ lower BER. Higher-order modulation ⇒ higher BER at same SNR but higher throughput. Rate adaptation dynamically picks the best modulation for current channel conditions.

802.11 MAC: CSMA/CA (not CD). DIFS before transmission. Random backoff (timer freezes during busy channel). Full-frame transmission. ACK after SIFS. RTS/CTS for hidden terminal mitigation.

802.11 frames: 4 MAC addresses (Addr1=receiver, Addr2=transmitter, Addr3=router, Addr4=ad hoc). Frame control field encodes type, subtype, power management, WEP, To/From DS bits.

4G LTE: UE ↔ eNB (radio) ↔ S-GW ↔ P-GW ↔ Internet (data plane). MME handles control (auth, mobility). HSS stores subscriber data. IMSI identifies subscriber on SIM. GTP-U tunneling end-to-end. OFDM/OFDMA radio access.

5G vs. 4G: Higher peak rate, lower latency, complete C/U-plane separation. Both support mobility.

Common Exam Traps – Chapter 7

- **CSMA/CD vs. CSMA/CA:** 802.11 uses CA (avoidance), *not* CD (detection). Wired Ethernet uses CD.
- **Who sends CTS?** The *AP* sends CTS in response to sender's RTS – not the sender.
- **Backoff timer behavior:** Timer counts down only when channel is *idle*. It *freezes* (not resets) when channel goes busy.
- **Higher SNR = better:** Do not confuse direction. Lower SNR ⇒ higher BER ⇒ worse performance.
- **Higher modulation = more fragile:** QAM-256 is faster but needs good channel. BPSK is slow but robust.
- **Both 4G and 5G have mobility:** A common distractor says only 5G supports mobility – false.
- **5G does not use CSMA/CA:** CSMA/CA is 802.11 WiFi. LTE and 5G use OFDMA with base-station scheduling.
- **RTS/CTS reduces but doesn't eliminate hidden terminal:** Two nodes can still collide on their RTS frames.

Quick Reference – Key Numbers

- 802.11b: 2.4 GHz, 11 Mbps
- 802.11g: 2.4 GHz, 54 Mbps
- 802.11n (WiFi 4): 2.4/5 GHz, 600 Mbps (MIMO)
- 802.11ac (WiFi 5): 5 GHz, 3.47 Gbps (MU-MIMO)
- 802.11ax (WiFi 6): 2.4/5/6 GHz, 14 Gbps (OFDMA)
- 802.11 beacon interval: ∼100 ms
- LTE synch signal: every 5 ms
- LTE light sleep: 100s of ms; deep sleep: 5–10 s
- LTE IMSI: 64 bits on SIM card
- 4G peak: ∼1 Gbps; 5G peak: ∼20 Gbps

TTM4100 – KTN

Chapter 8: Security

Computer Networking: A Top-Down Approach, 8th ed.
Kurose & Ross – NTNU Spring 2026

58 What Is Network Security?

Four Desirable Properties of Secure Communication

1. **Confidentiality** – only sender and intended receiver should understand message contents (message encrypted)
2. **Authentication** – sender and receiver can confirm each other's identity
3. **Message integrity** – message not altered in transit (without detection)
4. **Access and availability** – services must be accessible and available to authorised users (operational security)

58.1 The Language of Cryptography

The standard model uses three actors:

- **Alice** – legitimate sender
- **Bob** – legitimate receiver
- **Trudy** – the intruder

What Trudy Can Do

- **Eavesdrop** – intercept/record messages (sniff control and data messages)
- **Insert** – insert messages into the connection
- **Impersonate** – fake source address in packet (IP spoofing)
- **Hijack** – take over an ongoing connection by removing sender or receiver
- **Denial of Service** – prevent service from being used (e.g., SYN flooding)
- **Modify or delete** – alter or remove message content

Cryptography Notation

- Plaintext message: m
- Encryption algorithm: $K_A(\cdot)$ using key K_A
- Ciphertext: $c = K_A(m)$
- Decryption: $m = K_B(K_A(m))$
- **Symmetric key:** $K_A = K_B$ (same shared secret key)
- **Public key pair:** K_B^+ (Bob's public key), K_B^- (Bob's private key)
- Key property: $K_B^-(K_B^+(m)) = m$ and $K_B^+(K_B^-(m)) = m$

59 Symmetric Key Cryptography

Both parties share the same secret key. Key problem: how do two parties agree on a key in the first place?

59.1 Substitution Ciphers

Caesar Cipher – Simple Substitution

- Replace each letter by a letter k positions later in alphabet (wrapping around)
- Encode: shift letter by $+k$; Decode: shift by $-k$
- Example ($k = 3$): "abc" $\rightarrow$ "def"; "xyz" $\rightarrow$ "abc"
- Only 25 possible keys – trivially brute-forced
- **Monoalphabetic cipher:** substitute letter for letter (26! possible mappings).

Breakable by frequency analysis

Caesar Cipher Example – k

Encode "Protect" with $k = 7$:
P→W, r→y, o→v, t→a, e→l, c→j, t→a ⇒ `Wyvaljapvu`

Decode "Jhlzhy" with $k = 7$ (subtract 7):
J→C, h→a, l→e, z→s, h→a, y→r ⇒ `Caesar`

59.2 DES – Data Encryption Standard

DES

- 56-bit key, 64-bit block encryption
- 16 rounds of permutation/substitution using subkeys derived from main key
- **CBC – Cipher Block Chaining:** each plaintext block XOR-ed with previous ciphertext block before encryption; prevents identical plaintext blocks producing identical ciphertext
- **Security:** Brute-forceable in under 1 day with today's hardware (too weak)
- **3DES:** encrypt with K_1, decrypt with K_2, encrypt with K_3 – effectively 168-bit key

59.3 AES – Advanced Encryption Standard

AES

- 128-bit blocks; key length: 128, 192, or 256 bits
- At 1 billion DES keys/second: 2^{128} AES combinations ≈ **149 trillion years** to brute-force
- Replaced DES as US standard (NIST 2001)
- Used in TLS, 802.11 WPA2/WPA3, 4G LTE (AES on the radio link)

Symmetric Key Problem

n people wanting to communicate privately with each other all require **unique pairwise keys**:

$$\text{Number of keys} = \frac{n(n-1)}{2}$$

Example: 10 people need $\frac{10\times 9}{2} = 45$ keys. Scales poorly for large groups.

60 Public Key Cryptography

Public Key Cryptography – Key Idea

- Everyone has a **public key** K^+ (known to all) and a **private key** K^- (secret)
- Anyone can encrypt with public key; only owner can decrypt with private key
- **Solves** the symmetric key distribution problem – no shared secret needed upfront
- Mathematical requirement: $K^-(K^+(m)) = m$ and $K^+(K^-(m)) = m$
- Computationally infeasible to derive K^- from K^+

Key Difference – Symmetric vs. Public Key

Property	Symmetric	Public Key
Keys needed	$n(n-1)/2$	$2n$ – one pair per person
Key distribution	Requires secure channel	Public key openly shared
Speed	Very fast	Computationally expensive
Use in practice	Bulk data encryption	Key exchange, signatures

60.1 RSA – Rivest, Shamir, Adleman

RSA – How It Works

Key generation:

1. Choose two large primes p and q (1024 bits each)
2. Compute $n = p \cdot q$ and $z = (p-1)(q-1)$
3. Choose e such that $e < n$ and $\gcd(e, z) = 1$
4. Find d such that $ed \equiv 1 \pmod{z}$
5. Public key: (n, e); Private key: (n, d)

Encrypt: $c = m^e \bmod n$ **Decrypt:** $m = c^d \bmod n$

Security basis: factoring a large number n into $p \cdot q$ is computationally infeasible.

Important property: $K_B^-(K_B^+(m)) = m$ *and* $K_B^+(K_B^-(m)) = m$ – can apply in either order.

61 Authentication and Message Integrity

61.1 Message Integrity and Hash Functions

Cryptographic Hash Functions

A hash function $H(m)$ produces a fixed-length **message digest**. Properties required:

- Many-to-one: maps arbitrary-length m to fixed-length digest
- **Computationally infeasible** to find two messages with the same hash
- **Computationally infeasible** to find m given $H(m)$
- A small change in m produces a completely different hash

Common hash algorithms:

- **MD5:** 128-bit digest (RFC 1321) – now considered weak
- **SHA-1:** 160-bit digest – widely used
- **SHA-256:** 256-bit digest – current standard

Note: Internet checksum is NOT a cryptographic hash – too easy to find collisions. Hash is strictly better than checksum for integrity.

61.2 Message Authentication Code – HMAC

HMAC – Hash-based Message Authentication Code

Problem: A plain hash $H(m)$ doesn't prove who created the message – Trudy can modify m and recompute $H(m)$.

Solution – HMAC: combine message with a shared secret s:

$$\text{MAC} = H(m + s)$$

- Only parties who know secret s can compute or verify the MAC
- Receiver checks: recompute $H(m + s)$ and compare with received MAC
- Used in TLS, 802.11 WPA2, 4G LTE

61.3 Digital Signatures

Digital Signatures

Goal: verifiable, non-repudiable – Bob can prove to others that Alice sent this exact message.

How Bob signs message m:

1. Compute hash: $H(m)$
2. Encrypt hash with his private key: $K_B^-(H(m))$ – this is the **digital signature**
3. Send m and $K_B^-(H(m))$ together

Verification by receiver:

1. Decrypt signature with Bob's public key: $K_B^+(K_B^-(H(m))) = H(m)$
2. Independently compute $H(m)$ from the received message
3. If they match $\Rightarrow$ message is authentic and unaltered

Properties:

- **Non-repudiation:** only Bob could have produced $K_B^-(H(m))$
- Protects against both alteration and false authorship claims

61.4 Certification Authorities – CAs

Public Key Infrastructure and CAs

Problem: How do you know a public key really belongs to who claims it?

Solution: Certification Authority (CA) binds a public key to a specific entity.

1. Entity provides their public key to CA along with proof of identity
2. CA creates a **certificate** containing the entity's public key, signed with CA's private key
3. Anyone who trusts the CA can verify the certificate using the CA's public key

Browser trust: Web browsers come pre-installed with certificates from well-known CAs (e.g., VeriSign, Comodo). This forms the **chain of trust**.

Certificate contents: entity name, public key, valid date range, CA name, CA digital signature

62 Secure E-mail

Secure E-mail – All Three Goals

Secure e-mail combines public key, symmetric key, and hashing to provide confidentiality, authentication, and integrity.

Alice wants to send confidential, authenticated, integrity-protected message to Bob:

1. Generate random **symmetric session key** K_S
2. Encrypt message: $K_S(m)$ – fast symmetric encryption for the bulk data
3. Encrypt session key with Bob's public key: $K_B^+(K_S)$ – only Bob can recover K_S
4. Compute hash $H(m)$ and sign with Alice's private key: $K_A^-(H(m))$ – authentication + integrity
5. Send: $K_S(m)$, $K_B^+(K_S)$, $K_A^-(H(m))$

Three keys used: K_S (session), K_B^+ (Bob's public), K_A^- (Alice's private)

Bob's decryption:

1. Use K_B^- to decrypt $K_B^+(K_S) \Rightarrow$ recover K_S
2. Use K_S to decrypt $K_S(m) \Rightarrow$ recover m
3. Use Alice's K_A^+ to verify signature $\Rightarrow$ confirm integrity and authenticity

63 TLS – Transport Layer Security

TLS Overview

- **HTTPS** = HTTP over TLS; port **443**
- TLS sits between the application layer and TCP – a "sublayer" of the transport layer
- Provides: **confidentiality** via symmetric encryption, **integrity** via MAC/hashing, **authentication** via certificates (public key)
- Successor to SSL (Secure Sockets Layer)

63.1 TLS Handshake

TLS Handshake Steps

1. **TCP 3-way handshake** – establish TCP connection
2. **TLS Hello** – client sends list of supported cipher suites and a **nonce** (random number)
3. **Server responds** with: chosen cipher suite, server **certificate** (containing public key), server nonce
4. **Client verifies** certificate against trusted CAs
5. **Client sends** Pre-Master Secret (PMS) encrypted with server's public key: $K_S^+(PMS)$
6. Both sides independently derive the same **4 session keys** from PMS and nonces:
   - E_A – client-to-server encryption key
   - M_A – client-to-server MAC key
   - E_B – server-to-client encryption key
   - M_B – server-to-client MAC key
7. **Encrypted Handshake Message** – each side sends MAC of entire handshake to confirm agreement

63.2 TLS Records and TLS 1.3

TLS Records

Each TLS record contains:

- Type, version, length fields
- Encrypted data
- **MAC** (Message Authentication Code) – computed over data + sequence number, using key M_A or M_B

Sequence numbers prevent **replay attacks** – an attacker cannot re-send an old TLS record.

TLS 1.3 Changes

TLS 1.3 (current version) tightens the cipher suite to only 5 choices:

- Requires **Diffie-Hellman** for key exchange (provides forward secrecy)
- Requires **HMAC-SHA** for integrity
- Removes weak options (RSA key exchange, CBC-mode ciphers, MD5)
- Faster handshake (1-RTT instead of 2-RTT)

64 Network Layer Security – IPsec and VPNs

64.1 VPN – Virtual Private Network

VPN

- Institution's traffic sent over the public Internet but **encrypted** end-to-end
- Creates an encrypted "tunnel" so external traffic appears as normal IP packets
- Implemented using **IPsec** at the network layer
- Remote employees can securely access institutional resources over the public Internet

64.2 IPsec

IPsec Services

IPsec provides network-layer security with four services:

1. **Data integrity** – receiver can detect any modification of datagram in transit
2. **Origin authentication** – receiver can verify source of IPsec datagram
3. **Replay attack prevention** – sequence numbers prevent old packets being replayed
4. **Confidentiality** – payload encrypted (in ESP mode)

IPsec Modes and Protocols

Transport mode: only the IP payload is encrypted/authenticated; original IP header untouched
Tunnel mode: entire IP datagram is encrypted + authenticated and encapsulated in a new IP datagram (used in VPNs)

Two IPsec protocols:

- **AH – Authentication Header:** provides integrity and authentication but *no* confidentiality
- **ESP – Encapsulating Security Payload:** provides integrity, authentication, *and* confidentiality – far more commonly used

65 Security in Wireless and Mobile Networks

65.1 802.11 WiFi Security – WPA2/WPA3

802.11 Security – 4-Step Process

1. **Discovery:** AP advertises its presence and security capabilities (encryption/authentication suites)
2. **Mutual authentication and key derivation:** using WPA2/WPA3 and an Authentication Server (AS); shared master key established; uses **nonces** (random numbers) to generate session key
3. **Shared session key distribution:** pairwise transient key derived from master key; AP installs key for communicating with this device
4. **Encrypted communication:** data encrypted using the derived session key (AES-based in WPA2)

WPA2 Key Derivation Using Nonces

1. AS sends nonce N_{AS} to mobile
2. Mobile sends nonce N_M to AS
3. Both sides derive the same **Pairwise Master Key** from shared master key + nonces using HMAC
4. Session key derived from PMK – protects mobile-to-AP communication

Nonces prevent replay attacks: even if Trudy captures the exchange, she cannot reuse it.

65.2 4G LTE Authentication and Encryption

4G LTE Authentication – 5 Steps

Actors: Mobile device (SIM with shared key K_{HSS-M}), Base Station (BS), Mobility Management Entity (MME), Home Subscriber Service (HSS)

a **Mobile attaches** – sends IMSI (identity) to BS → MME; MME sends `AUTH_REQ(IMSI, VN info)` to HSS
b **HSS responds** – HSS derives authentication token + expected response $xres_{HSS}$ from K_{HSS-M}; sends `AUTH_RESP(auth_token, xres_HSS, keys)` back to MME
c **Mobile authenticates network** – mobile receives auth token, verifies it using its K_{HSS-M} (proves HSS is legitimate); mobile computes res_M using same crypto as HSS
d **Network authenticates mobile** – mobile sends res_M to MME; MME compares res_M with $xres_{HSS}$; if match ⇒ mobile authenticated; MME generates keys for BS
e **Key derivation** – mobile and BS determine session keys for encrypting data and control frames; **AES** can be used on the 4G radio link

Note: MME + HSS together play the role of the Authentication Server (analogous to WiFi AS).

4G LTE Key Insight

The SIM card stores the shared secret K_{HSS-M} which is also stored at the HSS. Authentication is **mutual**: the mobile verifies the network (prevents fake BS attacks), and the network verifies the mobile. The session key K_{BS-M} used for radio link encryption is derived fresh each session.

66 Operational Security – Firewalls

Firewall Definition

A **firewall** isolates an organisation's internal network from the larger Internet, allowing some packets to pass and blocking others. It sits between the trusted internal network and the untrusted public Internet.

66.1 Three Goals of Firewalls

Why Firewalls – Three Goals

1. **Prevent denial-of-service attacks** – e.g., SYN flooding: attacker establishes many bogus TCP connections, consuming resources so no capacity remains for legitimate connections
2. **Prevent illegal modification/access of internal data** – e.g., attacker replaces a web page with malicious content
3. **Allow only authorised access to inside network** – authenticate users/hosts; block all other access

Three types of firewalls: stateless packet filters, stateful packet filters, application gateways

66.2 Stateless Packet Filtering

Stateless Packet Filtering

- Internal network connected to Internet via a **router firewall**
- Filters **packet-by-packet**; decision to forward or drop based on:
  - Source/destination IP address
  - TCP/UDP source and destination port numbers
  - ICMP message type
  - TCP SYN and ACK bits
- Uses an **ACL – Access Control List**: table of (action, condition) pairs applied top to bottom

ACL Examples – Policy to Firewall Setting

Policy	Firewall Setting
No outside Web access	Drop all outgoing packets to any IP, port 80
No incoming TCP connections except to institution's public Web server	Drop all incoming TCP SYN to any IP except 130.207.244.203, port 80
Prevent Web-radios eating bandwidth	Drop all incoming UDP – except DNS and router broadcasts
Prevent smurf DoS attacks	Drop all ICMP to broadcast addresses
Prevent traceroute scanning	Drop all outgoing ICMP TTL-expired traffic
Block all incoming UDP + telnet	Block protocol=17, port=23; also block ACK=0 inbound TCP

Stateless Packet Filter Weakness

A stateless filter is a "heavy-handed tool" – it can admit packets that **make no sense** given the conversation state. For example, allowing inbound TCP ACK packets even when no TCP connection to that destination was ever initiated. An attacker can craft

such packets to exploit this.

66.3 Stateful Packet Filtering

Stateful Packet Filtering

- Tracks the **state of every TCP connection** – SYN, SYN-ACK, ACK, FIN
- Determines whether incoming/outgoing packets "**make sense**" given connection state
- ACL augmented with a "check connection state table" column
- Timeouts inactive connections – stops admitting packets after connection ends

Example: A packet with ACK=1, destination port=80 is only admitted if there is an established TCP connection to port 80 in the state table. Without state tracking, this packet would pass even if no connection exists.

66.4 Application Gateways

Application Gateways

- Filter packets on **application-layer data** as well as IP/TCP/UDP fields
- Example: allow only selected internal users to telnet outside:
  1. All telnet users must connect to the gateway
  2. Gateway authenticates user; if authorised, sets up second telnet connection to destination
  3. Gateway relays data between the two connections
  4. Router filter blocks all telnet connections not originating from gateway
- Each application requires its own gateway (HTTP gateway, FTP gateway, etc.)

66.5 Limitations of Firewalls and Gateways

Firewall Limitations

- **IP spoofing:** router cannot verify data "really" comes from claimed source
- **Multiple app gateways needed:** each application requiring special treatment needs its own gateway
- **Client software must know gateway address** (e.g., proxy settings in browser)
- **UDP all-or-nothing:** filters often use all-or-nothing policy for UDP traffic
- **Tradeoff:** degree of communication with outside world vs. level of security
- **Not foolproof:** many highly protected sites still suffer attacks

67 Summary – Where Security Is Applied

Security Techniques Applied Across Layers

Scenario	Layer	Mechanisms Used
Secure e-mail	Application	Public key + symmetric + hash
HTTPS / TLS	Transport	Symmetric key + hash + certificates
VPN	Network	IPsec – AH or ESP
IPsec	Network	AH or ESP (transport or tunnel mode)
802.11 WiFi	Link	WPA2/WPA3, HMAC, AES
4G LTE	Link/Physical	HMAC, AES over radio link
Firewalls	Network	Packet filtering, ACLs, state tables

68 Exam Questions

V23 Q1.5.1 – Properties of Secure Communication

Which of the following are desirable properties of secure communication?

a) Network reliability
b) **Confidentiality** ✓
c) Message integrity ✓
d) Authentication ✓
e) Operational security ✓
f) High bandwidth to transmit quickly

Answer: b, c, d, e. The four properties are confidentiality, authentication, message integrity, and operational security. Network reliability and bandwidth are not security properties.

V23/V24 Q1.5.2 – Number of Symmetric Keys

N people want to communicate privately using symmetric key encryption. How many keys are needed?

a) $N \times N$
b) $2N - 1$
c) $N(N-1)$
d) $\mathbf{N(N-1)/2}$ ✓
e) None of the above

Answer: d. Each pair of communicating parties needs a unique shared key. The number of unique pairs from N people is $\binom{N}{2} = \frac{N(N-1)}{2}$.
Example: 10 people: $\frac{10\times 9}{2} = 45$ keys.

V23/V24 Q1.5.3 – Message Integrity

For message integrity, which statements are correct?

a) Message integrity means confirming the identity of the sender
b) Message integrity means the receiver can detect whether the message was altered in transit ✓
c) Both checksumming and hashing techniques may be used ✓

d) **Generally, a hash provides a better integrity check than a checksum ✓**
e) To ensure message integrity, TCP must be used

Answer: b, c, d. Option a describes authentication, not integrity. Option e is false – integrity can be ensured at any layer and does not require TCP.

V24 Q1.5.4 – HTTP Streaming vs. UDP Streaming

HTTP streaming over TCP is more popular than UDP streaming. Major reasons include:

a) UDP is connectionless
b) **UDP lacks retransmission, ordering, and error-checking – results in higher error rate ✓**
c) UDP streaming has lower latency due to no handshakes
d) **Many firewalls block UDP traffic but allow HTTP traffic ✓**
e) None of the above

Answer: b, d. HTTP/TCP streaming benefits from reliability and passes through firewalls easily. Although UDP has lower latency, this is not a reason for HTTP being more popular; it is a reason in UDP's favour.

V24 Q1.5.5 – What Trudy Can Do – Alice-Bob Model

Trudy is an intruder in the Alice-Bob model. Which actions can she take?

a) **Sniffing and recording control messages on the channel ✓**
b) **Recording data messages on the channel ✓**
c) **Modifying or insertion of messages ✓**
d) **Deletion of message or message content ✓**
e) None of the above

Answer: a, b, c, d. Trudy has full access to the channel and can eavesdrop, record, modify, insert, and delete. Security mechanisms must protect against all of these threats.

V24 Q3 – SSL/TLS Wireshark Analysis

Q3.1 Was Wireshark packet 112 sent by the client or server?
Packet 112 source is 128.238.38.162 with destination port `https` – this is the **client** sending to the server.
Q3.2 What is the server's IP address and port number?
Server IP: **216.75.194.220**, port: **443** (https)
Q3.3 What will be the sequence number of the next TCP segment sent by the client?
Current segment: Seq=79, Len=204. Next sequence number = $79 + 204 =$ **283**
Q3.4 How many SSL records does packet 112 contain?
Packet 112 contains **3** SSL records: Client Key Exchange, Change Cipher Spec, Encrypted Handshake Message.
Q3.5 Does packet 112 contain a Master Secret or Encrypted Master Secret or neither?
Packet 112 contains a **Pre-Master Secret encrypted with the server's public key** ("Client Key Exchange"). The actual Master Secret is derived from the PMS – it is

not transmitted. So the answer is **neither** (neither a plaintext Master Secret nor the Encrypted Master Secret).

S2025 Q14 – Primary Purpose of a Firewall

What is the primary purpose of a firewall in network security?

a) To encrypt data
b) To block unauthorised access ✓
c) To manage network traffic
d) To provide VPN services
e) To monitor network performance

Answer: b. A firewall's primary purpose is to isolate the internal network and block unauthorised access, not to encrypt data (that is TLS/IPsec) or provide VPN services.

S2025 Q5 – Caesar Cipher – Symmetric Key Cryptography

Q5.3 Describe how the Caesar cipher works:
Caesar cipher shifts each letter in the plaintext by a fixed offset k positions forward in the alphabet (wrapping from Z back to A). Encryption: shift $+k$. Decryption: shift $-k$.
Q5.4 Encode "Protect your information" with $k = 7$:
P→W, r→y, o→v, t→a, e→l, c→j, t→a, (space), y→f, o→v, u→b, r→y, (space), i→p, n→u, f→m, o→v, r→y, m→t, a→h, t→a, i→p, o→v, n→u
Result: `Wyvaljaw fvby pumvythapvu`
Q5.5 Decode "Jhlzhy jpwoly" with $k = 7$ (subtract 7):
J→C, h→a, l→e, z→s, h→a, y→r, (space), j→c, p→i, w→p, o→h, l→e, y→r
Result: `Caesar cipher`

S2025 Q6 – Symmetric vs. Public Key Systems

Key difference between symmetric key and public key cryptography:

Symmetric Key	Public Key
Both parties share the **same secret key**	Each party has a **public/private key pair**
Key must be secretly distributed in advance	Public key is openly shared; private key secret
$N(N-1)/2$ keys for N users	Only $2N$ keys for N users
Fast – used for bulk data (AES, DES)	Slow – used for key exchange, signatures (RSA)
Symmetric: $K_A = K_B$	$K^+(K^-(m)) = m$ and $K^-(K^+(m)) = m$

In practice: combine both – use public key to securely exchange a symmetric session key, then use symmetric encryption for data (hybrid approach, as in TLS and secure e-mail).

V23 Q6 – Firewalls – Three Goals and Types

Three goals of a firewall:

1. Prevent denial-of-service attacks (e.g., SYN flooding)
2. Prevent illegal modification or access of internal data

3. Allow only authorised access to the inside network

Stateless vs. stateful packet filtering:

- **Stateless:** examines each packet independently based on header fields (src/dst IP, ports, flags); cannot detect "nonsense" packets that violate connection state
- **Stateful:** tracks the state of every TCP connection; only admits packets that make sense in context of existing connections; times out inactive connections; more secure but requires state storage

Application gateway: filters on application-layer content in addition to IP/TCP/UDP fields; most fine-grained control; requires separate gateway per application.

TTM4100 – KTN

Chapter 9: Multimedia Networking

Computer Networking: A Top-Down Approach, 8th ed.
Kurose & Ross – NTNU Spring 2026

69 9.1 – Multimedia Applications and Properties

Audio – Digitization

Analog-to-digital conversion (sampling):

- Analog signal sampled at a fixed frequency
- Each sample quantized to one of 2^n values, encoded with n bits

Typical parameters:

Use case	Sampling rate	Bit rate
Telephony	8 000 samples/sec	64 kbps (8 bit/sample)
CD audio	44 100 samples/sec	1.4 Mbps (16 bit, stereo)

Example: Telephony: $8\,000 \times 8\,\text{bit} = 64\,000\,\text{bps} = 64\,\text{kbps}$

Video – Digitization and compression

Basics:

- Video = sequence of images (frames), typically 24 frames/second
- Each image = array of pixels, each pixel encoded with bits (luma + chroma)

Compression techniques:

- **Spatial coding** (intra-frame): redundancy within a single image – send one example pixel + repeat count instead of all pixels
- **Temporal coding** (inter-frame): redundancy between frames – send the difference from the previous frame (changes only)

CBR vs. VBR:

- **CBR** (Constant Bit Rate): fixed encoding rate (simpler for the network)
- **VBR** (Variable Bit Rate): varies with content complexity (better quality)

Typical bit rates:

Format	Bit rate
MPEG-1 (CD-ROM)	~1.5 Mbps
MPEG-2 (DVD)	3–6 Mbps
MPEG-4	<1 Mbps

Three types of multimedia applications

1. **Streaming stored audio/video**
   - Pre-recorded content loaded from server
   - Can pause, rewind, skip (VCR functionality)
   - Examples: YouTube, Netflix
   - Delay tolerance: seconds (buffered)
2. **Conversational VoIP**
   - Two-way, real-time conversation
   - Very delay-sensitive: < 150 ms ideal, > 400 ms unacceptable
   - Examples: Skype, Teams, telephony over IP
3. **Streaming live audio/video**
   - Live broadcast to many receivers

- More tolerant of delay than VoIP, but still time-sensitive
- Examples: live TV, sports, radio

70 9.2 – Streaming Stored Video

Continuous Playout Constraint

Requirement: Playback must occur at the original recording rate – once started, frames must be displayed with correct timing.

The problem – network jitter:
- Packets arrive with varying delay (jitter)
- Without buffer: playback stutters
- Solution: **client-side buffer** – collects packets and plays back with a planned delay

Playout delay (initial buffering):
- Client waits before starting playback until enough data is buffered
- Trade-off: larger buffer = more robust against jitter, but longer wait time

Buffering and playback rate

Let x = network rate into buffer, r = playback rate (video bit rate):

State	Condition	Result
Buffer drains	$x < r$	Freeze/stall – buffer runs empty
Buffer fills	$x > r$	OK – buffer accumulates more data
Steady state	$x = r$	Smooth playback without freezing

UDP Streaming

Principle:
- Server sends video at a constant rate matched to client capacity
- UDP: no re-transmission, no congestion control
- RTP used to structure the stream
- Short playout delay: typically 2–5 seconds

Disadvantages of UDP streaming:
- Can be blocked by firewalls (UDP is often blocked)
- No guarantee of delivery
- Difficult to scale (server load)

HTTP Streaming – the dominant method

Principle:
- Video stored as a regular file on an HTTP server
- Client uses HTTP GET to fetch the file
- TCP controls the transfer rate (congestion control)
- Larger playout delay required (TCP rate variations)

Advantages of HTTP streaming:
- Passes through firewalls (port 80/443)

- Leverages existing HTTP infrastructure (CDN, caches)
- TCP re-transmission provides reliable delivery
- Easier to integrate with web applications

DASH – Dynamic Adaptive Streaming over HTTP:
- Video encoded at multiple quality levels (bit rates)
- Client selects the best possible quality based on available bandwidth
- Dynamic adaptation – switches quality during playback
- Example: Netflix, YouTube adaptive streaming

CDN – Content Distribution Networks

The problem: Video files are large; sending them from one central server is inefficient.

The CDN solution:
- Geographically distributed servers store copies of content
- User is directed to the nearest/best CDN node
- Reduces delay and load on the origin server
- Examples: Akamai, Cloudflare, Amazon CloudFront

Advantages:
- Scales to many simultaneous users
- Redundancy – if one node fails, others can take over
- Lower latency for end users

71 9.3 – Voice-over-IP – VoIP

VoIP – Characteristics and requirements

Delay requirements:
- < 150 ms: Good, noticeable delay
- 150–400 ms: Acceptable
- > 400 ms: Unacceptable for interactive speech

Speech characteristics:
- **Talk spurts** and **silence periods** – typically 50% of the time is silent
- During talk spurt: data generated at 64 kbps
- 20 ms per chunk: $64\,\text{kbps} \times 0.02\,\text{s} = 1280\,\text{bits} = 160\,\text{bytes}$ per packet
- Packet = 20 bytes RTP header + 8 bytes UDP header + 20 bytes IP header + 160 bytes data

Packet loss in VoIP

Two types of loss:
1. **Network loss**: packet dropped by router (queue overflow)
2. **Delay loss**: packet arrives too late to be played (after playout deadline)

Tolerance:
- VoIP typically tolerates 1–10% packet loss (depending on codec and FEC)
- Lower loss rate = better speech quality

Fixed Playout Delay

Principle:

- Client waits a fixed time q ms after sending before playback
- Packet with timestamp t is played at clock time $t + q$
- Packets arriving after $t + q$ are discarded (delay loss)

Trade-off:

- Large q: Fewer delay losses, but worse interactivity
- Small q: Better interactivity, but more delay losses

Adaptive Playout Delay – EWMA

Goal: Estimate network delay and jitter dynamically; adapt playout delay.

Definitions:

- t_i = timestamp of packet i (send time at sender)
- r_i = receive time of packet i (clock time at receiver)
- d_i = estimated network delay for packet i
- v_i = estimated jitter (deviation from average delay)
- α, β = smoothing factors (typically $\alpha = 0.1$, $\beta = 0.25$)

EWMA formulas:

$$d_i = (1 - \alpha)\, d_{i-1} + \alpha\,(r_i - t_i) \tag{17}$$

$$v_i = (1 - \beta)\, v_{i-1} + \beta\,|r_i - t_i - d_i| \tag{18}$$

Playout time for packet i:

$$p_i = t_i + d_i + K \cdot v_i$$

where K is a constant (typically 4) that determines how much jitter margin is added.

Talk spurt detection:

- If the difference between two timestamps $> 20\,\text{ms}$: the packet is the start of a new talk spurt
- At the start of a new talk spurt: adjust playout delay based on estimated d_i and v_i
- Adjustment happens only between talk spurts – not in the middle of a sentence

FEC – Forward Error Correction

Goal: Recover lost packets without re-transmission (for low latency).

Method 1 – Simple XOR redundancy:

- For each group of n packets, send one extra redundancy packet = XOR of all n packets
- If one packet is lost, it can be reconstructed from the $n-1$ others + the redundancy packet
- Overhead: $1/n$ extra bandwidth
- Works only if **one** packet per group is lost
- Introduces delay: must wait for the entire group

Method 2 – Piggyback low-quality copy:

- Append a low-quality copy of packet $n - 1$ to packet n
- Example: full PCM quality (64 kbps) + compressed GSM copy (13 kbps) from previous packet

- If packet $n-1$ is lost, the low-quality copy from packet n is used
- Loss of two consecutive packets produces an audible problem
- Low extra overhead, no additional delay beyond one packet

Method 3 – Interleaving:

- Split chunks and send them in mixed order
- Example: chunk 1 bits 1,5,9,...; chunk 2 bits 2,6,10,...
- Loss of one packet causes loss of scattered bits – easier to conceal
- No extra overhead, but introduces **more delay**

Skype – P2P VoIP Architecture

P2P architecture with supernodes:

- **Supernodes**: Skype clients with good bandwidth/availability – act as helper routers
- **Overlay network**: Logical network on top of the IP network
- Index of users and their IP addresses is distributed among the supernodes

NAT traversal:

- Many Skype clients are behind NAT – cannot be reached directly from outside
- Supernodes used as **relay**: packets sent via supernode to bypass NAT
- Enables VoIP between clients that otherwise cannot communicate directly

72 9.4.1 – RTP – Real-Time Transport Protocol

RTP – Overview

What is RTP?

- Specified in RFC 3550
- Defines packet structure for real-time audio and video
- Runs in end systems (not in network routers)
- Runs over **UDP** (not TCP)
- Applications: VoIP, video conferencing, streaming

What RTP provides:

- Payload type identification (which codec is used)
- Sequence numbering (detect loss and reordering)
- Timestamping (timing/synchronization of playback)
- Source identification (SSRC)

Important limitation:

- RTP does **not** provide QoS guarantees
- Routers see UDP – not RTP (RTP header is inside the UDP payload)
- No guarantee of delay, jitter, or loss in the network

RTP Header fields

Field	Size	Function
Payload type	7 bits	Identifies codec/format for audio data
Sequence number	16 bits	Increments by 1 per packet; detects loss and reordering
Timestamp	32 bits	Sampling time of the first byte in the packet; increments by 160 per packet for 8 kHz PCM audio
SSRC	32 bits	Synchronization source – identifies the source (sender)

RTP Payload Types – Common codecs

PT number	Codec	Bit rate	Comment
0	PCM mu-law	64 kbps	Telephony standard (North America/Japan)
3	GSM	13 kbps	Mobile telephony; good compression
7	LPC	2.4 kbps	Very low bit rate; lower quality
26	Motion JPEG	variable	Compressed video (JPEG per frame)
31	H.261	variable	Video conferencing (older standard)
33	MPEG-2 video	variable	DVD-quality video

Note: The payload type field is dynamic – applications can negotiate other types via SDP/SIP (not part of the curriculum).

RTP Timestamp – Calculation

Example – PCM audio at 8 kHz, 20 ms per packet:

- Sampling rate: 8 000 samples/sec
- Packet length: 20 ms = 0.02 s
- Samples per packet: $8\,000 \times 0.02 = 160$ samples
- Timestamp increments by **160** per packet (even though the packet is sent every 20 ms)

Timestamp is in units of sampling clock – not ms or seconds!

Common misconceptions about RTP

- **RTP ≠ QoS:** RTP does not guarantee low delay or low jitter. It is a packet structure, not a QoS mechanism.
- **RTP runs over UDP – not TCP:** TCP re-transmission and congestion control are too slow for real-time.
- **Routers do not see RTP:** The RTP header is hidden inside the UDP datagram.

The network treats it as ordinary UDP.
- **Sequence number ≠ guarantee:** Sequence numbers allow the receiver to *detect* loss, but not recover lost packets (that is FEC's job).

73 Summary – Key Concepts

Summary – Everything you need to know

9.1 – Multimedia properties:
- Audio: sampling 8 kHz telephony → 64 kbps; 44.1 kHz CD
- Video: 24 fps, spatial + temporal coding, CBR vs VBR
- 3 application types: stored streaming, VoIP, live streaming

9.2 – Stored streaming:
- Continuous playout constraint; client buffer against jitter
- If $x < r$: buffer drains (freezing); if $x > r$: OK
- HTTP streaming dominates (firewall-friendly, CDN, DASH)
- UDP streaming: lower latency, blocked by firewalls

9.3 – VoIP:
- Requirements: < 150 ms; 64 kbps during talk spurt, 160 bytes/20ms packet
- Loss: network loss + delay loss; tolerance 1–10%
- Fixed playout delay: trade-off loss vs. interactivity
- Adaptive EWMA: $d_i = (1 - \alpha)d_{i-1} + \alpha(r_i - t_i)$
- FEC: XOR redundancy, piggyback low quality, interleaving
- Skype: P2P with supernodes, overlay network, NAT relay

9.4.1 – RTP:
- RFC 3550; over UDP; in end systems, not routers
- Header: payload type (7b), seq# (16b), timestamp (32b), SSRC (32b)
- Timestamp increments by 160 per packet (8 kHz, 20 ms chunks)
- **Does not provide QoS**; routers only see UDP

74 Exam Questions

V24 – Q1.5.4 – HTTP vs UDP Streaming

Question: Multimedia streaming can be done using UDP or HTTP. Which of the following are reasons why HTTP streaming is more popular than UDP streaming?

Alternatives (select the correct ones):
a) HTTP streaming has lower delay (playout delay)
b) HTTP packets pass through firewalls; UDP is often blocked
c) UDP provides more reliable delivery than TCP
d) HTTP streaming leverages existing CDN infrastructure
e) UDP provides better video quality per kbps

Answer: b and d

Explanation:
- **b – Correct:** UDP is frequently blocked by corporate firewalls (port filtering). HTTP uses port 80/443 which is always open.
- **d – Correct:** HTTP content can be cached and distributed via CDN (Content

Delivery Networks), providing good scalability and low latency for users globally.

- **a – Wrong:** HTTP streaming requires *larger* playout delay because the TCP rate varies with congestion control.
- **c – Wrong:** TCP (used by HTTP) provides reliable delivery, not UDP.
- **e – Wrong:** UDP vs TCP does not affect the video encoding itself.

S2025 – Part I Q1 – CDN content delivery

Question: A CDN (Content Delivery Network) delivers content to users. What is the best answer for why CDN works effectively?

Alternative A: CDN stores copies of content on geographically distributed servers, and users are directed to the nearest server.

Answer: A

Explanation:

- CDN works by **replicating** content to many servers spread geographically
- The user is directed to their nearest CDN node (lowest RTT, fewest hops)
- Reduces load on the origin server and lowers delay
- This is the core concept behind services such as Netflix, YouTube, and Akamai

Typical exam topics – Ch. 9

Frequently tested concepts:

1. **VoIP delay requirements:** $< 150\,\text{ms}$ = good, $> 400\,\text{ms}$ = unacceptable
2. **Audio digitization:** $8\,000\,\text{samples/s} \times 8\,\text{bit} = 64\,\text{kbps}$ (telephony)
3. **Packet calculation:** 64 kbps, 20 ms chunk $\Rightarrow$ 160 bytes data per packet
4. **HTTP vs UDP streaming:** HTTP = firewall-friendly + CDN-compatible; UDP = lower latency
5. **EWMA formula:** $d_i = (1-\alpha)d_{i-1} + \alpha(r_i - t_i)$ – be able to explain the symbols
6. **FEC:** Be able to explain the XOR method and the piggyback method
7. **RTP header fields:** payload type (7 bit), seq# (16 bit), timestamp (32 bit, increments 160/packet), SSRC (32 bit)
8. **RTP does not provide QoS** – important pitfall
9. **CDN:** Geographic distribution of content for low latency and scalability
10. **Interleaving:** Reduces effect of burst loss, but increases delay

Curriculum boundary: 9.4.2 SIP is **not** in the curriculum – never answer SIP questions without explicit instruction.